ENME302 S2 2026

UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

Course Handbook - Volume II

Solving Partial Differential Equations

Computational and Applied Mechanical Analysis

Semester Two

Te Tari Pūhanga Pūrere | Mechanical Engineering

ENME302-26S2-HBK-VOL2

Te Kaupeka Pūhanga | Faculty of Engineering

ENME302: Computational and Applied Mechanical Analysis — Weeks 5–12

Solving Partial Differential Equations

Dr James Hewett

james.hewett@canterbury.ac.nz

Department of Mechanical Engineering

University of Canterbury

Semester 2, 2026

Contents

Chapter 1

Partial Differential Equations

1.1 Motivation

Most problems of engineering and physics involve variables which depend on space and time. For example:

- temperature variation in a room
- displacement in a deformed body
- velocity in a flowing fluid

The equation which governs the distribution and evolution of such variables usually takes the form of a *Partial Differential Equation* (PDE).

1.2 Reminder about PDEs

Given a function u that depends on both x and y, the partial derivative of u w.r.t. x is:

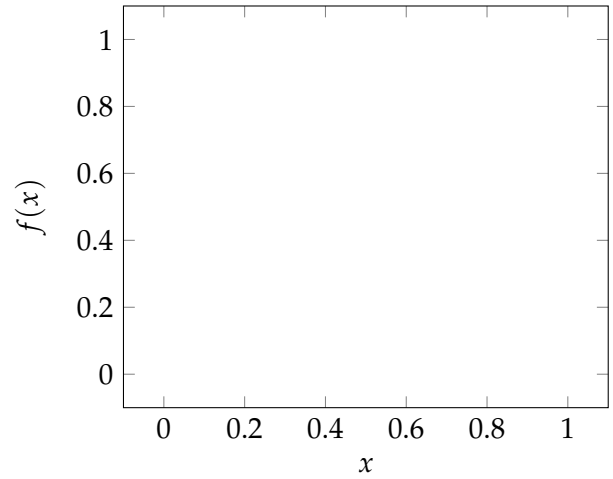
(1.1)

Similarly, the partial derivative of u w.r.t. y is:

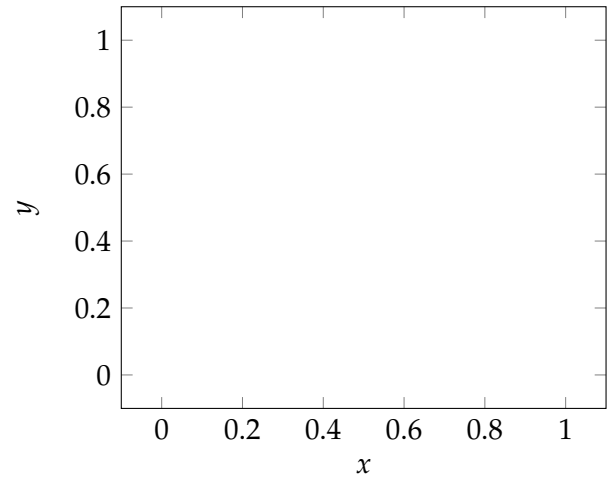
(1.2)

A PDE involves partial derivatives of an unknown function of two or more independent variables.

Function in 1-D



Isocontours in 2-D



Examples of PDEs

$$\frac{\partial^2 u}{\partial x^2} + 2xy\frac{\partial^2 u}{\partial y^2} + u = 1 \tag{1.3}$$

$$\frac{\partial^3 u}{\partial x^2 \partial y} + x\frac{\partial^2 u}{\partial y^2} + 8u = 5y \tag{1.4}$$

$$\left(\frac{\partial^2 u}{\partial x^2}\right)^3 + 6\frac{\partial^3 u}{\partial x \partial y^2} = x \tag{1.5}$$

$$\frac{\partial^2 u}{\partial x^2} + xu\frac{\partial u}{\partial y} = x \tag{1.6}$$

The variable u which we differentiate is called the *dependent* variable, whereas the ones we differentiate with respect to are called the *independent* variables. For example, it is clear from Equation 1.3 that the

dependent variable $u(x, y)$ is a function of two independent variables x and y.

Note that partial differentiation is sometimes denoted by using the independent variable with respect to which differentiation is taken as a subscript. For example,

(1.7)

1.3 How do we solve PDEs?

PDEs are difficult to solve but many different methods are available today to find solutions. Most techniques involve transforming the PDE into an *Ordinary Differential Equation* (ODE). Methods available for solving PDEs include the following.

- *Separation of variables*: this technique reduces a PDE of n independent variables into n ODEs.
- *Integral transforms*: such as the Laplace and Fourier transforms reduce the number of independent variables from n to $n - 1$; thus, a PDE of two variables can be reduced to an ODE.
- *Change of coordinate*: it is possible that by changing the coordinate system, the PDE becomes easier to solve. For example, a problem described by a PDE in a Cartesian coordinate system may be described by an ODE in a polar coordinate system.
- *Transformation of the dependent variable*: this method transforms the unknown of a PDE into a new unknown that is easier to find.
- *Numerical methods*: these methods rely on discretising the PDE, i.e. transforming the PDE which is continuous in time and space into a system of equations for discrete values of the unknown function at specific locations and times. This system of equations can then be solved on a computer. Often, this method is the only approach that will work.
- *Other methods*: perturbation methods, integral equations, etc...

We will focus mainly on numerical methods, and will also cover separation of variables to compare our numerical results with analytical solutions.

1.4 Some definitions

- A *solution* of a PDE in some region R of the space of the independent variables is a function for which all the partial derivatives appearing in the equation exist and which satisfies the equation everywhere in R. In general, the totality of solutions of a PDE is very large. For example, the functions:

  $$u = x^2 - y^2, \qquad u = e^x \cos y, \qquad u = \ln(x^2 + y^2)$$

  are all different but they are all solutions of:

  (1.8)

  The unique solution of a PDE corresponding to a given physical problem is obtained by the use of additional conditions (*boundary* conditions or *initial* conditions).

- The *order* of a PDE is that of the highest-order partial derivative in the equation (Equation 1.3 is 2nd and Equation 1.4 is 3rd order).

- A PDE is said to be *linear* if it is linear in the unknown function and all its derivatives, with coefficients depending only on the independent variables (x, y, t for example). In other words, the dependent variable u and all its derivatives appear in a linear fashion (they are not multiplied together or squared, for example). Equation 1.3 is linear but Equation 1.5 is not linear.

- A PDE is *homogeneous* if all terms contain the dependent variable or one of its derivatives.

1.5 Classifications of PDEs

Because they are prevalent in engineering and science, we will deal mostly with linear, second-order PDEs. For two independent variables, such PDEs take the following form:

$$A\frac{\partial^2 u}{\partial x^2} + B\frac{\partial^2 u}{\partial x \partial y} + C\frac{\partial^2 u}{\partial y^2} + D\frac{\partial u}{\partial x} + E\frac{\partial u}{\partial y} + Fu = G \tag{1.9}$$

where the coefficients A, B, C, D, E, F, and G may depend on x and y.

Depending on the values of A, B, and C, Equation 1.9 can be classified in one of the following categories: elliptic, parabolic, or hyperbolic.

This classification is useful because each category corresponds to dif-

ferent types of engineering problems, and different spatial solution techniques are required for each.

$B^2 - 4AC$	Category	Example
< 0	Elliptic	
$= 0$	Parabolic	
> 0	Hyperbolic	

Broadly speaking, the significance of the three categories are:

- *Elliptic problems* are characteristically steady boundary value problems. The solution at one point is influenced by conditions at all other points. The boundary of the solution domain is "closed" in the sense that boundary conditions are required on all bounding surfaces.
- *Parabolic problems* are characteristically diffusive. There is always a time (or time-like) coordinate and the solution domain generally has an open boundary in time (the "future"). The solution is defined by initial conditions and all past events at all points can influence the present and future state of the system.
- *Hyperbolic problems* are characteristically wavelike or propagative. As with parabolic problems, there is generally a time (or time-like) coordinate and an open boundary in time. The solution is determined by initial conditions but information propagates in space with a finite speed. Only a subset of past events influence the "present".

Notes:

- According to the definitions above, a PDE is homogeneous if $G(x, y)$ is identically zero for all x and y.

- If the coefficient A, B, C, D, E, F, and G are constant, the PDE is said to have *constant coefficients*.
- The PDE in Equation 1.9 was written in terms of the independent spatial variables x and y. In many cases, the PDE will involve the time t and therefore would be written in terms of x and t.

1.6 PDEs in engineering

Elliptic problems

- Steady-state heat transfer in a plate:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0 \quad \text{with} \quad T: \text{ temperature}$$

- Groundwater flow through an aquifer:

$$\frac{\partial^2 h}{\partial x^2} + \frac{\partial^2 h}{\partial y^2} = 0 \quad \text{with} \quad h: \text{ hydraulic head}$$

- Maxwell's equation describing electrostatic:

$$\frac{\partial^2 \varphi}{\partial x^2} + \frac{\partial^2 \varphi}{\partial y^2} = 0 \quad \text{with} \quad \varphi: \text{ electric potential}$$

Parabolic problems

- Transient heat conduction, e.g. heating/cooling of a long thin rod:

$$\rho c \frac{\partial T}{\partial t} = k \frac{\partial^2 T}{\partial x^2} \quad \text{with} \quad T: \text{ temperature}$$

- Fick's law to describe species diffusion:

$$\frac{\partial \varphi}{\partial t} = D \frac{\partial^2 \varphi}{\partial x^2} \quad \text{with} \quad \varphi: \text{ concentration}$$

Hyperbolic problem

- Wave equation:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$$

with u: vertical displacement and $c = \sqrt{T/\rho}$ wave propagation on a string (with T: tension).

When $c =$ speed of sound and $u = p$ (i.e. the acoustic pressure) $\Longrightarrow$ sound propagation.

1.7 Exercises

Question 1

The Laplace equation in 2-D is given by:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0 \tag{1.10}$$

(a) If $T_1(x, y)$ and $T_2(x, y)$ are solutions of Equation 1.10, show that

$$T = \alpha T_1 + \beta T_2$$

is also a solution, where α and β are constant.

(b) Show that this superposition principle no longer holds if Equation 1.10 is replaced by the Poisson equation

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = f(x, y) \tag{1.11}$$

Is the Poisson equation homogeneous?

(c) Verify that the following functions are solutions of the Laplace equation:

$$T = 2xy$$

$$T = x^3 - 3xy^2$$

Question 2

Are the following equations elliptic, parabolic, or hyperbolic?

(a) Laplace equation:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0 \tag{1.12}$$

(b) Heat equation:

$$\rho c \frac{\partial T}{\partial t} = k \frac{\partial^2 T}{\partial x^2} \tag{1.13}$$

(c) Wave equation:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2} \tag{1.14}$$

(d) Tricomi equation:

$$y \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0 \tag{1.15}$$

Question 3

(a) Probably the easiest of all PDEs to solve is the equation:

$$\frac{\partial u(x,y)}{\partial x} = 0 \tag{1.16}$$

Can you solve this equation? Find all functions $u(x,y)$ that satisfy this equation.

Hint: consider the general solution of the ODE $\dfrac{\mathrm{d}^2u}{\mathrm{d}x^2} - u = 0$

(b) Find a solution $u(x,y)$ of the PDE:

$$\frac{\partial^2 u}{\partial x^2} - u = 0 \tag{1.17}$$

Question 4

Show that $u(x,t) = v(x+ct) + w(x-ct)$ is a solution of the wave equation:

Hint: use a change of variables, e.g. $r = x + ct$*, then apply the chain rule.*

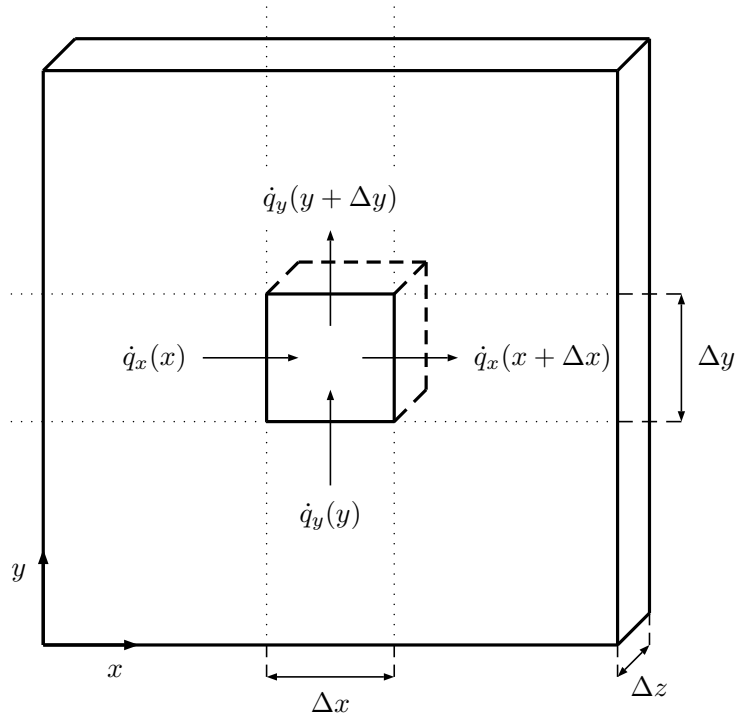
$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2} \tag{1.18}$$

Chapter 2

Elliptic PDEs: derivation and analytical method

2.1 Where do elliptic PDEs come from?

Often PDEs are obtained from integral statements of physical principles such as conservation of mass or energy. As an illustrative example, we will derive the steady heat equation for a thin plate of thickness Δz as shown in the figure below. The front and back face are insulated such that the problem only depends on x and y.



with $\vec{\dot{q}} = (\dot{q}_x, \dot{q}_y)$ is the heat flux vector in $\mathrm{W/m^2}$.

Consider the small element of size $\Delta x \Delta y \Delta z$ shown above. At steady state, the flow of heat into the element over Δt must equal the flow out:

or by dividing throughout by $\Delta x \Delta y \Delta z \Delta t$:

and in the limit of infinitesimal Δx, Δy:

(2.1)

Equation 2.1 is a PDE which at this stage involves two unknowns $\dot{q}_x$ and $\dot{q}_y$ which are the two components of the heat flux vector. The problem is not well-posed as we have more unknowns than equations. In order to close the system, we need to invoke Fourier's law which relates the heat flux to the temperature gradient according to:

(2.2)

where $\hat{\mathbf{i}}$ and $\hat{\mathbf{j}}$ are the unit vectors in the x and y direction, and k is the thermal conductivity in $\mathrm{W/m\,K}$. Fourier's law indicates that heat fluxes are related to the temperature gradient: the larger the temperature gradient, the larger the heat flux. Also the direction of the heat flux is such that heat travels from hot to cold.

Plugging Equation 2.2 into Equation 2.1 yields:

or if the thermal conductivity is constant:

(2.3)

As with Ordinary Differential Equations (ODEs), boundary conditions (b.c.) are needed to obtain a unique solution. The three main types of boundary conditions are:

- Prescribed temperature on the boundary $\implies$ *Dirichlet* boundary condition. For example:

(2.4)

- Prescribed heat flux on the boundary $\implies$ *Neumann* boundary condition. For example:

(2.5)

- Prescribed transfer coefficient on the boundary $\implies$ *Robin* boundary condition. For example:

(2.6)

  where h is the convective heat transfer coefficient.

2.2 Analytical methods

Analytical methods are rare and difficult for PDEs. However, they are very useful to check the validity of numerical solutions. If the PDE is linear, homogeneous, and the domain is simple enough, the method of separation of variables can sometimes be used. The general idea behind the method of separation of variables is as follows. Suppose we wish to solve:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0 \tag{2.7}$$

we seek a solution of the form:

$$u(x, y) = X(x)Y(y) \tag{2.8}$$

where X is a function of x only and Y is a function of y only.

Let's plug Equation 2.8 into Equation 2.7 and note that:

because Y is not a function of x, and

where the partial derivative is replaced by the full derivative because X is a function of x only. Likewise:

dividing throughout by XY and rearranging gives:

The "trick" of the method is to note that the left-hand side is a function of x only and the right-hand side a function of y only. Since they must be equal for *all* x and y, both sides must be equal to a constant λ.

We therefore obtain *two* ordinary differential equations:

(2.9)

The type of solution depends on the sign of λ and we have a variety of possible solutions:

$$\lambda = -\mu^2 < 0 :$$
$$u(x,y) = \big(A\sin(\mu x) + B\cos(\mu x)\big)\big(Ce^{\mu y} + De^{-\mu y}\big) \tag{2.10}$$
$$\lambda = \mu^2 > 0 :$$
$$u(x,y) = \big(Ae^{\mu x} + Be^{-\mu x}\big)\big(C\sin(\mu y) + D\cos(\mu y)\big) \tag{2.11}$$
$$\lambda = 0 :$$
$$u(x,y) = (Ax + B)(Cy + D) \tag{2.12}$$

By using the definitions of hyperbolic functions, it is sometimes useful to write the above Equations 2.10 and 2.11 as:

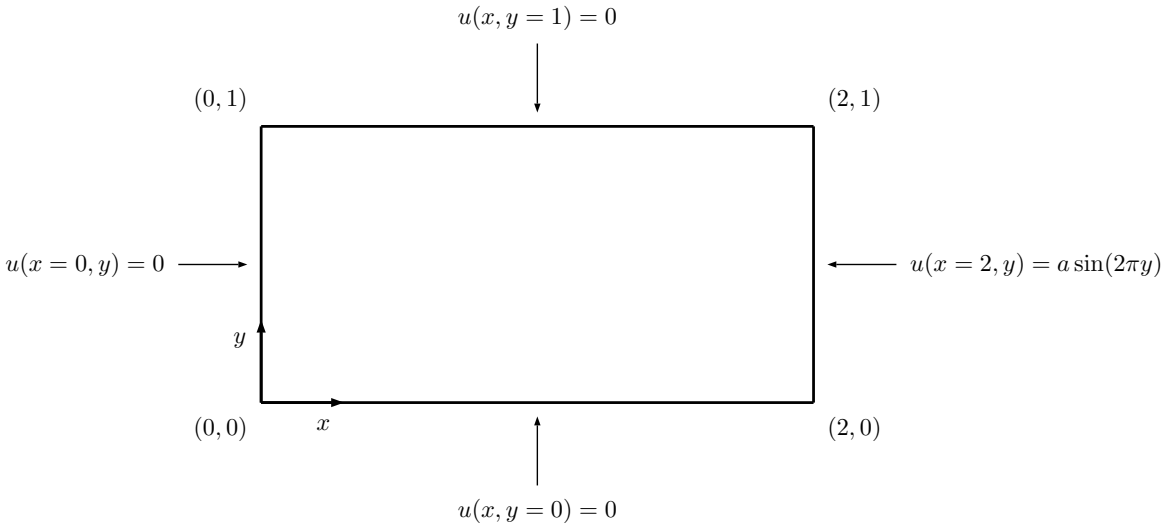
$$u(x,y) = \big(A\sin(\mu x) + B\cos(\mu x)\big)\big(C\cosh(\mu y) + D\sinh(\mu y)\big)$$
$$\text{and } u(x,y) = \big(A\sinh(\mu x) + B\cosh(\mu x)\big)\big(C\sin(\mu y) + D\cos(\mu y)\big)$$

2.3 Example 1

Use the method of separation of variables to solve:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0 \quad \text{on} \quad 0 \leqslant x \leqslant 2 \quad \text{and} \quad 0 \leqslant y \leqslant 1$$

subject to the following boundary conditions (b.c.):



First, show that $u(x, y) = (Ax + B)(Cy + D)$ is not a possible solution:

- b.c. $u(x, y = 0) = 0$ for all x

- b.c. $u(x = 0, y) = 0$ for all y

- b.c. $u(x, y = 1) = 0$ for all x

Second, show that $u(x,y) = \big(A\sin(\mu x) + B\cos(\mu x)\big)\big(Ce^{\mu y} + De^{-\mu y}\big)$ is also not a possible solution:

- b.c. $u(x, y = 0) = 0$

- b.c. $u(x = 0, y) = 0$

- b.c. $u(x, y = 1) = 0$

Finally, we choose $u(x,y) = \big(Ae^{\mu x} + Be^{-\mu x}\big)\big(C\sin(\mu y) + D\cos(\mu y)\big)$ as the solution type:

- b.c. $u(x, y = 0) = 0$

- b.c. $u(x, y = 1) = 0$

- b.c. $u(x = 0, y) = 0$

- b.c. $u(x = 2, y) = a \sin(2\pi y)$

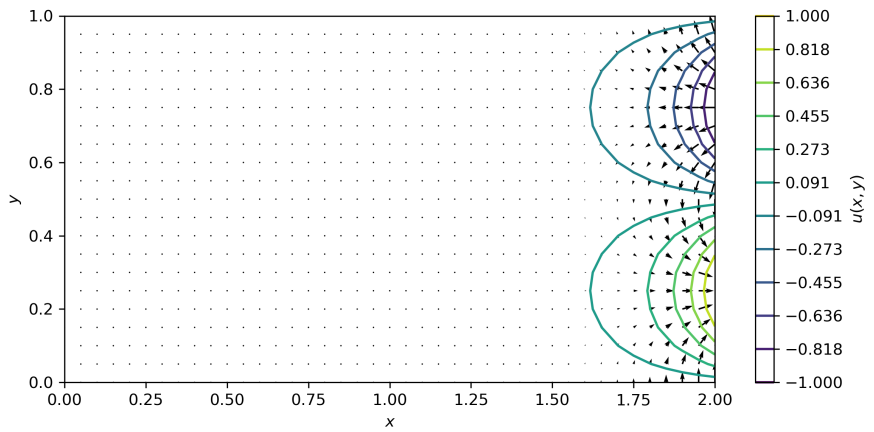
The final solution is:

Python code to plot the solution:

Python demonstration
Plotting Example 1

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm

# Parameters
Lx = 2. # size of domain in x-direction
Ly = 1. # size of domain in y-direction
a = 1. # coefficient of right hand boundary

# Grid
dx = 0.05 # space between grid points in x-direction
dy = dx # space between grid points in y-direction
Nx = int(Lx/dx)+1 # number of grid points in x-direction
Ny = int(Ly/dy)+1 # number of grid points in y-direction
x = np.linspace(0, Lx, Nx) # x-coordinate of grid points
y = np.linspace(0, Ly, Ny) # y-coordinate of grid points
u = np.zeros((Ny, Nx)) # PDE solution matrix
X, Y = np.meshgrid(x, y) # matrix of x and y coordinates

# Calculate the PDE solution of all x and y
for j in range(Ny):
    for i in range(Nx):
```

```
23          u[j,i] = a*np.sinh(2*np.pi*X[j,i])
                *np.sin(2*np.pi*Y[j,i])/np.sinh(4*np.pi)
24
25  # Plot the solution using a surface plot
26  fig1, ax1 = plt.subplots(subplot_kw={'projection':'3d'})
27  surf = ax1.plot_surface(X, Y, u, cmap=cm.viridis)
28  ax1.set_xlabel('$x$')
29  ax1.set_ylabel('$y$')
30  ax1.set_zlabel('$u(x,y)$')
31
32  # Plot the solution using a contour plot
33  clevels = np.linspace(np.min(u), np.max(u), 12)
34  fig2, ax2 = plt.subplots()
35  C = ax2.contour(X, Y, u, clevels) # note:
        ax2.contourf() for filled contours, and ax2.clabel()
        for labelling contours
36  cbar = fig2.colorbar(C)
37  cbar.ax.set_ylabel('$u(x,y)$')
38  cbar.ax.set_yticks(clevels)
39  ax2.set_xlabel('$x$')
40  ax2.set_ylabel('$y$')
41
42  # Calculate gradient of solution and add to contour plot
43  qy, qx = np.gradient(u, dx, dy) # calcs gradients
        across dim1 (rows: y) then dim2 (cols: x)
44  ax2.quiver(X, Y, qx, qy, scale=200)
```



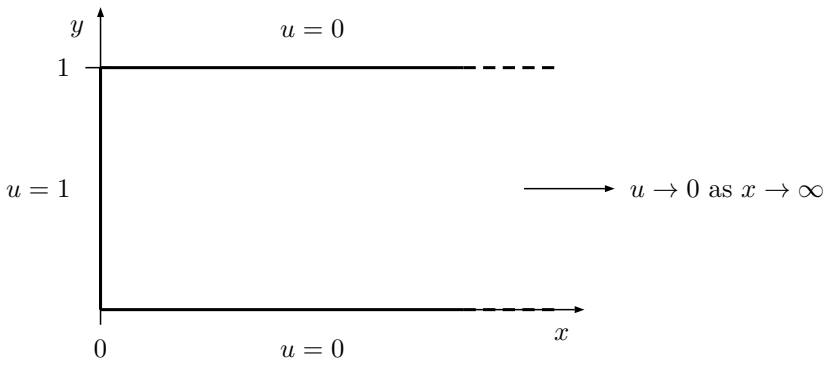
2.4 Example 2

Again, solve the Laplace equation:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0$$

using the separation of variables, but now in the semi-infinite region of $0 \leqslant y \leqslant 1$ and $x \geqslant 0$, and subject to boundary conditions:

$$u(x, y = 0) = 0 \quad (x \geqslant 0) \tag{a}$$
$$u(x, y = 1) = 0 \quad (x \geqslant 0) \tag{b}$$
$$u(x, y) \to 0 \text{ as } x \to \infty \tag{c}$$
$$u(x = 0, y) = 1 \tag{d}$$



The first and third solutions (Equations 2.10 and 2.12) are not good candidates because they cannot satisfy $u(x, y) \to 0$ as $x \to \infty$. Therefore, we choose the second solution $u(x, y) = \left(Ae^{\mu x} + Be^{-\mu x}\right)\left(C\sin(\mu y) + D\cos(\mu y)\right)$.

- b.c. $u(x, y) \to 0$ as $x \to \infty$

- b.c. $u(x, y = 0) = 0$

- b.c. $u(x, y = 1) = 0$

- b.c. $u(x = 0, y) = 1$
  To find a particular solution which also satisfies this b.c., we assume that the solution is a linear superposition of all possible u_n:

  and then to satisfy this b.c.:

  in order to find C'_n, we refer to a classic result of the Fourier series:

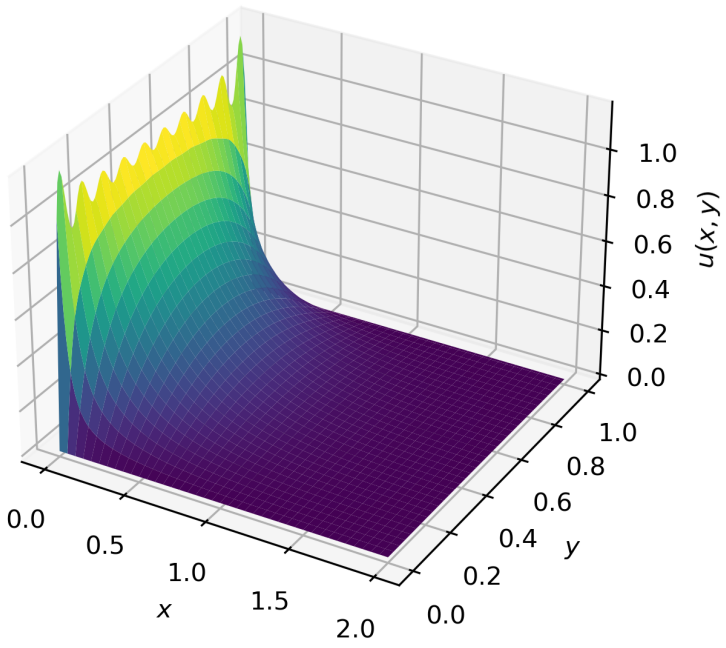
The final solution is:

Python demonstration
Plotting Example 2

Python code to plot the solution:

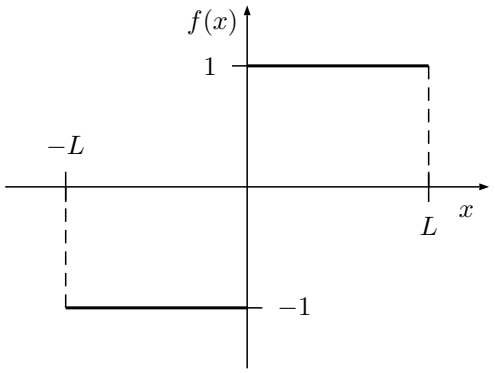
```
...

nmax = 20 # maximum number of terms in Fourier series

...

# Calculate the PDE solution of all x and y
for j in range(Ny):
    for i in range(Nx):
        n = 1 # index of a single harmonic
```

```
11          iterate = True # used as a stop condition
12          u[j,i] = 0 # initialise the Fourier series
13          while iterate:
14              # Calculate new term in Fourier series
15              if np.mod(n, 2) == 0: # n is even
16                  tmp = 0
17              else: # n is odd
18                  tmp = (4/n/np.pi)
                        *np.exp(-n*np.pi*X[j,i])
                        *np.sin(n*np.pi*Y[j,i])
19              u[j,i] += tmp # add new term
20              n += 1
21              if n > nmax:
22                  iterate = False # stop if n exceeds the
                        maximum number of terms allowed
23
24  ...
```



2.5 Calculating the Fourier sine series

$$f(x) = \sum_{n=1}^{\infty} b_n \sin\left(\frac{n\pi x}{L}\right) \quad \text{with} \quad b_n = \frac{2}{L}\int_0^L f(x)\sin\left(\frac{n\pi x}{L}\right)\mathrm{d}x$$



Example

Find the Fourier sine series for the function:

$$f(x) = 1 \text{ for } 0 \leqslant x \leqslant 1$$

Python demonstration
Fourier series

2.6 Exercises

Question 1

What are the Dirichlet and Neumann boundary conditions?

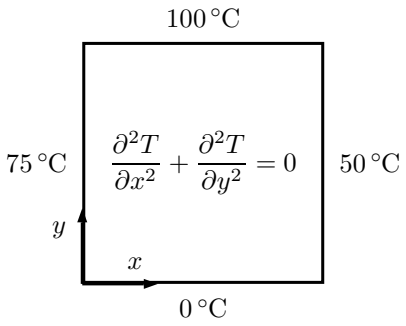
Question 2

A rectangular steel plate occupies the region $0 \leqslant x \leqslant L$, $0 \leqslant y \leqslant W$. The temperature on three sides of the plate ($x = 0$, $x = L$ and $y = 0$) is held at $0\,^{\circ}\mathrm{C}$. On the fourth side ($y = W$), the steel is held at a uniform temperature T_0. The temperature in the plate is governed by the Laplace equation in 2-D.

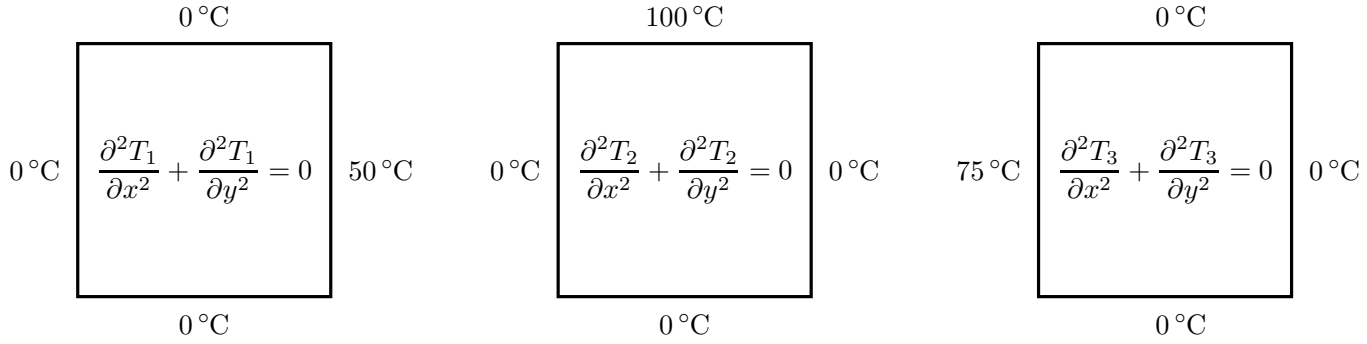
Obtain an expression for the temperature at all points in the plate using the method of separation of variables. Plot the solution and show the heat fluxes in Python, using $k = 43\,\mathrm{W/m\,K}$, $W = 1\,\mathrm{m}$, $H = 1\,\mathrm{m}$ and $T_0 = 100\,^{\circ}\mathrm{C}$. Calculate the total heat flux through the top boundary.

Question 3

Repeat Question 2, now with the boundary conditions:



Note: because the Laplace equation is linear, the best way to solve this problem is by adding the solutions of the three following subproblems:



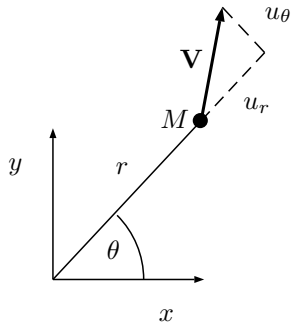
i.e. then finally $T(x, y) = T_1(x, y) + T_2(x, y) + T_3(x, y)$.

Question 4

Potential flow theory approximates the flow of an inviscid, irrotational and incompressible fluid. Such flows are governed by the Laplace equation such that in a polar coordinate system:

$$\frac{1}{r}\frac{\partial}{\partial r}\left(r\frac{\partial \varphi}{\partial r}\right) + \frac{1}{r^2}\frac{\partial^2 \varphi}{\partial \theta^2} = 0$$

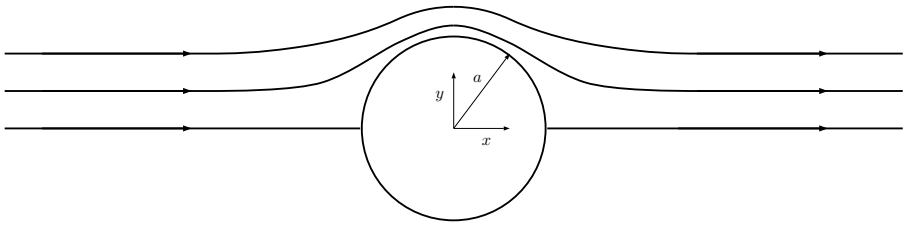
where φ is the stream function.



The stream function is related to the velocity components, in polar coordinates:

$$u_r = \frac{1}{r}\frac{\partial \varphi}{\partial \theta} \quad \text{and} \quad u_\theta = -\frac{\partial \varphi}{\partial r}$$

We wish to consider the flow past a cylinder of radius a. Far upstream (and from the cylinder) the velocity is uniform with a magnitude of U in the x-direction.



At the cylinder boundary the no-penetration boundary condition applies such that the velocity normal to the boundary is zero.

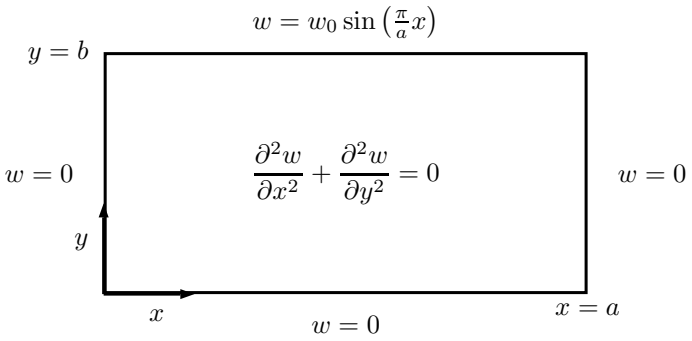
(a) Show that the following stream function is a good candidate for this flow:

$$\varphi(r, \theta) = U\left(r - \frac{a^2}{r}\right)\sin\theta$$

(b) Use Python to plot streamlines for this flow (i.e. lines of constant stream function), with $U = 1\,\mathrm{m/s}$ and $a = 1\,\mathrm{m}$.

Question 5

Consider a stretched elastic membrane on a rectangular frame that has a prescribed out of plane displacement along one of its boundaries (and zero along the other boundaries). The out of plane displacement $w(x, y)$ (the membrane displacement in the z-direction) is governed by the Laplace equation:



(a) Use the method of separation of variables to calculate the displacement throughout the membrane.

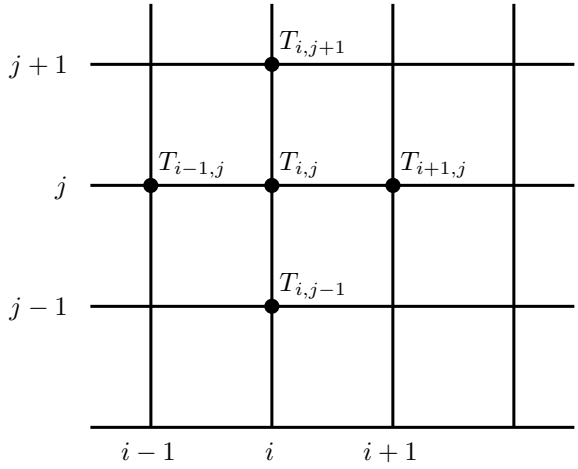
(b) Solve the same problem as above, but with $w|_{y=b} = w_0 \sin\left(\frac{\pi}{2a}x\right)$ and $\left.\frac{\partial w}{\partial x}\right|_{x=a} = 0$.

Chapter 3

Elliptic PDEs: Finite Difference method equations

3.1 Characteristics of a Finite Difference model

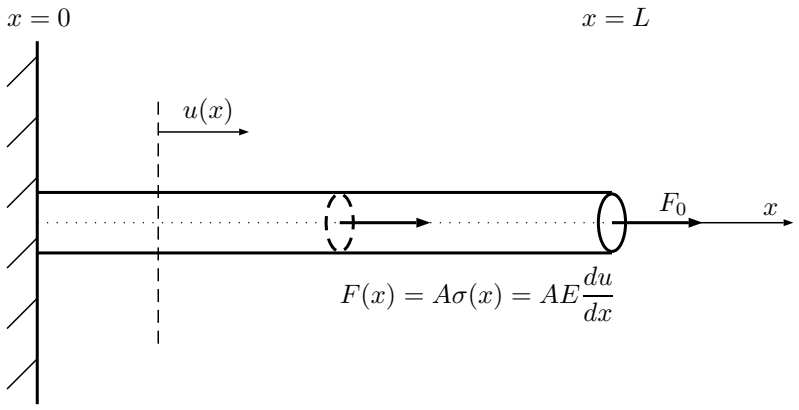
The computational domain is covered by a grid of points aligned in two (or three) orthogonal directions.



The problem is specified in terms of PDEs and a mathematical statement of the boundary conditions (b.c.). The values of the dependent variable (T for example) are defined at the grid points i, j. The derivatives at the grid points are approximated by Finite Difference expressions accurate to a given order in Δx and Δy. The discrete solution is forced to satisfy the governing PDEs and b.c. at the grid points to give a set of algebraic equations for the unknowns $T_{i,j}$.

3.2 Example in 1-D

An elastic rod is fixed at one end, and loaded with F_0 at the other.



where u: displacement, σ: stress (axial), A: rod cross-section area.

The equation governing the displacement inside the rod is:

(3.1)

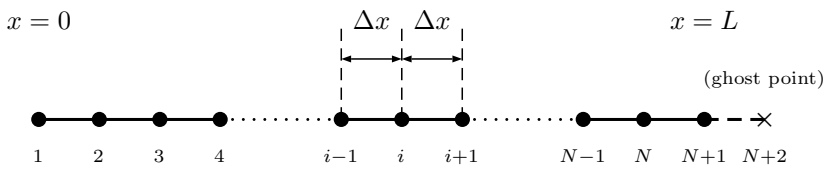
with a full derivative because the dependent variable u depends on x only, and is subject to:

(3.2)

and

(3.3)

First, we need to create a mesh of grid points. We choose a uniform mesh with equidistant points separated by Δx:



Finite Difference expressions are obtained by using the Taylor series expansion, for example:

(3.4)

and

(3.5)

where $\mathcal{O}(\Delta x^4)$ is the order of the *remainder*.

Adding Equations 3.4 and 3.5 yields:

(3.6)

with a second-order accurate formula. This scheme is a *central difference* scheme.

Assuming a constant A and E, the governing equation can be written as:

or when discretised at point i:

(3.7)

How to impose the boundary conditions?

At $x = 0$, Equation 3.7 involves u_0 which is outside of the physical domain since u_1 is located at $x = 0$. We do not need Equation 3.7 there since we know that $u_1 = 0$ through the boundary condition.

At the right-hand boundary ($x = L$), we add a *ghost point* u_{N+2} in order to impose the boundary condition. The equilibrium equation requires:

(3.8)

The boundary condition requires that:

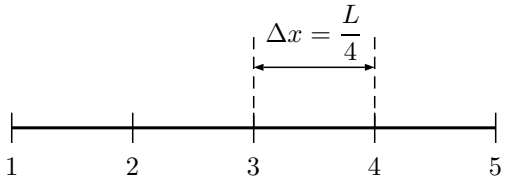
(3.9)

by using the central difference formula. Combining Equations 3.8 and 3.9 and eliminating u_{N+2} gives:

(3.10)

Summary

If we assume that the rod is covered by five points:



the discretised equations will be:

$$\begin{array}{rrrrrcl}
u_1 & & & & & = & 0 \\
-u_1 & +2u_2 & -u_3 & & & = & 0 \\
 & -u_2 & +2u_3 & -u_4 & & = & 0 \\
 & & -u_3 & +2u_4 & -u_5 & = & 0 \\
 & & & -u_4 & +u_5 & = & \frac{F_0 \Delta x}{AE}
\end{array}$$

and this algebraic system of equations can be written in matrix form:

$$\begin{bmatrix}
1 & 0 & 0 & 0 & 0 \\
-1 & 2 & -1 & 0 & 0 \\
0 & -1 & 2 & -1 & 0 \\
0 & 0 & -1 & 2 & -1 \\
0 & 0 & 0 & -1 & 1
\end{bmatrix}
\begin{Bmatrix}
u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5
\end{Bmatrix}
=
\begin{Bmatrix}
0 \\ 0 \\ 0 \\ 0 \\ \frac{F_0 \Delta x}{AE}
\end{Bmatrix}$$

Analytical solution

Integrating the governing equation:

Boundary at $x = L$:

Integrating for a second time:

Boundary at $x = 0$:

Therefore:

Python demonstration
FD in 1-D with five grid points, and with an arbitrary number of points.

3.3 Finite Difference schemes

Derivation of higher order FD formula

Consider the 4-point difference approximation of the first derivative using the points: u_{i-1}, u_i, u_{i+1}, and u_{i+2}. Using Taylor series expansion:

$$\begin{aligned}
u_{i-1} &= u_i - \left.\frac{\partial u}{\partial x}\right|_i \Delta x + \left.\frac{\partial^2 u}{\partial x^2}\right|_i \frac{\Delta x^2}{2} - \left.\frac{\partial^3 u}{\partial x^3}\right|_i \frac{\Delta x^3}{6} + \mathcal{O}(\Delta x^4) \\
u_i &= u_i \\
u_{i+1} &= u_i + \left.\frac{\partial u}{\partial x}\right|_i \Delta x + \left.\frac{\partial^2 u}{\partial x^2}\right|_i \frac{\Delta x^2}{2} + \left.\frac{\partial^3 u}{\partial x^3}\right|_i \frac{\Delta x^3}{6} + \mathcal{O}(\Delta x^4) \\
u_{i+2} &= u_i + \left.\frac{\partial u}{\partial x}\right|_i (2\Delta x) + \left.\frac{\partial^2 u}{\partial x^2}\right|_i \frac{(2\Delta x)^2}{2} + \left.\frac{\partial^3 u}{\partial x^3}\right|_i \frac{(2\Delta x)^3}{6} + \mathcal{O}(\Delta x^4)
\end{aligned}$$

We are going to assume that the FD approximation of the 1st order derivative takes the following form:

Substituting the above equations from the Taylor series expansion:

Since we have four coefficients (α_{i-1}, α_i, α_{i+1}, and α_{i+2}), we may choose these coefficients such that the terms corresponding to $\frac{\partial u}{\partial x}|_i$ sum to one, while the terms corresponding to u_i, $\frac{\partial^2 u}{\partial x^2}|_i$, and $\frac{\partial^3 u}{\partial x^3}|_i$ sum to zero. Or in

matrix notation, we need to solve:

$$\begin{bmatrix} & & \\ & & \\ & & \\ & & \\ & & \\ & & \end{bmatrix} \begin{Bmatrix} \\ \\ \\ \\ \\ \end{Bmatrix} = \begin{Bmatrix} \\ \\ \\ \\ \\ \end{Bmatrix}$$

which gives:

$$\begin{Bmatrix} \\ \\ \\ \\ \\ \end{Bmatrix} = \begin{Bmatrix} \\ \\ \\ \\ \\ \end{Bmatrix}$$

and substituting back into our general form:

yields a 3rd order accurate FD formula.

Central FD coefficients

Table of coefficients for central FD scheme, with uniform grid spacing:

Derivative	Accuracy	u_{i-2}	u_{i-1}	u_i	u_{i+1}	u_{i+2}
1	2		-1/2	0	1/2	
	4	1/12	-2/3	0	2/3	-1/12
2	2		1	-2	1	
	4	-1/12	4/3	-5/2	4/3	-1/12

See `https://en.wikipedia.org/wiki/Finite_difference_coefficient` for a more complete list.

For example, FD of the first order derivative, 2nd order accurate, using the central difference scheme:

and for a 4th order accurate FD:

Likewise, FD for the second order derivative, 2nd order accurate:

Error analysis of FD formula

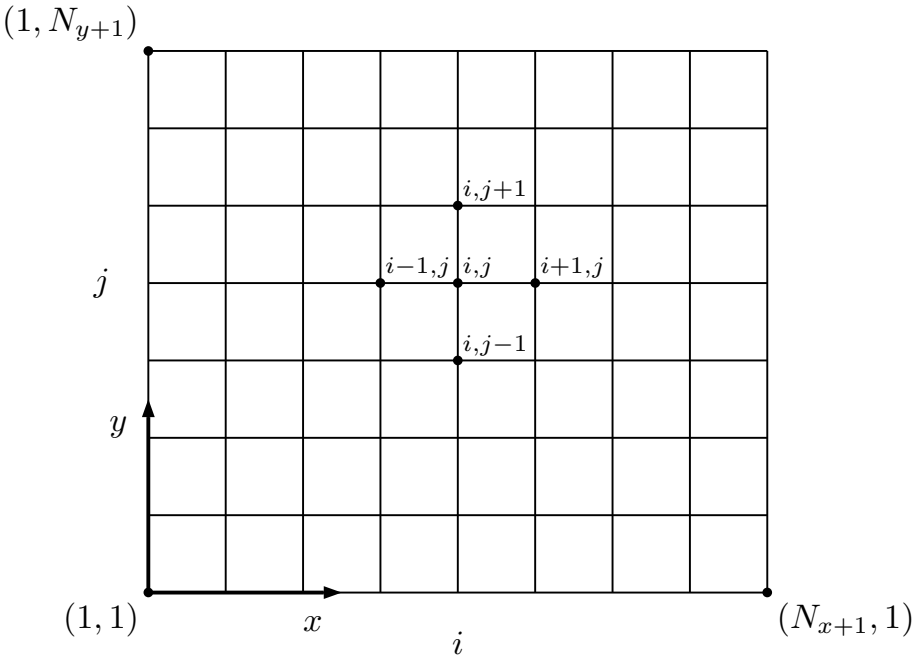
Consider the polynomial:

$$y = 1 + 2x - 0.5x^2 + 0.1x^3 + 0.025x^5$$

Python demonstration
FD error analysis

3.4 Extension to 2-D

We aim to solve the energy equation on a flat plate as described in Chapter 2. The plate is covered by a uniform grid of points.



The energy conservation equation derived before gives:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

Using the central difference scheme developed above, we can discretise this equation as follows:

(3.11)

(3.12)

(3.13)

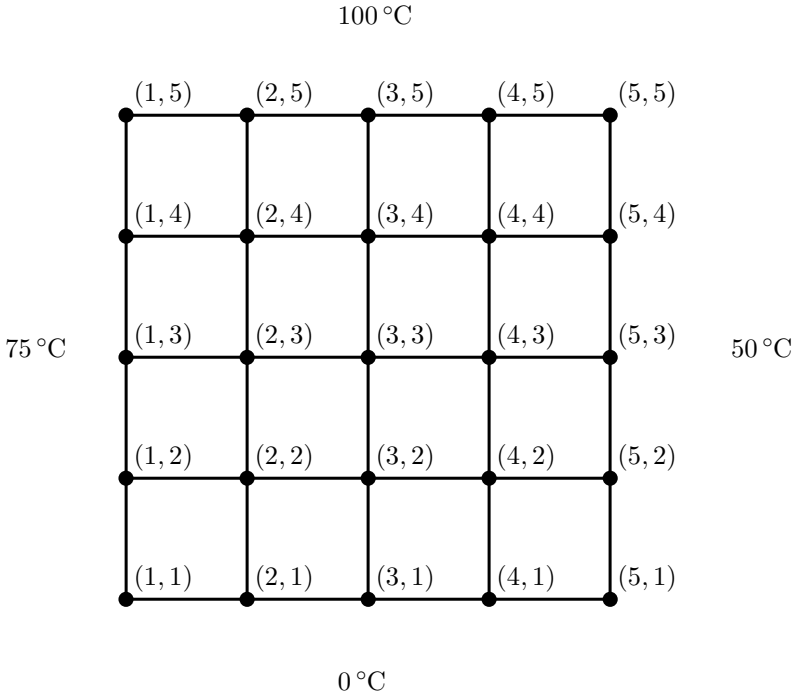
If the grid is square such that $\Delta x = \Delta y$, Equation 3.13 simplifies to:

(3.14)

and this relationship holds for *all* interior points.

Let's consider an example

Solve the heat equation on a plate with fixed temperatures on all sides.



All the points on the boundary need not be considered because the temperature is imposed by the boundary conditions there. We only

need to worry about internal grid points. For node $(2,2)$, we have according to Equation 3.14:

where $T_{1,2} = 75\,^{\circ}\mathrm{C}$ and $T_{2,1} = 0\,^{\circ}\mathrm{C}$.

Similarly, for all other interior nodes, we find:

$$
\begin{array}{rrrrrcr}
-4T_{2,2} & +T_{3,2} & +T_{2,3} & & & = & -75\,^{\circ}\mathrm{C} \\
T_{2,2} & -4T_{3,2} & +T_{4,2} & +T_{3,3} & & = & 0\,^{\circ}\mathrm{C} \\
T_{3,2} & -4T_{4,2} & +T_{4,3} & & & = & -50\,^{\circ}\mathrm{C} \\
T_{2,2} & -4T_{2,3} & +T_{3,3} & +T_{2,4} & & = & -75\,^{\circ}\mathrm{C} \\
T_{3,2} & +T_{2,3} & -4T_{3,3} & +T_{4,3} & +T_{3,4} & = & 0\,^{\circ}\mathrm{C} \\
T_{4,2} & +T_{3,3} & -4T_{4,3} & +T_{4,4} & & = & -50\,^{\circ}\mathrm{C} \\
T_{2,3} & -4T_{2,4} & +T_{3,4} & & & = & -175\,^{\circ}\mathrm{C} \\
T_{3,3} & +T_{2,4} & -4T_{3,4} & +T_{4,4} & & = & -100\,^{\circ}\mathrm{C} \\
T_{4,3} & +T_{3,4} & -4T_{4,4} & & & = & -150\,^{\circ}\mathrm{C}
\end{array}
\tag{3.15}
$$

$\Longrightarrow$ system of 9 equations for 9 unknowns.

This system of linear equations can be written in matrix form as follows:

(3.16)

$$\underline{\underline{\boldsymbol{A}}} = \begin{array}{c} \begin{matrix} T_{2,2} & T_{3,2} & T_{4,2} & T_{2,3} & T_{3,3} & T_{4,3} & T_{2,4} & T_{3,4} & T_{4,4} \end{matrix} \\ \begin{bmatrix} -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -4 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & -4 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & -4 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & -4 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & -4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 \end{bmatrix} \end{array},$$

$$\vec{\boldsymbol{T}} = \begin{Bmatrix} T_{2,2} \\ T_{3,2} \\ T_{4,2} \\ T_{2,3} \\ T_{3,3} \\ T_{4,3} \\ T_{2,4} \\ T_{3,4} \\ T_{4,4} \end{Bmatrix} \quad \text{and} \quad \vec{\boldsymbol{b}} = \begin{Bmatrix} -75\,^{\circ}\mathrm{C} \\ 0\,^{\circ}\mathrm{C} \\ -50\,^{\circ}\mathrm{C} \\ -75\,^{\circ}\mathrm{C} \\ 0\,^{\circ}\mathrm{C} \\ -50\,^{\circ}\mathrm{C} \\ -175\,^{\circ}\mathrm{C} \\ -100\,^{\circ}\mathrm{C} \\ -150\,^{\circ}\mathrm{C} \end{Bmatrix}$$

where $\underline{\underline{\boldsymbol{A}}}$ is a pentadiagonal matrix. This system of equations can easily be solved (for example, in Python).

Python demonstration
FD in 2-D

Chapter 4

Elliptic PDEs: Finite Difference method solutions

4.1 A simple solution technique to solve linear systems of equations: The Liebmann Method

Having discretised the heat equation, a system of linear algebraic equations is derived (see Equation 3.15). In the example we considered, we only had 9 unknowns (or degrees of freedom). Frequently, the number of unknowns is much larger than that. For example, solving the heat equation on a 20×20 grid leads to 400 unknowns. Therefore the matrix $\underline{\underline{\boldsymbol{A}}}$ in Equation 3.16 will have 400^2 entries. However each line in the matrix $\underline{\underline{\boldsymbol{A}}}$ will have at most 5 entries and therefore, the vast majority of the entries in matrix $\underline{\underline{\boldsymbol{A}}}$ will be *zeros*. $\Longrightarrow$ This is an inefficient use of computer memory. Direct solution methods such as Gauss elimination or LU decomposition require the full storage of this matrix $\underline{\underline{\boldsymbol{A}}}$. An attractive alternative is iterative methods such as the Gauss-Seidel method which when applied to PDEs is referred to as the Liebmann method.

The idea is to start from an initial guess to the solution and then iteratively try to improve on this initial guess. The first step of this method involves rewriting Equation 3.14 as:

(4.1)

The temperature at all nodes in the computational domain is updated according to Equation 4.1 and the most recent estimate of the temperature is used on the right-hand side of Equation 4.1.

Example

Considering the system of equations in Equation 3.15 and assuming a zero temperature as an initial guess, we get:

and so on for all other nodes.

This process is repeated multiple times and the question then arises as to when the iterations should stop. If we denote by $T_{i,j}^{\text{new}}$ the new computed value of $T_{i,j}$ and $T_{i,j}^{\text{old}}$ the old one, a good measure is the relative error $\varepsilon_{i,j}$ defined as:

(4.2)

and when the norm of relative error defined as:

(4.3)

falls below a stopping criterion, convergence is achieved. Note that this method is best suited for systems which are diagonally dominant, i.e. the term on the diagonal of $\underline{\underline{\boldsymbol{A}}}$ is dominant.

The convergence can be improved by using over-relaxation such that:

(4.4)

where λ is the over-relaxation factor which takes a value between 1 and 2. For example, following on from the example above and choosing $\lambda = 1.5$:

4.2 Derived variables and results visualisation

Once the solution to a PDE has been found it needs to be post-processed to visualise and analyse the results. A common way to visualise the results is to plot isolines (contour lines) which are lines on which the dependent variable is constant (e.g. isotherms). Another variable of interest, one might like to plot, is the heat flux:

In order to evaluate the derivatives, we may use central-difference:

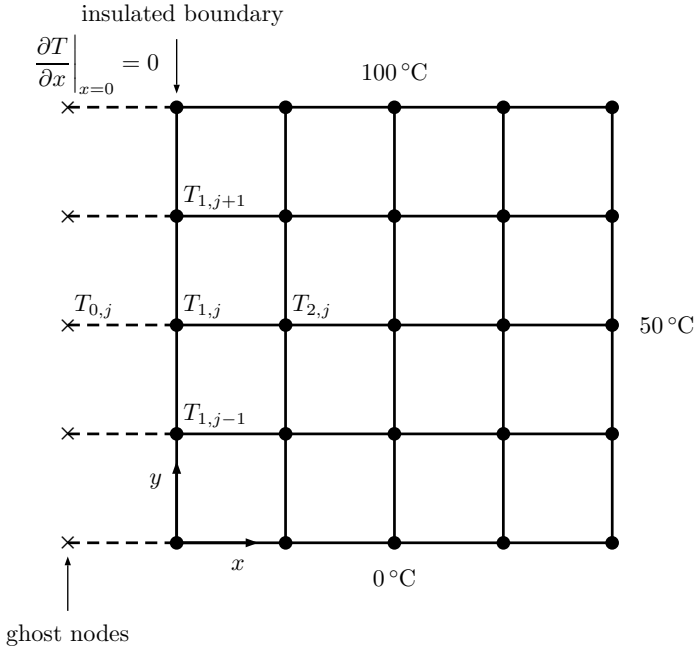
`Python demonstration`
Liebmann method, and over-relaxation.

(4.5a)

(4.5b)

4.3 Boundary conditions

We have already seen how to deal with the Dirichlet boundary condition (e.g. temperature imposed on the boundary). How do we deal with the Neumann boundary condition for which the gradient is imposed at the boundary (e.g. insulated boundary condition for which $\nabla T \cdot \hat{\mathbf{n}} = 0$)? The idea is to introduce *ghost nodes* outside of the computational domain.



Applying Equation 3.14 to the node $T_{1,j}$ gives:

(4.6)

We can next use the ghost node to explicitly discretise the boundary condition $\frac{\partial T}{\partial x}\Big|_{x=0} = 0$. The boundary condition requires that:

(4.7)

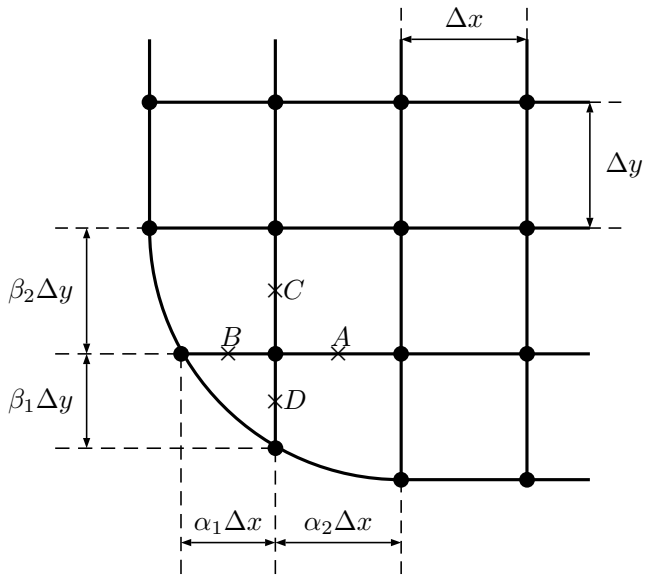
Combining Equations 4.6 and 4.7 gives:

(4.8)

where $\frac{\partial T}{\partial x}\Big|_{x=0} = 0$ if insulated. This new equation no longer includes the unknown value of T at the ghost node but includes the boundary condition in terms of the derivative. Other nodes can be treated in exactly the same way.

4.4 Curved boundaries

Many domains are not as simple as the square domain we have considered so far. Such domains may include a curved boundary. The Finite Difference method can be adapted to deal with such cases. Consider the domain below with a curved boundary.



We want to discretise the following equation on this irregular domain:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0 \tag{4.9}$$

In order to estimate the second order derivatives, we start by estimating the first order derivatives at A, B, C and D. In the x-direction:

or equivalently:

$$\left.\frac{\partial^2 T}{\partial x^2}\right|_{i,j} = \frac{2}{\Delta x^2}\left(\frac{T_{i+1,j} - T_{i,j}}{\alpha_2(\alpha_1 + \alpha_2)} - \frac{T_{i,j} - T_{i-1,j}}{\alpha_1(\alpha_1 + \alpha_2)}\right) \tag{4.10}$$

Likewise, for the y-direction:

$$\left.\frac{\partial^2 T}{\partial y^2}\right|_{i,j} = \frac{2}{\Delta y^2}\left(\frac{T_{i,j+1} - T_{i,j}}{\beta_2(\beta_1 + \beta_2)} - \frac{T_{i,j} - T_{i,j-1}}{\beta_1(\beta_1 + \beta_2)}\right) \tag{4.11}$$

Finally, substituting Equations 4.10 and 4.11 into Equation 4.9 gives:

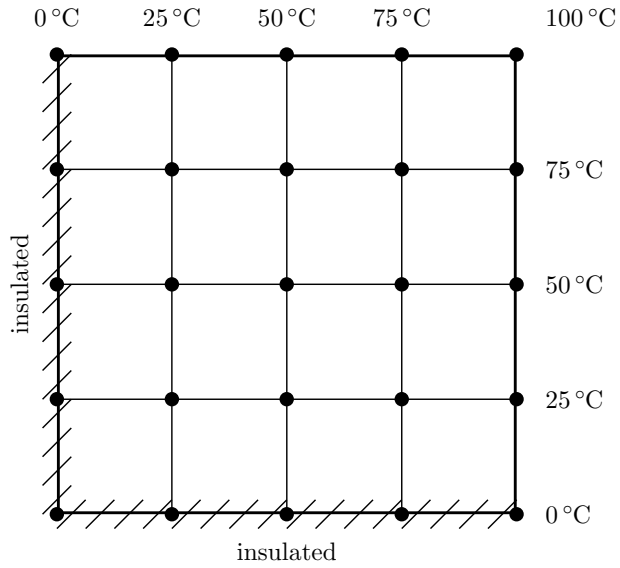
$$\begin{aligned} &\frac{2}{\Delta x^2}\left(\frac{T_{i+1,j} - T_{i,j}}{\alpha_2(\alpha_1 + \alpha_2)} - \frac{T_{i,j} - T_{i-1,j}}{\alpha_1(\alpha_1 + \alpha_2)}\right) \\ &+\frac{2}{\Delta y^2}\left(\frac{T_{i,j+1} - T_{i,j}}{\beta_2(\beta_1 + \beta_2)} - \frac{T_{i,j} - T_{i,j-1}}{\beta_1(\beta_1 + \beta_2)}\right) = 0 \end{aligned} \tag{4.12}$$

for the node near the curved boundary.

4.5 Exercises

Question 1

Use Liebmann's method to solve for the square heated plate, assume $\Delta x = \Delta y$. Iterate until the norm of the % relative error decreases below 0.01 %.



(a) Starting from an initial guess of $T = 0\,^{\circ}\text{C}$ for all unknown temperature values, how many iterations are required for convergence?

(b) Write the solution down directly on the plate illustrated above. How does it compare to the solution obtained using Python with `np.linalg.solve(A, b)`?

(c) Using over-relaxation, can you improve the rate of convergence of the Liebmann method? What is the optimal value of the over-relaxation parameter to optimise convergence?

Question 2

Discretise the Laplace equation for steady heat conduction in the unit square (i.e. 1×1, with the origin at the bottom left corner), given:

$$\begin{aligned} \left.\frac{\partial u}{\partial x}\right|_{x=0} &= 1 \\ u(x=1,y) &= y^2 \\ u(x,y=0) &= 0 \\ u(x,y=1) &= x \end{aligned}$$

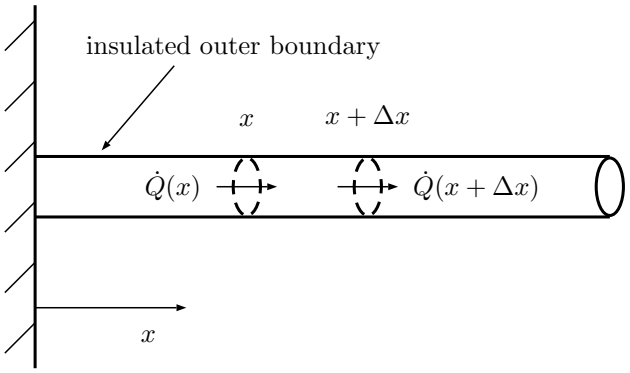
Use a grid of 5×5 nodes and choose $\Delta x = \Delta y = 0.25$.

Chapter 5

Transient PDEs

5.1 Parabolic PDEs: example

As discussed earlier Parabolic PDEs generally describe transient phenomena (i.e. time-dependent) and like many PDEs can be derived from an integral statement of conservation laws. As an illustration let's derive the transient heat equation in 1-D for a rod of constant cross-section A as shown below.



Consider an element of the rod between x and $x + \Delta x$. Conservation of energy requires that:

$$\begin{array}{c}\text{Net increase in heat}\\ \text{in the rod element}\\ \text{in unit time}\end{array} = \begin{array}{c}\text{Heat in} - \text{heat out}\\ \text{through the boundary}\\ \text{in unit time}\end{array} \tag{5.1}$$

Since the heat in the rod element is: $\rho c V T(x,t) = \rho c A \Delta x T(x,t)$

The net increase in heat in unit time is: $\rho c A \Delta x \frac{\partial T}{\partial t}$

The heat in through the left boundary is: $\dot{Q}(x,t)$

The heat out through the right boundary is: $\dot{Q}(x + \Delta x, t)$

Statement 5.1 therefore requires:

Using Taylor series expansion: $\dot{Q}(x + \Delta x) = \dot{Q}(x) + \frac{\partial \dot{Q}}{\partial x}\Delta x + ...$

However, Fourier's law requires that $\dot{Q}(x) = -kA\frac{\partial T}{\partial x}$, therefore:

$$\implies \underbrace{\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}}_{\text{Parabolic equation}} \tag{5.2}$$

where $\alpha = \frac{k}{\rho c}$ is the heat diffusivity.

The units of α can be determined:

The equations describing more complex phenomena such as the time-dependent electromagnetic equations or the equations of fluid mechanics have the same basic structure but with additional terms or with couplings to other equations. Being able to solve equations of type Equation 5.2 is a first step towards being able to solve more complex equations.

5.2 First solution method: the method of separation of variables

Assume a solution of the form $T(x,t) = F(x)G(t)$ for Equation 5.2:

dividing through by αFG gives:

As before, since the LHS is a function of t only and the RHS is a function of x only, this can only hold if both sides are equal to a constant:

Consider two cases:

1. $C < 0$ ($= -\lambda^2$ say), we have 2 ODEs:

$$\begin{aligned} & \frac{d^2F}{dx^2} + \lambda^2 F = 0 \quad \text{and} \quad \frac{dG}{dt} + \alpha\lambda^2 G = 0 \\ \implies \quad & T(x,t) = (A\cos(\lambda x) + B\sin(\lambda x))\, e^{-\alpha\lambda^2 t} \end{aligned} \tag{5.3}$$

2. $C > 0 = \lambda^2$, we have 2 ODEs:

$$\begin{aligned} & \frac{d^2F}{dx^2} - \lambda^2 F = 0 \quad \text{and} \quad \frac{dG}{dt} - \alpha\lambda^2 G = 0 \\ \implies & T(x,t) = \left(Ae^{\lambda x} + Be^{-\lambda x}\right) e^{\alpha\lambda^2 t} \end{aligned} \tag{5.4}$$

Equation 5.4 is generally not useful as it tends to infinity as t goes to infinity, instead, Equation 5.3 is generally the preferred solution. As before, the constants A and B are set in order to satisfy the initial and boundary conditions.

Example

Find the temperature distribution over time, $T(x,t)$, in a laterally insulated copper bar 80 cm long if the initial temperature profile is $100\sin(\frac{\pi x}{80})$ and the ends are kept at 0 °C.

Assume a solution of the form:

$$T(x,t) = (A\cos(\lambda x) + B\sin(\lambda x))\, e^{-\alpha\lambda^2 t}$$

The boundary conditions requires that $T(x=0) = T(x=80) = 0$:

Hence, a possible solution will be:

There exists an infinite number of solutions which satisfy the boundary conditions: one for each value of n. We now need to find a combination of all these solutions which also satisfies the initial conditions.

Finally the solution will be:

How long will it take for the maximum temperature in the rod to drop below 50 °C, given $\alpha_{\text{copper}} = 1.11 \times 10^{-4}\,\text{m}^2/\text{s} = 1.11\,\text{cm}^2/\text{s}$?

Since the maximum temperature will be at $x = 40\,\text{cm}$, we need to calculate t_f such that:

5.3 Dimensionless PDEs

We will show here how PDEs can be written in dimensionless form. By writing a problem in dimensionless form, specific equations from physics, engineering, chemistry or biology which originally looked different become one and the same. It is also very useful to reduce the number of independent variables governing a problem (e.g. Reynolds number in Navier-Stokes) and identify key dominating physics.

Consider the same problem from above:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \quad \text{subject to} \quad \begin{cases} T(x=0,t) = 0 \\ T(x=L,t) = 0 \\ T(x,t=0) = T_0 \sin\left(\frac{\pi x}{L}\right) \end{cases} \tag{5.5}$$

This equation, as it is, has dimensions:

$$\left[\frac{\partial T}{\partial t}\right] = \frac{\mathrm{K}}{\mathrm{s}} \quad \text{and} \quad \left[\alpha \frac{\partial^2 T}{\partial x^2}\right] = \frac{\mathrm{m}^2}{\mathrm{s}} \frac{\mathrm{K}}{\mathrm{m}^2} = \frac{\mathrm{K}}{\mathrm{s}}$$

and we aim to make this equation dimensionless.

First we introduce the following dimensionless variables:

$$\widetilde{T} = \frac{T}{T_0} \quad \text{and} \quad \widetilde{x} = \frac{x}{L} \quad \text{and} \quad \widetilde{t} = \frac{t}{t_0} \tag{5.6}$$

where L: length scale, t_0: time scale (yet to be determined), T_0: temperature scale.

Introduce the dimensionless variables into the governing PDE:

$$\frac{\partial T}{\partial t} = \frac{\partial}{\partial t}(T_0 \widetilde{T}) = T_0 \frac{\partial \widetilde{T}}{\partial \widetilde{t}} \frac{d\widetilde{t}}{dt} = \frac{T_0}{t_0} \frac{\partial \widetilde{T}}{\partial \widetilde{t}}$$

$$\frac{\partial T}{\partial x} = \frac{\partial}{\partial x}(T_0 \widetilde{T}) = T_0 \frac{\partial \widetilde{T}}{\partial x} = T_0 \frac{\partial \widetilde{T}}{\partial \widetilde{x}} \frac{d\widetilde{x}}{dx} = \frac{T_0}{L} \frac{\partial \widetilde{T}}{\partial \widetilde{x}}$$

$$\implies \frac{\partial^2 T}{\partial x^2} = \frac{\partial}{\partial x}\left(\frac{T_0}{L} \frac{\partial \widetilde{T}}{\partial \widetilde{x}}\right) = \frac{\partial}{\partial \widetilde{x}}\left(\frac{T_0}{L} \frac{\partial \widetilde{T}}{\partial \widetilde{x}}\right) \frac{d\widetilde{x}}{dx} = \frac{T_0}{L^2} \frac{\partial^2 \widetilde{T}}{\partial \widetilde{x}^2}$$

The governing PDE becomes:

$$\frac{T_0}{t_0} \frac{\partial \widetilde{T}}{\partial \widetilde{t}} = \frac{T_0}{L^2} \alpha \frac{\partial^2 \widetilde{T}}{\partial \widetilde{x}^2}$$

Let's multiply both sides by $\frac{L^2}{\alpha}$:

$$\frac{L^2}{\alpha t_0}\frac{\partial \widetilde{T}}{\partial \widetilde{t}} = \frac{\partial^2 \widetilde{T}}{\partial \widetilde{x}^2}$$

This equation is dimensionless because the RHS has no dimension.

We have not chosen the time scale t_0 at this point. Let's choose $t_0 = \frac{L^2}{\alpha}$, the governing PDE becomes:

$$\frac{\partial \widetilde{T}}{\partial \widetilde{t}} = \frac{\partial^2 \widetilde{T}}{\partial \widetilde{x}^2} \implies \text{the governing PDE is } \textit{free} \text{ of parameter} \tag{5.7}$$

The boundary conditions, $T(x = 0, t) = T(x = L, t) = 0$, need to be non-dimensionalised as well:

$$T_0\widetilde{T}(\widetilde{x} = 0, \widetilde{t}) = T_0\widetilde{T}(\widetilde{x} = 1, \widetilde{t}) = 0 \tag{5.8}$$

$$\implies \widetilde{T}(\widetilde{x} = 0, \widetilde{t}) = \widetilde{T}(\widetilde{x} = 1, \widetilde{t}) = 0$$

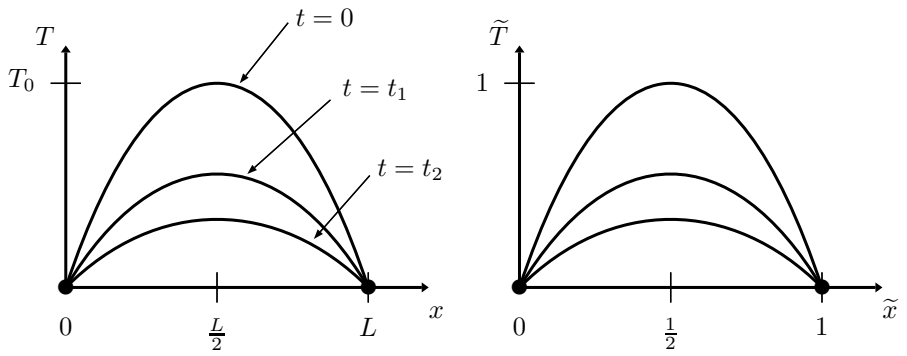
Likewise, the initial condition is:

$$T(x, t = 0) = T_0 \sin\left(\frac{\pi x}{L}\right)$$

$$\implies T_0\widetilde{T} = T_0 \sin(\pi\widetilde{x})$$

$$\implies \widetilde{T}(\widetilde{x}, \widetilde{t} = 0) = \sin(\pi\widetilde{x})$$

Let's compare the two problems in the physical space and dimensionless space:



Physical space Dimensionless space

The solution of the dimensionless PDE is:

$$\widetilde{T}(\widetilde{x}, \widetilde{t}) = \sin(\pi\widetilde{x}) \exp\left(-\pi^2\widetilde{t}\right)$$

5.4 Method of similarity solution

Consider a *semi-infinite* slab initially at temperature T_1. At $t = 0$, the surface of the slab is initially brought to T_∞. We want to find the temperature distribution in the slab. The problem is formally stated as:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \quad \text{subject to} \quad \begin{cases} T(x=0,t) = T_\infty \\ T(x \to \infty, t) \to T_1 \end{cases} \tag{5.9}$$

Let's introduce the dimensionless temperature:

$$\widetilde{T} = \frac{T - T_\infty}{T_1 - T_\infty} \tag{5.10}$$

In order to make the governing equation dimensionless, we need a characteristic length (such as L before) and a characteristic time (such as t_0). For that problem however, there is no obvious choice for L (the body is semi-infinite!). The only way the problem can be made dimensionless is by combining x and t into a new dimensionless variable. If we go through a formal non-dimensionalisation procedure, we could find that:

$$\widetilde{T} = \widetilde{T}\left(\frac{x}{\sqrt{\alpha t}}\right) \tag{5.11}$$

i.e. the dimensionless temperature is a function of a single variable.

For a reason which will become clear later, call this single variable:

$$\eta = \frac{x}{2\sqrt{\alpha t}} \tag{5.12}$$

Let's now use the chain rule to calculate:

$$\frac{\partial \widetilde{T}}{\partial t} = \frac{d\widetilde{T}}{d\eta}\frac{\partial \eta}{\partial t} = -\frac{\eta}{2t}\frac{d\widetilde{T}}{d\eta} \tag{5.13}$$

$$\frac{\partial \widetilde{T}}{\partial x} = \frac{d\widetilde{T}}{d\eta}\frac{\partial \eta}{\partial x} = \frac{1}{2\sqrt{\alpha t}}\frac{d\widetilde{T}}{d\eta}$$

$$\frac{\partial^2 \widetilde{T}}{\partial x^2} = \frac{\partial}{\partial x}\left(\frac{\partial \widetilde{T}}{\partial x}\right) = \frac{d}{d\eta}\left(\frac{1}{2\sqrt{\alpha t}}\frac{d\widetilde{T}}{d\eta}\right)\frac{\partial \eta}{\partial x}$$

$$= \frac{1}{4\alpha t}\frac{d^2\widetilde{T}}{d\eta^2} = \left(\frac{\eta}{x}\right)^2 \frac{d^2\widetilde{T}}{d\eta^2} \tag{5.14}$$

Plugging Equations 5.13 and 5.14 into Equation 5.9 gives:

Integrating once more gives:

where z is a dummy variable. For our choice of $\widetilde{T}$, we have $\widetilde{T}(0) = 0$. The second constant is obtained from the second boundary condition for which $\widetilde{T}(\eta \to \infty) \to 1$. This gives:

And the formula for $\widetilde{T}$ is:

$$\widetilde{T}(\eta) = \frac{2}{\sqrt{\pi}} \int_0^{\eta} e^{-z^2} dz = \underbrace{\text{erf}}_{\text{Gaussian error function}}(\eta)$$

Python demonstration
Plotting with and without for loops.

5.5 Exercises

Question 1

A uniform steel bar of length L, density ρ, specific heat c, and conductivity k is initially at temperature T_0. Both ends of the bar are then suddenly cooled down to 0 °C. If the temperature in the bar is governed by the 1-D unsteady diffusion equation with $\alpha = \frac{k}{\rho c}$, show that the temperature in the bar at any time t is given by:

$$T(x,t) = \sum_{n=1,3,5,\ldots} \frac{4T_0}{n\pi} \sin\left(\frac{n\pi}{L}x\right) e^{-\alpha\left(\frac{n\pi}{L}\right)^2 t} \tag{5.15}$$

Write a Python script to calculate the temperature $T(x,t)$ at time t and N equally spaced points along the bar. Your script should assign values to the input parameters: T_0, L, k, ρ, c, N and t before it performs this calculation.

Take $k = 554\,\mathrm{W/m\,°C}$, $\rho = 7833\,\mathrm{kg/m^3}$, $c = 465\,\mathrm{J/kg\,°C}$, $T_0 = 100\,°\mathrm{C}$ and $L = 1\,\mathrm{m}$. Estimate the time which will elapse before the temperature at any point in the bar has dropped below 5 °C.

Question 2

Solve the heat conduction equation:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \quad \text{subject to} \quad \begin{cases} T(x=0,t) = 0 & \text{for } t > 0 \\ \left.\frac{\partial T}{\partial x}\right|_{x=L} = 0 & \text{for } t > 0 \\ T(x,t=0) = T_0 \sin\left(\frac{3\pi}{2L}x\right) \end{cases} \tag{5.16}$$

Question 3

The heat equation for a fluid flowing with velocity (u,v) can be written:

$$\rho c \left(u\frac{\partial T}{\partial x} + v\frac{\partial T}{\partial y}\right) = k\left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}\right) + Q$$

where ρ is the density, c the heat capacity, k the thermal conductivity, and Q the heat source. Assume a known velocity (u,v) and an unknown temperature T.

(a) What is the order of this PDE?

(b) Is this PDE linear?

(c) Is this PDE homogeneous?

(d) Is this PDE elliptic, parabolic, or hyperbolic?

(e) Discretise this PDE using central difference.

(f) Non-dimensionalise this PDE using the following dimensionless variables:

$$\tilde{u} = \frac{u}{V_0}; \quad \tilde{v} = \frac{v}{V_0}; \quad \tilde{x} = \frac{x}{L}; \quad \tilde{y} = \frac{y}{L}; \quad \text{and } \tilde{T} = \frac{T}{T_0}$$

where V_0 is the characteristic flow velocity, L the characteristic length, and T_0 the characteristic temperature.

Question 4

You are to model the flow of treacle past a cylinder of diameter D. The treacle flows with velocity V_0. The PDEs governing the flow are the Navier–Stokes equations which can be written in the following form:

$$\begin{aligned} \rho\left(u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y}\right) &= -\frac{\partial p}{\partial x} + \mu\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) \\ \rho\left(u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y}\right) &= -\frac{\partial p}{\partial y} + \mu\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right) - \rho g \end{aligned}$$

where (u, v) is the velocity field, ρ the density, p the pressure, and μ the viscosity.

Consider the following dimensionless variables:

$$\tilde{x} = \frac{x}{D}; \quad \tilde{y} = \frac{y}{D}; \quad \tilde{u} = \frac{u}{V_0}; \quad \tilde{v} = \frac{v}{V_0}; \quad \tilde{p} = \frac{p}{p_0} \text{ with } p_0 = \frac{\mu V_0}{D}$$

(a) Non-dimensionalise the Navier–Stokes equations and show that the process leads to the introduction of a dimensionless number, the Reynolds number:

$$\mathrm{Re} = \frac{\rho V_0 D}{\mu}$$

(b) The viscosity of the treacle is $\mu = 100\,\mathrm{Pa\,s}$ and the density is $\rho = 1430\,\mathrm{kg/m^3}$. If the velocity $V_0 = 0.1\,\mathrm{m/s}$ and the cylinder diameter $D = 0.1\,\mathrm{m}$, deduce which term can be neglected in the dimensionless Navier–Stokes equations.

Chapter 6

Transient PDEs: numerical solutions

6.1 Introductory comments

When a PDE contains derivatives with respect to space and time, the spatial and time discretisation are usually treated separately, i.e.:

1. First, discretise in space using techniques described earlier.
2. The discrete values of the solution at the grid points $T_i(t)$ are then discretised in time.
3. The discretised solution is marched forward in time.

Consider for example the heat equation:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \tag{6.1}$$

We can discretise this equation in space first to obtain a system of coupled Ordinary Differential Equations (ODEs) in time:

$$\frac{\partial T_i}{\partial t} = \alpha \left(\frac{T_{i-1} - 2T_i + T_{i+1}}{\Delta x^2} + \underbrace{\mathcal{O}(\Delta x^2)}_{\substack{\text{2nd order accurate} \\ \text{discretisation} \\ \text{in space}}} \right) \qquad i = 1, 2, ..., N \tag{6.2}$$

which holds for all nodes within the computational domain ($i = 1, 2, ..., N$). We must now integrate Equation 6.2 in time. The time interval for which we require a solution is divided into sub-intervals:

and we define $T_i^n = T_i(t^n)$, i.e. the temperature at node i and time t^n.

How do we discretise in time?

By using a Finite Difference approximation of the time derivative on the LHS, equations are formed which can be used to step the solution forward in time.

1. **Explicit method**

   In an explicit method, the time derivative on the LHS of Equation 6.2 is formulated using values of T_i at the $(n+1)^{\text{th}}$ and n^{th} time step while the RHS is evaluated only at past times:

   (6.3)

2. **Implicit method**

   In an implicit method, the current and past information are included in the evaluation of the RHS of Equation 6.2. This gives a time-marching algorithm of the form:

   (6.4)

   Unknowns at the current time step $(n+1)$ appear on both sides of Equation 6.4 and a set of simultaneous algebraic equations must therefore be solved to obtain the solution of the new time step.

6.2 The Forward in Time Central in Space (FTCS) scheme

The integration scheme developed for scalar ODEs carry over to systems of coupled ODEs. We can use for example the Euler explicit method to integrate in time:

(6.5)

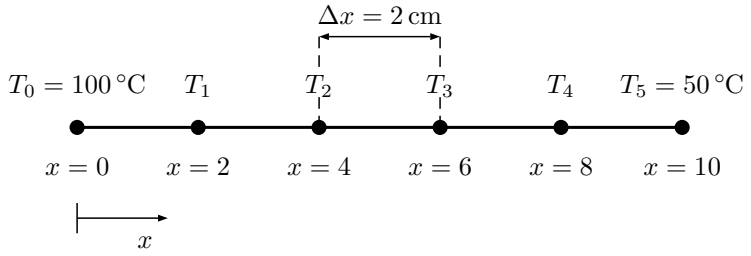
which is first order accurate in time.

Combining Equations 6.2 and 6.5 yields:

(6.6)

where the RHS is evaluated at the old time step.

Consider the computational grid of length $L = 10\,\text{cm}$ with spacing $\Delta x = 2\,\text{cm}$, and an initial temperature of $0\,^\circ\text{C}$:



Given the parameters above with a thermal diffusivity $\alpha = 0.835\,\text{cm}^2/\text{s}$, and selecting a time step $\Delta t = 0.1\,\text{s}$, we have:

Let's apply the FTCS scheme to update the temperature. At $t = 0.1\,\text{s}$:

Doing the same for all other points gives:

$$T_3^1 = \underbrace{T_3^0}_{0} + 0.020\,875\Big(\underbrace{T_2^0}_{0} - 2\,\underbrace{T_3^0}_{0} + \underbrace{T_4^0}_{0}\Big) = 0$$
$$T_4^1 = \underbrace{T_4^0}_{0} + 0.020\,875\Big(\underbrace{T_3^0}_{0} - 2\,\underbrace{T_4^0}_{0} + \underbrace{T_5^0}_{50}\Big) = 1.0438$$

At $t = 0.2\,\text{s}$:

$$T_2^2 = \underbrace{T_2^1}_{0} + 0.020\,875\Big(\underbrace{T_1^1}_{2.0875} - 2\,\underbrace{T_2^1}_{0} + \underbrace{T_3^1}_{0}\Big) = 0.043\,577$$
$$T_3^2 = \underbrace{T_3^1}_{0} + 0.020\,875\Big(\underbrace{T_2^1}_{0} - 2\,\underbrace{T_3^1}_{0} + \underbrace{T_4^1}_{1.0438}\Big) = 0.021\,788$$
$$T_4^2 = \underbrace{T_4^1}_{1.0438} + 0.020\,875\Big(\underbrace{T_3^1}_{0} - 2\,\underbrace{T_4^1}_{1.0438} + \underbrace{T_5^1}_{50}\Big) = 2.0439$$

And so on...

`Python demonstration`
Explicit with $\Delta t = 0.5\,\text{s}, 1\,\text{s}, 5\,\text{s}$

6.3 Convergence and stability

Convergence means that as Δt and Δx tend to zero, the numerical solution tends towards the true solution. Stability means that errors at any stage of the computation are not amplified but are attenuated as the computation progresses. Stability and convergence are satisfied for the FTCS scheme provided:

(6.7)

This stability criterion is very limiting because as Δx is made smaller, Δt has to be made much smaller (e.g. if Δx is halved, Δt needs to be quartered) which means that the computational time increases substantially as the mesh is refined. On the other hand, the FTCS scheme has the advantage that it is very easy to code and implement.

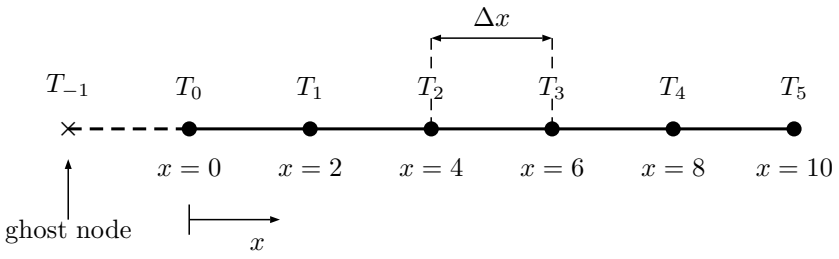
6.4 Neumann boundary conditions

Boundary conditions involving derivatives (Neumann BC) can be implemented using the idea of ghost nodes, as introduced before.

Example

Solve the same problem as before but subject to:

$$\left.\frac{dT}{dx}\right|_{x=0} = \beta$$



The discretised equation at node 0 gives:

(6.8)

The boundary condition can also be discretised such that:

(6.9)

Combining Equations 6.8 and 6.9 gives:

The same form applies for all time steps.

6.5 A simple implicit method: Backward in Time Central in Space (BTCS)

The limitations of the explicit scheme can be overcome by choosing an implicit scheme for which the spatial derivative is evaluated at the new time step $(n+1)$:

(6.10)

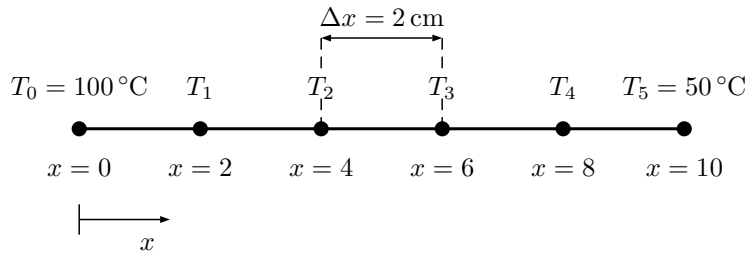
The only difference between Equation 6.10 and Equation 6.6 is that the RHS now involves terms at the new time step. This scheme may also be rewritten:

(6.11)

Unlike the explicit scheme for which we had an explicit expression to march forward in time, we now have a set of simultaneous linear equations to solve. This numerical method is therefore more complex to implement and at first sight appears more computational expensive. However, it is unconditionally stable and therefore there is no limitation on the size of the time step which is a very attractive feature.

Example

Let's solve the same problem as above but now with the BTCS scheme.



Applying the BTCS scheme gives:

Likewise, we can form the following equations:

$$-0.020\,875T_1^1 + 1.041\,75T_2^1 - 0.020\,875T_3^1 = \underbrace{T_2^0}_{0}$$

$$-0.020\,875T_2^1 + 1.041\,75T_3^1 - 0.020\,875T_4^1 = \underbrace{T_3^0}_{0}$$

$$-0.020\,875T_3^1 + 1.041\,75T_4^1 = T_4^0 + 0.020\,875 \times 50$$

This system of equations can be written in matrix format as follows:

$$\begin{bmatrix} 1.041\,75 & -0.020\,875 & 0 & 0 \\ -0.020\,875 & 1.041\,75 & -0.020\,875 & 0 \\ 0 & -0.020\,875 & 1.041\,75 & -0.020\,875 \\ 0 & 0 & -0.020\,875 & 1.041\,75 \end{bmatrix} \begin{Bmatrix} T_1^1 \\ T_2^1 \\ T_3^1 \\ T_4^1 \end{Bmatrix} = \begin{Bmatrix} 2.0875 \\ 0 \\ 0 \\ 1.043\,75 \end{Bmatrix}$$

This system of equations can be solved for the temperature at $t = 0.1\,\text{s}$: $T_1^1 = 2.0047$; $T_2^1 = 0.0406$; $T_3^1 = 0.0209$; $T_4^1 = 1.0023$. When solving for the next time step, the matrix $[\underline{\underline{A}}]$ remains the same, whereas the right-hand side vector differs from one time step to the next because it is a function of the solution at the "*old*" time step t^n.

Although the BTCS offers the advantage of being unconditionally stable, it is still *only* first order accurate. This can be improved upon by using the Crank-Nicolson method.

6.6 The Crank-Nicolson method

An improvement on the BTCS scheme which is second-order accurate in space and time is to develop difference approximations at the midpoint of the time increment. Accordingly:

And the final discretisation reads:

(6.12)

Upon rearrangement, Equation 6.12 can also be written as:

(6.13)

Example

Solve the same problem as above using the Crank-Nicolson method.

The equations will be:

and likewise:

$$
\begin{aligned}
-0.020\,875T_1^1 + 2.041\,75T_2^1 - 0.020\,875T_3^1 &= 0 \\
-0.020\,875T_2^1 + 2.041\,75T_3^1 - 0.020\,875T_4^1 &= 0 \\
-0.020\,875T_3^1 + 2.041\,75T_4^1 &= 2.0875
\end{aligned}
$$

This system of equations can be written in matrix form as follows:

$$\begin{bmatrix} 2.041\,75 & -0.020\,875 & 0 & 0 \\ -0.020\,875 & 2.041\,75 & -0.020\,875 & 0 \\ 0 & -0.020\,875 & 2.041\,75 & -0.020\,875 \\ 0 & 0 & -0.020\,875 & 2.041\,75 \end{bmatrix} \begin{Bmatrix} T_1^1 \\ T_2^1 \\ T_3^1 \\ T_4^1 \end{Bmatrix}$$

$$= \begin{Bmatrix} 4.175 \\ 0 \\ 0 \\ 2.0875 \end{Bmatrix}$$

This system of equations can be solved for the temperature at $t = 0.1\,\text{s}$: $T_1^1 = 2.045$; $T_2^1 = 0.021$; $T_3^1 = 0.0107$; $T_4^1 = 1.0225$. To solve the temperature at $t = 0.2\,\text{s}$, the right-hand side vector must be changed to:

$$\vec{b} = \begin{Bmatrix} 8.1801 \\ 0.0841 \\ 0.0427 \\ 4.0901 \end{Bmatrix}$$

The matrix $[\underline{\underline{A}}]$ remains unchanged. The solution at $t = 0.2\,\text{s}$ is: $T_1^2 = 4.0073$; $T_2^2 = 0.0826$; $T_3^2 = 0.0422$; $T_4^2 = 2.0036$.

6.7 Performance comparison

We have discussed three discretisation schemes: FTCS, BTCS and Crank-Nicolson. There are many others but these are the most basic ones. Let's now compare their relative performance.

Consider the problem from earlier for which we found the temperature distribution in a laterally insulated copper bar 80 cm long and with initial temperature $T(x, t = 0) = 100 \sin\left(\frac{\pi x}{80}\right)$.

Solve this problem with the FTCS, BTCS and Crank-Nicolson and estimate the error in the middle of the rod at $t = 100\,\text{s}$. Choose $\Delta x = 8\,\text{cm}$. Compare the performance of the various schemes.

Python demonstration
Implicit and Crank-Nicolson schemes

6.8 Exercises

Question 1

Consider the temperature history in an insulated rod. The temperature satisfies:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \tag{6.14}$$

for $0 \leqslant x \leqslant L$, $t > 0$. The initial condition is $T(x, t = 0) = f(x)$ and the end conditions are $T(x = 0, t) = T_0$ and $T(x = L, t) = T_1$. The steady-state solution as $t \to \infty$ is $T(x, t) = T_0(1 - \frac{x}{L}) + T_1 \frac{x}{L}$.

(a) Compute the solution using the FTCS with $L = 1$, $\alpha = 1$, $\Delta x = 0.1$, $f(x) = x^2$, $T_0 = 0$, $T_1 = 1$. Choose Δt such that $\lambda = \frac{\alpha \Delta t}{\Delta x^2} = 0.45$. Integrate for 60 time steps and plot the solution.

(b) Repeat the same but with Δt such that $\lambda = 0.55$. What happens to the solution?

(c) Solve the same problem using the Crank-Nicolson scheme with $\lambda = 0.45$ and $\lambda = 0.55$. Observe the differences.

Question 2

In a chemical engineering manufacturing system, a process involves the diffusion of salt into a layer of water. The layer is of thickness L and initially has a uniform salt concentration c_0. At time $t = 0$, the layer is brought into contact with a saline solution at one surface and the concentration is raised to c_s while the other surface is maintained at concentration c_0. The mass diffusivity a.k.a. diffusion coefficient of salt in water is denoted by D. The governing PDE for the concentration of salt in water $c(x, t)$ where x is the coordinate distance measured from the surface whose concentration is raised to c_s is:

$$\frac{\partial c}{\partial t} = D \frac{\partial^2 c}{\partial x^2} \quad \text{subject to} \quad \begin{cases} c(x, t = 0) = c_0 & \\ c(x = 0, t) = c_s & \text{for } t > 0 \\ c(x = L, t) = c_0 & \text{for } t > 0 \end{cases} \tag{6.15}$$

(a) Non-dimensionalise this equation using the following dimensionless variables:

$$\tilde{x} = \frac{x}{L}; \quad \tilde{t} = \frac{Dt}{L^2} \quad \text{and } \tilde{c} = \frac{c - c_0}{c_s - c_0}$$

and show that the corresponding dimensionless equation is:

$$\frac{\partial \tilde{c}}{\partial \tilde{t}} = \frac{\partial^2 \tilde{c}}{\partial \tilde{x}^2} \quad \text{subject to} \quad \begin{cases} \tilde{c}(\tilde{x}, \tilde{t} = 0) = 0 \\ \tilde{c}(\tilde{x} = 0, \tilde{t} > 0) = 1 \\ \tilde{c}(\tilde{x} = 1, \tilde{t} > 0) = 0 \end{cases} \tag{6.16}$$

(b) Discretise this PDE using the BTCS numerical scheme.

(c) Discuss the advantages and disadvantages of the BTCS relative to FTCS.

Chapter 7

Hyperbolic PDEs: wave equation

7.1 Example of wave equation: the vibrating string

As discussed previously, PDEs often arise as an expression of basic physical principle. We will derive here the partial differential equation which governs the motion of a vibrating string like the one on a violin.

Consider the string shown below. Its deflection is denoted by $u(x,t)$.

In order to derive the governing equation, we will apply Newton's Law to a small element of the string between x and $x + \Delta x$. The string is subject to a tension force which is tangential to the curve of the string. Because the element of string has *no* net motion in the x-direction, the forces (projected on x) must balance:

(7.1)

In the vertical direction, the unbalanced forces produce an acceleration of the string element. Accordingly:

(7.2)

But since $T_2 \cos\beta = T_1 \cos\alpha = T$, we can rearrange this equation as:

(7.3)

But since $\tan\alpha = \left.\frac{\partial u}{\partial x}\right|_x$ and $\tan\beta = \left.\frac{\partial u}{\partial x}\right|_{x+\Delta x}$, we obtain:

(7.4)

Letting Δx go to zero and introducing $c^2 = \frac{T}{\rho}$, we get:

(7.5)

$\implies$ This equation is the 1-D wave equation which governs the problem. As discussed earlier, this is a hyperbolic equation.

This equation requires *two* boundary conditions $u(x = 0, t)$ and $u(x = L, t)$ and *two* initial conditions $u(x, t = 0) = f(x)$ and $\left.\frac{\partial u}{\partial t}\right|_{t=0} = g(x)$.

7.2 The method of separation of variables applied to the wave equation

Just like we did for the parabolic PDE, we will assume a solution of the form:

(7.6)

Replacing this assumed solution in the governing PDE (Equation 7.5):

$$\frac{\partial u}{\partial x} = G\frac{dF}{dx} \quad \text{and} \quad \frac{\partial^2 u}{\partial x^2} = G\frac{d^2F}{dx^2}$$
$$\implies F\frac{d^2G}{dt^2} = c^2 G\frac{d^2F}{dx^2}$$

Dividing throughout by c^2FG gives:

$$\frac{1}{c^2G}\frac{d^2G}{dt^2} = \frac{1}{F}\frac{d^2F}{dx^2} = \alpha \tag{7.7}$$

Because the LHS is a function of t only and the RHS is a function of x only, this can only hold if both are equal to a constant α. We can then distinguish two cases depending on the sign of α:

1. $\alpha = -\lambda^2 < 0$ leads to the following two ODEs:

$$\begin{aligned} &\frac{d^2G}{dt^2} + c^2\lambda^2 G = 0 \quad \text{and} \quad \frac{d^2F}{dx^2} + \lambda^2 F = 0 \\ \implies &G(t) = A\cos(\lambda ct) + B\sin(\lambda ct) \\ \text{and } &F(x) = C\cos(\lambda x) + D\sin(\lambda x) \\ \implies &u(x,t) = \big(A\cos(\lambda ct) + B\sin(\lambda ct)\big) \\ &\qquad\qquad \times \big(C\cos(\lambda x) + D\sin(\lambda x)\big) \end{aligned} \tag{7.8}$$

2. $\alpha = \lambda^2 > 0$ leads to the following two ODEs:

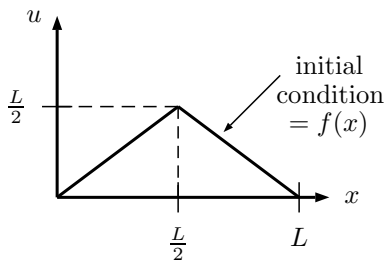
$$\begin{aligned} &\frac{d^2G}{dt^2} - c^2\lambda^2 G = 0 \quad \text{and} \quad \frac{d^2F}{dx^2} - \lambda^2 F = 0 \\ \implies &G(t) = Ae^{\lambda ct} + Be^{-\lambda ct} \\ \text{and } &F(x) = Ce^{\lambda x} + De^{-\lambda x} \\ \implies &u(x,t) = \left(Ae^{\lambda ct} + Be^{-\lambda ct}\right)\left(Ce^{\lambda x} + De^{-\lambda x}\right) \end{aligned} \tag{7.9}$$

As usual, the constants A, B, C and D are determined by using the boundary and initial conditions. Case 1 is usually more useful. If the initial conditions can be expressed in the form of a Fourier series in x, Equation 7.8 can be used to progress the solution forward in time.

Example

Solve the wave equation (Equation 7.5) for the vibration of a string stretched between the two points $x = 0$ and $x = L$ and subject to the following conditions:

- $u(x = 0, t) = 0$ (fixed at $x = 0$)
- $u(x = L, t) = 0$ (fixed at $x = L$)
- $\left.\frac{\partial u}{\partial t}\right|_{t=0} = 0$ (zero initial velocity)
- $u(x, t = 0) = \begin{cases} x & (0 \leqslant x \leqslant \frac{L}{2}) \\ L - x & (\frac{L}{2} \leqslant x \leqslant L) \end{cases}$



We seek a solution of the form:

$$u(x,t) = \big(A\cos(\lambda ct) + B\sin(\lambda ct)\big)\big(C\cos(\lambda x) + D\sin(\lambda x)\big)$$

- Boundary condition $u(x=0,t)=0$ requires that $C=0$; and
- Boundary condition $u(x=L,t)=0$ requires that $\lambda = \frac{n\pi}{L}$:

  $$u(x,t) = \left(A'\cos\left(\frac{n\pi}{L}ct\right) + B'\sin\left(\frac{n\pi}{L}ct\right)\right)\sin\left(\frac{n\pi}{L}x\right)$$

  where $A' = AD$ and $B' = BD$.
- Initial condition $\left.\frac{\partial u}{\partial t}\right|_{t=0} = 0$ yields:

  $$\left(-A'\frac{n\pi c}{L}\sin\left(\frac{n\pi}{L}ct\right) + B'\frac{n\pi c}{L}\cos\left(\frac{n\pi}{L}ct\right)\right)\sin\left(\frac{n\pi}{L}x\right) = 0$$

  where $t=0$, which requires that $B'=0$.

A particular solution will have the form:

$$u_n(x,t) = A_n\cos\left(\frac{n\pi}{L}ct\right)\sin\left(\frac{n\pi}{L}x\right) \tag{7.10}$$

Each value of n corresponds to a different solution. We now need to choose a correct combination of all these particular solutions u_n to match the initial condition $u(x,t=0) = f(x)$.

The initial condition can be expressed as a Fourier sine series as follows:

$$f(x) = \sum_{n=1}^{\infty} b_n\sin\left(\frac{n\pi}{L}x\right) \quad \text{with} \quad b_n = \frac{4L}{\pi^2 n^2}\sin\left(\frac{n\pi}{2}\right) \tag{7.11}$$

Combining Equations 7.10 and 7.11, we have:

$$u(x,t=0) = \sum_{n=1}^{\infty} u_n = \sum_{n=1}^{\infty} A_n\sin\left(\frac{n\pi}{L}x\right) = \sum_{n=1}^{\infty} b_n\sin\left(\frac{n\pi}{L}x\right)$$

with $A_n = b_n$.

Therefore, the solution will finally be:

$$u(x,t) = \sum_{n=1}^{\infty}\frac{4L}{n^2\pi^2}\sin\left(\frac{n\pi}{2}\right)\cos\left(\frac{n\pi}{L}ct\right)\sin\left(\frac{n\pi}{L}x\right)$$

7.3 Numerical solution of the wave equation: explicit scheme

Test problem: the vibrating string

Consider the string displacement $u(x,t)$ satisfies:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2} \qquad 0 \leqslant x \leqslant L \quad \text{and} \quad t > 0$$

subject to the initial and boundary conditions:

$$\begin{aligned} u(x, t=0) &= f(x) \\ \left.\frac{\partial u}{\partial t}\right|_{t=0} &= g(x) \\ u(x=0, t) &= 0 \\ u(x=L, t) &= 0 \end{aligned}$$

Let's approximate the time and space derivatives by:

(7.12)

which we can rewrite as:

(7.13)

This is a two-step explicit scheme since it involves the new time step $n+1$ and two older time steps n and $n-1$. Equation 7.13 cannot be applied at $t = 0$ since the solution at two previous time steps is required.

In order to deal with this issue, we introduce a ghost point at $t = t^{-1} = -\Delta t$. We can then approximate the second initial condition by 2nd order central difference:

Substituting into Equation 7.13 gives:

$$\begin{aligned} u_i^1 &= 2(1-\lambda^2)u_i^0 + \lambda^2(u_{i+1}^0 + u_{i-1}^0) - u_i^1 + 2\Delta t g_i \\ \implies 2u_i^1 &= 2(1-\lambda^2)u_i^0 + \lambda^2(u_{i+1}^0 + u_{i-1}^0) + 2\Delta t g_i \\ \implies u_i^1 &= (1-\lambda^2)u_i^0 + \frac{\lambda^2}{2}(u_{i+1}^0 + u_{i-1}^0) + \Delta t g_i \end{aligned}$$

Sample computation:

Compute a solution for the particular case: $L = 1$, $c = 1$, $g(x) = 0$ and with

$$f(x) = \begin{cases} \frac{2x}{L} & (0 \leqslant x \leqslant \frac{L}{2}) \\ 2\left(1 - \frac{x}{L}\right) & (\frac{L}{2} \leqslant x \leqslant L) \end{cases}$$

It can be shown that this scheme is conditionally stable for $\lambda < 1$ ($c\Delta t < \Delta x$). Just like for parabolic PDEs, this stability restriction can be removed by using implicit schemes.

Python demonstration
Stability is conditional on λ

Chapter 8

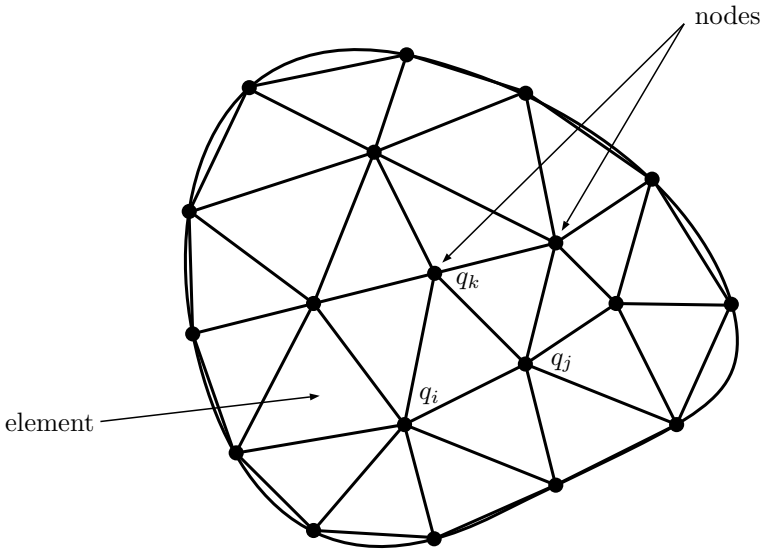
Finite Element method for PDEs (1-D)

8.1 Background and motivation

So far, we have used the *Finite Difference* approach whereby the domain is divided into a grid of nodes. The derivatives of the PDE are then approximated at these nodes to form a system of algebraic equations which can then be solved. Although, we have seen a method to deal with irregular boundaries, this method is *not* well suited for complex geometries, complex boundary conditions, or heterogeneous composition.

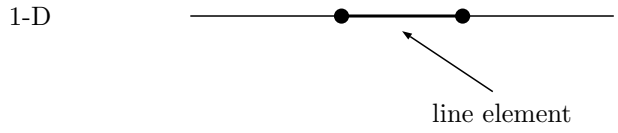
The Finite Element (FE) method provides an alternative which is better suited for such problems. The characteristics of a FE model are as follows:

- A region of space is divided into finite subdomains, *elements*
- Each element is defined by a topology of *nodes*
- The nodal values of the dependent variable q_i, q_j, q_k ... are the unknown quantities
- Interpolation is used to define the dependant variable within each element
- The physics is imposed in a "weak" or integrated sense based on the governing equation or an equivalent physical principle, e.g. principle of virtual work

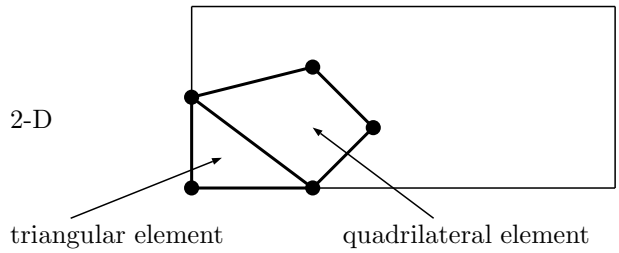


8.2 Discretisation of the domain into small elements

In 1-D, these elements are line elements:



In 2-D, elements are typically triangular or quadrilateral elements:



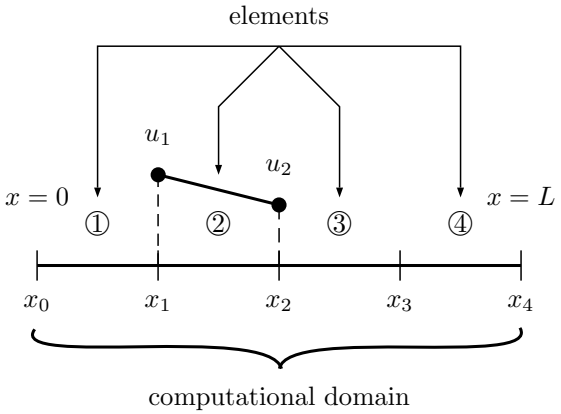
8.3 Deriving the element equations

This involves two stages:

- Choosing an *interpolation* function which approximates the solution within the element given nodal values.
- Finding the nodal values by invoking the governing PDE.

8.3.1 Interpolation function

Polynomials are typically used as the interpolating function given their simplicity. For example, in 1-D, the simplest possible interpolation is the linear interpolation.



Assume the dependent variable:

(8.1)

on each element labelled ①, ②, ③, ④, and a_0 and a_1 are unknown constants at this stage which we aim to express in terms of u_1 and u_2: the values of the dependent variable u at x_1 and x_2. We must have:

(8.2a)

(8.2b)

The constants a_0 and a_1 can easily be expressed in terms of x_1, x_2, u_1, u_2 as follows:

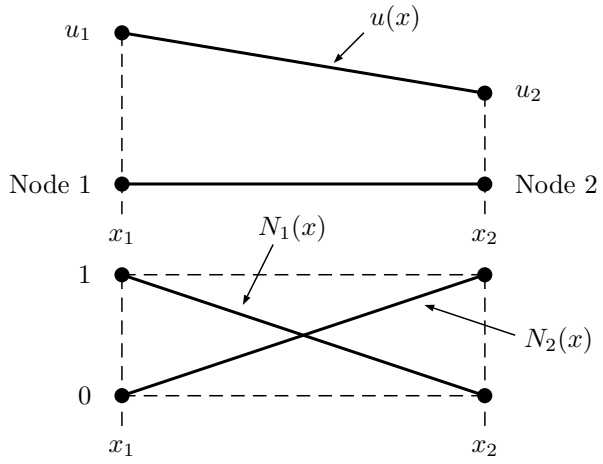
(8.3)

Plugging in Equation 8.3 into Equation 8.1 gives:

(8.4)

(8.5)

$N_1(x)$ and $N_2(x)$ are known as "shape" functions. The shape function has the property that its value is *1* at its own node ($N_1(x_1) = 1$ or $N_2(x_2) = 1$) but *0* at all other nodes ($N_1(x_2)$ or $N_2(x_1) = 0$).



In the FE framework, we also need to know $\frac{du}{dx}$ or $\int_{x_1}^{x_2} u dx$. These can easily be evaluated using Equation 8.5:

(8.6)

where $\frac{u_2-u_1}{x_2-x_1}$ is the usual definition of the slope. Likewise:

These integrals can be evaluated formally or one can just note that they correspond to the area under $N_1(x)$ and $N_2(x)$, i.e.:

(8.7)

which is identical to the trapezoidal rule of integration.

8.3.2 Nodal values

Once the interpolation function has been selected, one needs to find the required nodal values u_1, u_2 such that the "*trial*" solution $u(x) = N_1(x)u_1 + N_2(x)u_2$ matches the exact solution in some sense (yet to be discussed). We will discuss later the "method of weighted residuals" to derive the *element equations* which mathematically take the form:

$$\underline{\underline{\boldsymbol{K}}}^e \vec{\boldsymbol{u}}^e = \vec{\boldsymbol{F}}^e \tag{8.8}$$

where $\underline{\underline{K}}^e$: element stiffness matrix (or local stiffness matrix), $\vec{u}^e$ a column vector of unknowns at the nodes, $\vec{F}^e$: a column vector reflecting the effect of external influences applied to the node (or local forcing vector).

8.4 Assembly

Once the element equations have been derived, they must be linked together or assembled to represent the entire system. The assembly process is based on the concept of continuity: continuity of the solution at the nodes and possibly continuity of the derivatives, i.e. matching solution at common element boundaries. Once all elements have been assembled, the entire system is expressed in matrix form as follows:

$$\underline{\underline{K}}^g \vec{u}^g = \vec{F}^g \tag{8.9}$$

where $\underline{\underline{K}}^g$: global stiffness matrix, $\vec{u}^g$, $\vec{F}^g$: global column vector of unknowns and forcing.

8.5 Boundary conditions

Up until this stage, the boundary conditions have not been applied. The application of the constraints imposed on the PDE changes the resulting system of equations to:

$$\underline{\underline{K}} \vec{u} = \vec{F} \tag{8.10}$$

which is now the final form including boundary conditions.

8.6 Solving the system of equations

Using a solver such as the Liebmann method or other more effective direct or iterative solvers.

8.7 Post-processing

Plotting the solution and interpreting it.

8.8 Example of approximating with shape functions

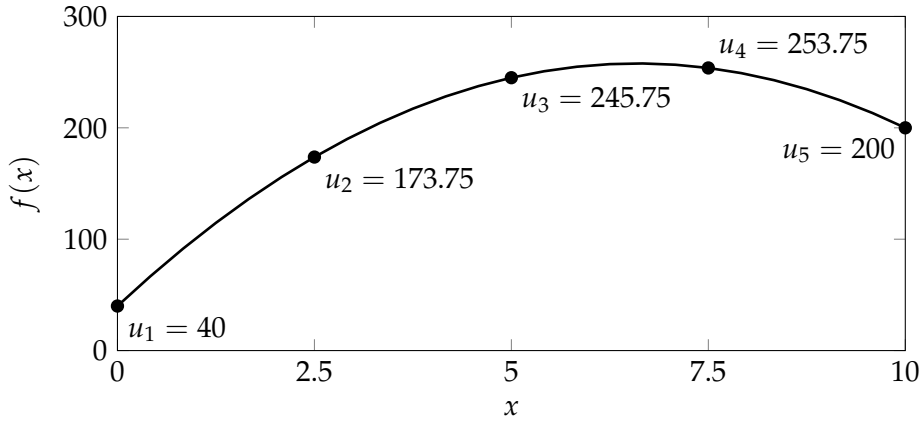
Compare linear and quadratic elements to approximate the function:

$$f(x) = -5x^2 + 66x + 40 \tag{8.11}$$

for $0 \leqslant x \leqslant 10$. Use four linear elements, then two quadratic elements.

8.8.1 Linear elements

First, we plot the function and piecewise linear segments to visualise:



Using linear interpolation on each element, the function $f(x)$ can be approximated by $\widetilde{f}(x)$ which is a piecewise linear function defined by:

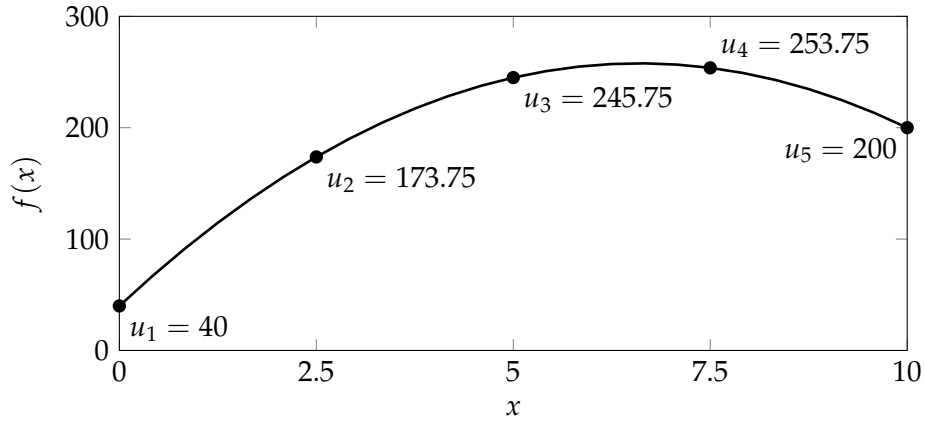
(8.12)

with the shape functions as derived earlier:

(8.13)

8.8.2 Quadratic elements

Second, consider the same function with two quadratic elements:



Let's now derive a quadratic approximation such that:

(8.14)

Introduce a normalised local coordinate system:

(8.15)

Assume that:

(8.16)

where a, b, c are unknown constants.

We know that:

(8.17a)

(8.17b)

(8.17c)

Solving for a, b, c:

(8.18a)

(8.18b)

(8.18c)

(8.19a)

(8.19b)

(8.19c)

For the linear element we had $u(x) = N_1(x)u_1 + N_2(x)u_2$ and for this quadratic element we now have $u(x) = N_1(x)u_1 + N_2(x)u_2 + N_3(x)u_3$.

Substituting our constants a, b, c into the polynomial:

(8.20a)

(8.20b)

(8.20c)

Checking that $N_1(\xi)$ is a valid shape function:

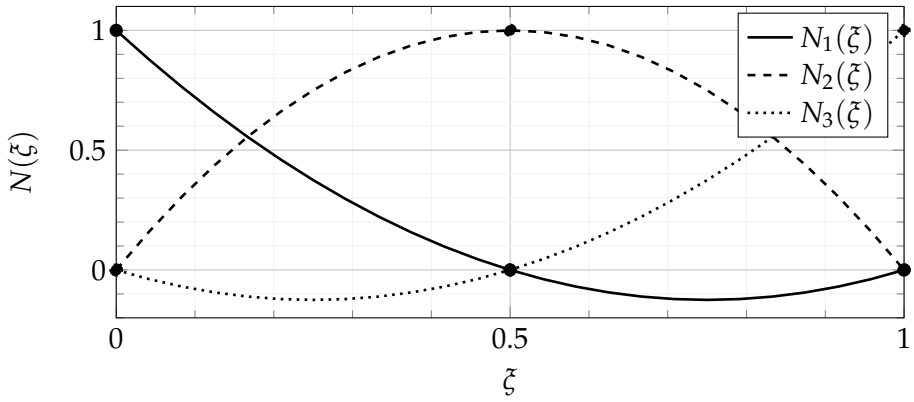
(8.21a)

(8.21b)

(8.21c)

and likewise for $N_2(\xi)$ and $N_3(\xi)$.

Plotting these shape functions:



Python demonstration
Linear vs quadratic interpolation.

Chapter 9

Finite Element application (1-D)

In order to illustrate the FE solution process, we consider the problem of a long thin rod subject to fixed boundary conditions and a continuous heat source along its axis. The equation governing the temperature distribution in this long thin rod is:

$$\frac{d^2T}{dx^2} = -f(x) \tag{9.1}$$

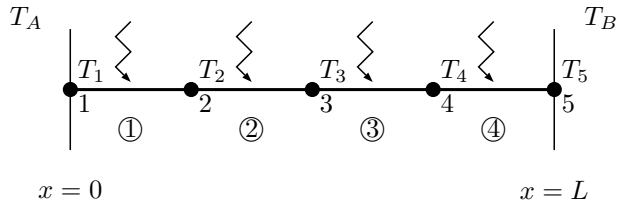
subject to:

$$T(0) = T_A \quad \text{and} \quad T(L) = T_B \tag{9.2}$$

Although this problem involves an Ordinary Differential Equation instead of a PDE, it serves as a useful illustration of the FE methodology.

9.1 Discretisation of the domain into small elements

We split the domain into 4 equal linear elements of size $\frac{L}{4}$.



9.2 Element equations: the method of weighted residuals

The governing DE can be rewritten as:

(9.3)

Assume we have an approximate solution $\widetilde{T}$ to Equation 9.3. When this approximate solution is plugged in Equation 9.3, the left-hand side will not be precisely equal to zero (if it were, $\widetilde{T}$ would be the exact solution), a small residual $R(x)$ will remain instead such that:

(9.4)

The Method of Weighted Residuals consist in finding a minimum for the residual according to the following formula:

(9.5)

where D is the solution domain and W_i a set of linearly independent weighting functions. The choice of a weighting function is large, but a very popular method known as the Galerkin method consists of choosing the interpolation functions (also known as shape functions) for the weighting functions such that:

(9.6)

Restricting ourself to a single element (i.e. $x_1 \leqslant x \leqslant x_2$) to obtain the element matrix, we obtain:

(9.7)

(9.8)

At this stage, we will use the rule of integration by parts to lower the order of the term $\frac{d^2\widetilde{T}}{dx^2}$ from two to one. Accordingly:

Consider $i = 1$ first, and remember we have seen that $N_1(x) = \frac{x_2 - x}{x_2 - x_1}$.

Likewise, for $i = 2$, we have seen that $N_2(x) = \frac{x - x_1}{x_2 - x_1}$.

Equation 9.8 can therefore be rewritten as:

(9.9a)

(9.9b)

Remembering that:

Substituting these expressions into Equation 9.9a gives:

Likewise:

We can therefore rewrite Equations 9.9a and 9.9b in matrix form:

$$\frac{1}{(x_2 - x_1)} \underbrace{\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}}_{\text{Element stiffness matrix}} \begin{Bmatrix} T_1 \\ T_2 \end{Bmatrix} = \underbrace{\begin{Bmatrix} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_1} \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_2} \end{Bmatrix}}_{\text{Effect of boundary condition}} + \underbrace{\begin{Bmatrix} \int_{x_1}^{x_2} f N_1 dx \\ \int_{x_1}^{x_2} f N_2 dx \end{Bmatrix}}_{\text{Effect of external load}} \tag{9.10}$$

This set of equations represent the element equations. We have so far

developed the element equations for one element.

Example

Develop the element equations for a 10 cm long rod with boundary conditions $T(0) = 40$ and $T(10) = 200$, and a uniform heat source $f(x) = 10$. Each element has a length of $\Delta x = 2.5$ cm. The governing equation is given by:

For element ① we need to calculate:

Similarly:

Therefore the element equations for element ① will be:

(9.11a)

(9.11b)

Or in matrix format:

$$① \quad \underbrace{\begin{bmatrix} & \\ & \end{bmatrix}}_{\underline{\underline{K}}^e_1} \begin{Bmatrix} & \end{Bmatrix} = \underbrace{\begin{Bmatrix} & \\ & \end{Bmatrix}}_{\vec{F}^e_1}$$

Similarly, the element equations for element ②, ③ and ④ are:

② $\underbrace{\begin{bmatrix} \\ \\ \end{bmatrix}}_{\underline{\underline{K}}^e_2} \begin{Bmatrix} \\ \\ \end{Bmatrix} = \underbrace{\begin{Bmatrix} \\ \\ \end{Bmatrix}}_{\vec{F}^e_2}$

③ $\underbrace{\begin{bmatrix} \\ \\ \end{bmatrix}}_{\underline{\underline{K}}^e_3} \begin{Bmatrix} \\ \\ \end{Bmatrix} = \underbrace{\begin{Bmatrix} \\ \\ \end{Bmatrix}}_{\vec{F}^e_3}$

④ $\underbrace{\begin{bmatrix} \\ \\ \end{bmatrix}}_{\underline{\underline{K}}^e_4} \begin{Bmatrix} \\ \\ \end{Bmatrix} = \underbrace{\begin{Bmatrix} \\ \\ \end{Bmatrix}}_{\vec{F}^e_4}$

9.3 Assembly

The assembly process involves adding up all of the element stiffness matrices $\underline{\underline{K}}^e_{2\times2}$ into the global stiffness matrix $\underline{\underline{K}}^g_{5\times5}$. The global stiffness matrix has a size of 5×5 because the system has 5 degrees of freedom.

Add $\underline{\underline{K}}^e_1$ to the global stiffness matrix $\underline{\underline{K}}^g$:

$$\begin{bmatrix} (0.4) & (-0.4) & 0 & 0 & 0 \\ (-0.4) & (0.4) & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{Bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \\ T_5 \end{Bmatrix}$$

Add $\underline{\underline{K}}^e_2$ to the global stiffness matrix $\underline{\underline{K}}^g$:

$$\begin{bmatrix} 0.4 & -0.4 & 0 & 0 & 0 \\ -0.4 & 0.4+(0.4) & (-0.4) & 0 & 0 \\ 0 & (-0.4) & (0.4) & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{Bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \\ T_5 \end{Bmatrix}$$

Add $\underline{\underline{\boldsymbol{K}}}_3^e$ to the global stiffness matrix $\underline{\underline{\boldsymbol{K}}}^g$:

$$\begin{bmatrix} 0.4 & -0.4 & 0 & 0 & 0 \\ -0.4 & 0.8 & -0.4 & 0 & 0 \\ 0 & -0.4 & 0.4+(0.4) & (-0.4) & 0 \\ 0 & 0 & (-0.4) & (0.4) & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{Bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \\ T_5 \end{Bmatrix}$$

Add $\underline{\underline{\boldsymbol{K}}}_4^e$ to the global stiffness matrix $\underline{\underline{\boldsymbol{K}}}^g$:

$$\begin{bmatrix} 0.4 & -0.4 & 0 & 0 & 0 \\ -0.4 & 0.8 & -0.4 & 0 & 0 \\ 0 & -0.4 & 0.8 & -0.4 & 0 \\ 0 & 0 & -0.4 & 0.4+(0.4) & (-0.4) \\ 0 & 0 & 0 & (-0.4) & (0.4) \end{bmatrix} \begin{Bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \\ T_5 \end{Bmatrix}$$

Finally, the global stiffness matrix reads:

$$\underbrace{\begin{bmatrix} 0.4 & -0.4 & 0 & 0 & 0 \\ -0.4 & 0.8 & -0.4 & 0 & 0 \\ 0 & -0.4 & 0.8 & -0.4 & 0 \\ 0 & 0 & -0.4 & 0.8 & -0.4 \\ 0 & 0 & 0 & -0.4 & 0.4 \end{bmatrix}}_{\underline{\underline{\boldsymbol{K}}}^g} \underbrace{\begin{Bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \\ T_5 \end{Bmatrix}}_{\vec{\boldsymbol{T}}^g}$$

We can also assemble the global forcing vector $\vec{\boldsymbol{F}}^g$ which has five entries (one per degree of freedom). Local forcing vectors for elements ①, ②, ③ and ④ are as follows.

$$\vec{F}_1^e = \begin{Bmatrix} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_1} + 12.5 \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_2} + 12.5 \end{Bmatrix} \quad ; \quad \vec{F}_2^e = \begin{Bmatrix} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_2} + 12.5 \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_3} + 12.5 \end{Bmatrix}$$

$$\vec{F}_3^e = \begin{Bmatrix} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_3} + 12.5 \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_4} + 12.5 \end{Bmatrix} \quad ; \quad \vec{F}_4^e = \begin{Bmatrix} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_4} + 12.5 \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_5} + 12.5 \end{Bmatrix}$$

Add $\vec{F}_1^e$ to global forcing vector $\vec{F}^g$:

$$\begin{Bmatrix} \left(-\left.\frac{d\widetilde{T}}{dx}\right|_{x_1} + 12.5\right) \\ \left(\left.\frac{d\widetilde{T}}{dx}\right|_{x_2} + 12.5\right) \\ 0 \\ 0 \\ 0 \end{Bmatrix} = \begin{Bmatrix} \\ \\ \\ \\ \end{Bmatrix}$$

Add $\vec{F}_2^e$ to global forcing vector $\vec{F}^g$:

$$\begin{Bmatrix} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_1} + 12.5 \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_2} + 12.5 + \left(-\left.\frac{d\widetilde{T}}{dx}\right|_{x_2} + 12.5\right) \\ \left(\left.\frac{d\widetilde{T}}{dx}\right|_{x_3} + 12.5\right) \\ 0 \\ 0 \end{Bmatrix} = \begin{Bmatrix} \\ \\ \\ \\ \end{Bmatrix}$$

Add $\vec{F}_3^e$ to global forcing vector $\vec{F}^g$:

$$\begin{Bmatrix} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_1} + 12.5 \\ 25 \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_3} + 12.5 + \left(-\left.\frac{d\widetilde{T}}{dx}\right|_{x_3} + 12.5\right) \\ \left(\left.\frac{d\widetilde{T}}{dx}\right|_{x_4} + 12.5\right) \\ 0 \end{Bmatrix} = \begin{Bmatrix} \\ \\ \\ \\ \end{Bmatrix}$$

Add $\vec{F}_4^e$ to global forcing vector $\vec{F}^g$:

$$\left\{\begin{array}{c} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_1} + 12.5 \\ 25 \\ 25 \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_4} + 12.5 + \left(-\left.\frac{d\widetilde{T}}{dx}\right|_{x_4} + 12.5\right) \\ \left(\left.\frac{d\widetilde{T}}{dx}\right|_{x_5} + 12.5\right) \end{array}\right\} = \underbrace{\left\{\begin{array}{c} \\ \\ \\ \\ \\ \end{array}\right\}}_{\vec{F}^g}$$

9.4 Boundary conditions

The internal boundary conditions cancel out as the equations are assembled. Applying the boundary condition $T(0) = T_A = 40$ for the first equation:

and similarly $T(10) = T_B = 200$ for the last equation:

and then the system of equations can be written as:

$$\begin{array}{lccccc} \left.\frac{d\widetilde{T}}{dx}\right|_{x_1} & -0.4T_2 & & & = & -3.5 \\ & 0.8T_2 & -0.4T_3 & & = & 41 \\ & -0.4T_2 & +0.8T_3 & -0.4T_4 & = & 25 \\ & & -0.4T_3 & +0.8T_4 & = & 105 \\ & & & -0.4T_4 \quad -\left.\frac{d\widetilde{T}}{dx}\right|_{x_5} & = & -67.5 \end{array}$$

9.5 Solution

This system of equations $\underline{\underline{K}}\vec{T} = \vec{F}$ can be solved to give:

$$\left.\frac{d\widetilde{T}}{dx}\right|_{x_1} = 66, \quad T_2 = 173.75, \quad T_3 = 245, \quad T_4 = 253.75, \quad \left.\frac{d\widetilde{T}}{dx}\right|_{x_5} = -34$$

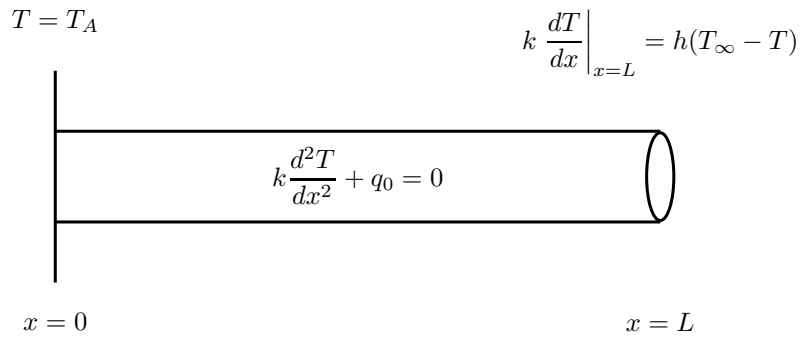
9.6 Post-processing

See results on a graph.

9.7 Exercises

Question 1

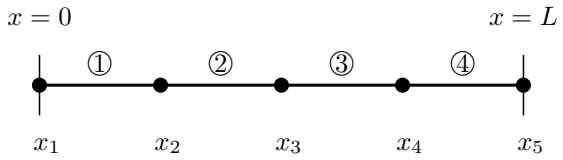
Consider the steady heat transfer problem with uniform heating, with the temperature fixed at one end and convective heat transfer at the other as illustrated:



To simplify the algebra in what follows, we take $T_A = T_\infty = 0$, $\frac{q_0}{k} = \alpha$ and $\frac{h}{k} = \beta$. This problem therefore simplifies to:

$$\frac{d^2T}{dx^2} + \alpha = 0 \quad \text{in } 0 \leqslant x \leqslant L \quad \text{subject to} \quad \begin{cases} T(0) = 0 \\ \left.\frac{dT}{dx}\right|_{x=L} = -\beta T \end{cases} \tag{9.12}$$

We are to solve this problem using four linear elements of equal length:



(a) Find the element equations for element ①, ②, ③, and ④.

(b) Assemble all the element equations in a global stiffness matrix and a global forcing vector.

(c) Assuming $L = 1\,\text{m}$, $\alpha = 200\,\text{K/m}^2$ and $\beta = 0$, calculate the numerical solution obtained with the FE method and compare to the true solution given by:

$$T_{\text{exact}}(x) = -\frac{\alpha x}{2}\left(x - L\frac{2+\beta L}{1+\beta L}\right) \tag{9.13}$$

Question 2

A version of the Poisson equation that occurs in mechanics is the following model for the vertical displacement of a bar with a distributed load $P(x)$.

$$A_c E \frac{\partial^2 u}{\partial x^2} = P(x) \tag{9.14}$$

where A_c is the cross-sectional area, E the Young's modulus, u the deflection, and x the distance measured along the length.

If the bar is fixed at both ends (i.e. $u = 0$), use the Finite Element method to model its deflection for $A_c = 0.1\,\mathrm{m}^2$, $E = 200 \times 10^9\,\mathrm{Pa}$, $L = 10\,\mathrm{m}$, and $P(x) = 1000\,\mathrm{N/m}$. Employ a value of $\Delta x = 2\,\mathrm{m}$.

Question 3

The total error in a numerical simulation is the sum of the round-off error and the discretisation error. Describe what these are and plot the total error as a function of the step size Δx.

Chapter 10

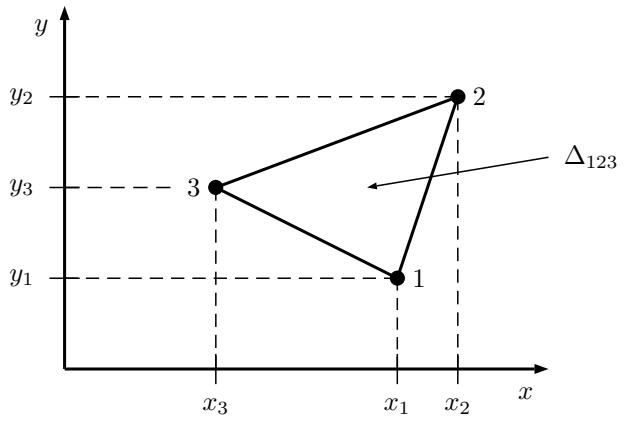
Finite Element extension to multiple dimensions

10.1 Problems in 2-D

The principles of the FE method for 1-D problems presented during the last two sections extend to 2-D and 3-D, however, the calculus is more involved and the bookkeeping required to actually code the method is markedly more complex. We will highlight here the key differences and use COMSOL for the implementation of the FE method.

10.2 Discretisation

It typically involves breaking up the computational domain into triangles or quadrilaterals. Equations are then derived for each element.



10.3 Choosing the interpolation function

Typically, the unknown function is assumed to vary linearly or quadratically over the element. For the purpose of illustration, we will consider linear triangular elements as shown above.

For an interpolation function, we will consider the simplest possible:

where a, b, c are (yet) unknown constants. These constants are set by using the conditions at each node:

which can equivalently be written as:

$$\begin{bmatrix} & & \\ & & \\ & & \end{bmatrix} \begin{Bmatrix} \\ \\ \end{Bmatrix} = \begin{Bmatrix} \\ \\ \end{Bmatrix}$$

This system of equations for a, b, c can be solved using basic algebra and the solution is:

$$c = \frac{1}{2A_e}\Big(u_1(x_2y_3 - x_3y_2) + u_2(x_3y_1 - x_1y_3) + u_3(x_1y_2 - x_2y_1)\Big)$$
$$a = \frac{1}{2A_e}\Big(u_1(y_2 - y_3) + u_2(y_3 - y_1) + u_3(y_1 - y_2)\Big)$$
$$b = \frac{1}{2A_e}\Big(u_1(x_3 - x_2) + u_2(x_1 - x_3) + u_3(x_2 - x_1)\Big)$$

where A_e is the area of the triangular element:

$$A_e = \frac{1}{2}\Big((x_2y_3 - x_3y_2) + (x_3y_1 - x_1y_3) + (x_1y_2 - x_2y_1)\Big) \tag{10.1}$$

We need to rewrite these equations as:

(10.2)

Simple rearrangement gives:

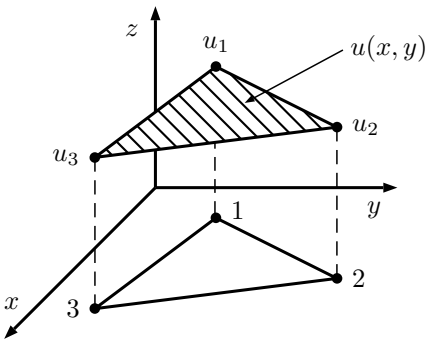
$$N_1(x,y) = \frac{1}{2A_e}\Big((x_2y_3 - x_3y_2) + (y_2 - y_3)x + (x_3 - x_2)y\Big) \qquad (10.3a)$$

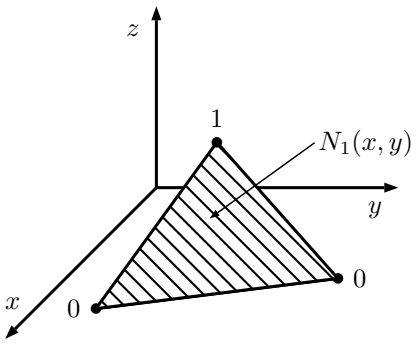
$$N_2(x,y) = \frac{1}{2A_e}\Big((x_3y_1 - x_1y_3) + (y_3 - y_1)x + (x_1 - x_3)y\Big) \qquad (10.3b)$$

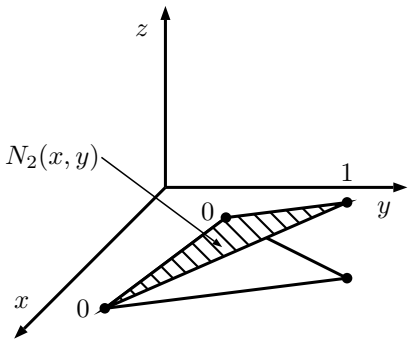
$$N_3(x,y) = \frac{1}{2A_e}\Big((x_1y_2 - x_2y_1) + (y_1 - y_2)x + (x_2 - x_1)y\Big) \qquad (10.3c)$$

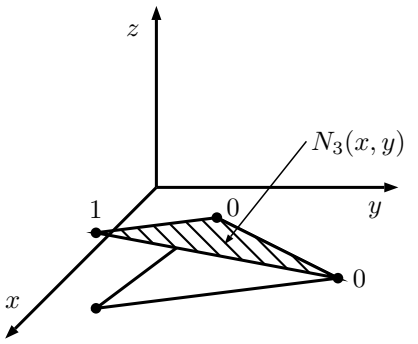
where N_1, N_2, N_3 are the shape functions. Check that the shape functions satisfy the usual properties of shape functions.

These shape functions are:









10.4 Evaluating the element equations

In order to evaluate the element equations, we can also use in multiple dimensions the Method of Weighted Residuals. The idea is to find the required values u_i such that:

(10.4)

where D is the 2-D solution domain and W_i a set of linearly independent weighting functions.

Using the Galerkin method, the weighting functions are replaced by the shape functions such that:

(10.5)

The process is illustrated for the following PDE:

(10.6)

and this PDE is known as the Poisson equation.

If we assume an approximate solution $\tilde{u}(x, y)$, the residuals are defined as:

(10.7)

In order to progress further, we need to integrate by parts to reduce the order of the second derivative $\frac{\partial^2 \tilde{u}}{\partial x^2} + \frac{\partial^2 \tilde{u}}{\partial y^2}$. This is achieved by invoking Green's theorem which says that:

(10.8)

The second integral on the right-hand side is a contour integral over the contour C of the computational domain D. Choosing $N_i(x, y)$ to be zero at the boundary, this second integral is equal to zero and combining Equations 10.5, 10.7 and 10.8 gives:

(10.9)

and this final form is known as the weak form of the governing PDE.

The element equations are obtained by considering Equation 10.9 over a specific element (for example the triangular element $\triangle_{123}$):

(10.10)

Remembering that $\tilde{u}(x,y) = N_1(x,y)u_1 + N_2(x,y)u_2 + N_3(x,y)u_3$ over the element $\triangle_{123}$ and $N_i = N_1, N_2, N_3$, we can build a system of three equations for the three unknowns u_1, u_2, u_3.

$$\begin{aligned} \implies \iint_{\triangle_{123}} \Bigg(& \left(\frac{\partial N_1}{\partial x} u_1 + \frac{\partial N_2}{\partial x} u_2 + \frac{\partial N_3}{\partial x} u_3 \right) \frac{\partial N_1}{\partial x} \\ & + \left(\frac{\partial N_1}{\partial y} u_1 + \frac{\partial N_2}{\partial y} u_2 + \frac{\partial N_3}{\partial y} u_3 \right) \frac{\partial N_1}{\partial y} + \rho N_1 \Bigg) dx dy = 0 \end{aligned} \tag{10.11}$$

Since N_1, N_2, N_3 are linear functions of x and $y \implies \frac{\partial N_1}{\partial x}$, $\frac{\partial N_1}{\partial y}$, $\frac{\partial N_2}{\partial x}$, $\frac{\partial N_2}{\partial y}$, $\frac{\partial N_3}{\partial x}$ and $\frac{\partial N_3}{\partial y}$ are all constants. Equation 10.11 can be calculated explicitly to give:

$$\begin{aligned} \frac{1}{4A_e} \Bigg(& u_1 \Big((y_2 - y_3)^2 + (x_2 - x_3)^2 \Big) \\ & + u_2 \Big((y_3 - y_1)(y_2 - y_3) + (x_3 - x_1)(x_2 - x_3) \Big) \\ & + u_3 \Big((y_1 - y_2)(y_2 - y_3) + (x_1 - x_2)(x_2 - x_3) \Big) \Bigg) + \frac{A_e \rho_e}{3} = 0 \end{aligned} \tag{10.12}$$

Equation 10.12 is of the form:

$$a_{11}u_1 + a_{12}u_2 + a_{13}u_3 = b_1 \tag{10.13a}$$

choosing $N_i = N_2$ in Equation 10.10 yields another equation of the form:

$$a_{21}u_1 + a_{22}u_2 + a_{23}u_3 = b_2 \tag{10.13b}$$

and finally choosing $N_i = N_3$ yields:

$$a_{31}u_1 + a_{32}u_2 + a_{33}u_3 = b_3 \tag{10.13c}$$

Equations 10.13a, 10.13b and 10.13c form the element equations.

10.5 Assembly

The assembly process in multiple dimensions becomes more difficult to implement but the idea is identical to 1-D as described before.

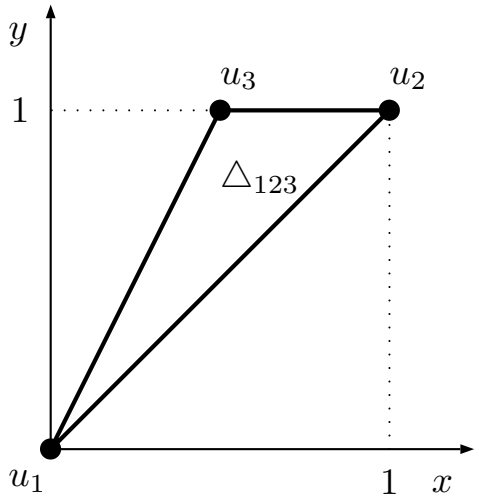
$\Longrightarrow$ We will not attempt to implement the Finite Element method in multiple dimensions. Rather, we are learning about the commercial FE software COMSOL Multiphysics.

10.6 Exercises

Question 1

Consider the element $\triangle_{123}$ below which is such that:

$$u(x_1, y_1) = u(0, 0) = u_1$$
$$u(x_2, y_2) = u(1, 1) = u_2$$
$$u(x_3, y_3) = u(\tfrac{1}{2}, 1) = u_3$$



Assume that over this element, the function $u(x, y)$ is approximated by $\tilde{u}(x, y) = ax + by + c$.

(a) Find the shape functions N_1, N_2, N_3 such that:

$$\tilde{u}(x, y) = N_1(x, y)u_1 + N_2(x, y)u_2 + N_3(x, y)u_3 \tag{10.14}$$

(b) Deduce the value of $\tilde{u}$ at $x = \frac{1}{2}$ and $y = \frac{3}{4}$ if $u_1 = 0$, $u_2 = \frac{1}{2}$ and $u_3 = 1$.

(c) Over the element, the Poisson equation needs to be satisfied such

that:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \rho \tag{10.15}$$

where ρ is a constant. Applying the Galerkin method leads to the equivalent integral statement over the element:

$$\iint_{\triangle_{123}} \left(\frac{\partial \tilde{u}}{\partial x} \frac{\partial N_i}{\partial x} + \frac{\partial \tilde{u}}{\partial y} \frac{\partial N_i}{\partial y} + \rho N_i \right) dxdy = 0 \quad \text{for} \quad i = 1, 2, 3 \tag{10.16}$$

and choosing successively $N_i = N_1$, N_2, N_3, derive the element equations for element $\triangle_{123}$.

Hint: $\iint_{\triangle_{123}} \rho N_i dxdy = \frac{\rho A_e}{3}$

Question 2

(a) Describe the algebraic grid generation and the elliptic grid generation methods for developing structured meshes.

(b) Illustrate the key difference between these two methods using a couple of suitable sketches.

Chapter 11

Consistency, stability and convergence[a]

a *These lecture notes are adapted from "Consistency, Stability, and Convergence" by L.-C. Lee, and also sourced from: "Computational Engineering" by M. Schäfer.*

Numerical methods are typically employed when there is no analytical solution (e.g. using separation of variables) to the equation we are solving (e.g. the Navier-Stokes equations in fluid mechanics). In contrast to analytical methods, the numerical solutions inevitably introduce errors due to approximations. This chapter is focussed on understanding the source and behaviour of these errors for different numerical schemes.

For illustrative purposes, we're going to analyse the 1-D heat equation derived earlier, namely Equation 5.2. Consider the initial-boundary value problem governing the temperature $T(x,t)$ of:

(11.1)

where $\alpha = \frac{k}{\rho c}$ is the heat diffusivity. The boundary and initial conditions are:

(11.2)

11.1 Definitions

Classification of solutions evaluated at a point in space and time, (x_i, t_n):

- Exact solution of the PDE: $T(x_i, t_n)$
- Exact solution of the discretised equation: T_i^n
- Final computed solution from the numerical scheme: $\widehat{T}_i^n$

Types of errors:

- Model error, e.g. mesh does not precisely capture the geometry.
- Discretisation error: $|T(x_i, t_n) - T_i^n|$
- Solution error: $|T_i^n - \widehat{T}_i^n|$
- Total numerical error: $|T(x_i, t_n) - \widehat{T}_i^n|$

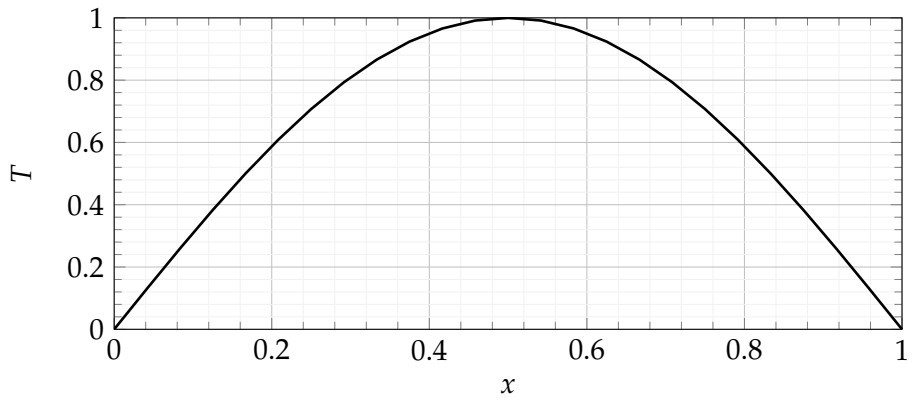
Flow chart for progression of errors:

11.2 Consistency

All numerical schemes involve some form of error arising from the discretisation process, i.e. when partitioning the solution space Δx and time Δt for approximating derivatives. Consistency implies that the discretisation equations and the differential equations are equivalent as $\Delta x \to 0$ and $\Delta t \to 0$.

11.2.1 Example: FTCS numerical scheme

For a demonstration, consider the Forward in Time Central in Space (FTCS) numerical scheme when applied to our problem in Equations 11.1 and 11.2. The initial condition and discretisation of the problem domain is shown below.



Applying the Forward in Time (Euler, Explicit) approximation of the time derivative gives:

(11.3)

and using the Central Difference approximation for the spatial derivative:

(11.4)

then combining yields the discretised equation:

(11.5)

We know that the exact solution to the PDE is not going to satisfy the discretised equation because otherwise $T(x_i, t_n) = T_i^n$. Therefore, substituting the exact solution $T(x_i, t_n)$ into the discretised equation should produce a residual, which we can define as the truncation error:

(11.6)

Using Taylor series expansions about (x_i, t_n), we can derive each of these terms:

(11.7)

(11.8)

(11.9)

Substituting the terms into Equation 11.6 yields:

(11.10)

The FTCS numerical scheme is therefore unconditionally consistent because:

(11.11)

which states that the discretised and continuous equations are equivalent as Δx and Δt becomes infinitesimally small.

The truncation error τ_n also determines the order of the error in the numerical scheme and how fast the error decays as the spatial and temporal step sizes are refined. For example in the FTCS scheme, the consistency order is 1 and 2 with respect to time and space.

11.2.2 Example: Dufort-Frankel numerical scheme

The Dufort-Frankel discretisation method is a classical example of a scheme that is not consistent. The discretisation is given by:

(11.12)

Applying a similar consistency analysis as for the FTCS scheme, we find that the truncation error for the Dufort-Frankel scheme is:

(11.13)

The scheme is only consistent if $\frac{\Delta t}{\Delta x}$ vanishes, i.e. Δt must approach zero faster than Δx when $\Delta t \to 0$ and $\Delta x \to 0$. Therefore, this scheme is only conditionally consistent. Consider $\frac{\Delta t}{\Delta x} = 1$, and now the truncation error is:

(11.14)

which makes the numerical scheme consistent with another PDE (i.e. not the problem we are trying to solve):

(11.15)

11.3 Stability

A numerical scheme is said to be stable if the solution errors (e.g. round-off errors) are bounded for the entire problem domain and for all time steps. The stability analysis of a numerical scheme investigates how the solution error $|T_i^n - \widehat{T}_i^n|$ propagates.

11.3.1 Physical interpretation

From our intuition of heat transfer, we expect the temperature T governed by Equation 11.1 to decrease monotonically over time and eventually reach a steady state of $T(x,t) = 0$ as $t \to \infty$ since both boundaries are fixed at $T = 0$. The analytical solution to this equation (using separation of variables) is:

(11.16)

which matches our physical intuition of T decreasing monotonically over time.

We can also solve this problem numerically and check for the stability of the numerical scheme. For the sake of simplicity, consider a computational domain of just three grid points, which gives one degree of freedom to solve:

(11.17)

As the temperature must be greater than or equal to zero for all time in this problem, then so must both T_2^{n+1} and T_2^n. In order to satisfy this criterion:

(11.18)

Python demonstration
Stability with varying λ.

11.3.2 Von Neumann stability analysis

Although the analysis above provided a guide to understanding the numerical stability, the method was not rigorous. Now we are going to perform a more formal approach using the Von Neumann stability analysis. The solution error ε_i^n was defined earlier as the difference between the final computed solution $\widehat{T}_i^n$ and the exact solution of the discretised equation T_i^n. Therefore, we can rearrange:

(11.19)

Substituting $\widehat{T}_i^n$ into Equation 11.5, the discretised equation, gives:

(11.20)

where the first group of terms are equal to zero because they satisfy Equation 11.5. Therefore, the solution error also satisfies Equation 11.5, which implies that the error propagates in a similar fashion as $\widehat{T}_i^n$, such that:

(11.21)

Recall that the complete Fourier series, including both the cosine and sine terms, are given by:

(11.22)

which is the trigonometric form with a fundamental frequency of $\omega_0 = \frac{2\pi}{T}$. We can also write the Fourier series in exponential form:

(11.23)

where $I = \sqrt{-1}$ is the imaginary unit.

Suppose that the error at time $t = t_n$ is given by:

(11.24)

where k is a constant. The form here represents a single term of the Fourier series representation of the error distributed along the grid points. As the finite difference equation is linear and the Fourier modes are uncoupled, it is sufficient to only study the propagation of this single term. Substituting this form into Equation 11.21 and rearranging yields:

(11.25)

Noting that $e^{Ik(i+1)\Delta x} = e^{Iki\Delta x}e^{Ik\Delta x}$ and $2\cos(k\Delta x) = e^{Ik\Delta x} + e^{-Ik\Delta x}$, we can simplify the above equation which then gives:

(11.26)

which shows that whether the error is magnified between subsequent time steps or not is dependent on the amplification factor γ.

The error will not magnify if the stability criterion $|\gamma| \leqslant 1$ (i.e. $-1 \leqslant \gamma \leqslant 1$) is satisfied, otherwise the error will grow. The expression for γ can be simplified using the double angle formula $\cos(k\Delta x) = 1 - 2\sin^2(\frac{k\Delta x}{2})$ which yields:

(11.27)

The inequality $\gamma \leqslant 1$ requires that $1 - 4\lambda \sin^2\left(\frac{k\Delta x}{2}\right) \leqslant 1 \implies \lambda \geqslant 0$. Although because $\Delta x \geqslant 0$ and $\Delta t \geqslant 0$, the condition $\lambda \geqslant 0$ is unconditionally satisfied. Moreover, the inequality $\gamma \geqslant -1$ requires that $1 - 4\lambda \sin^2\left(\frac{k\Delta x}{2}\right) \geqslant -1 \implies \lambda \leqslant \frac{1}{2\sin^2\left(\frac{k\Delta x}{2}\right)}$. The smallest possible value of $\frac{1}{2\sin^2\left(\frac{k\Delta x}{2}\right)} = \frac{1}{2}$, and therefore we have arrived at the same stability criterion as we found earlier with using our physical intuition:

(11.28)

In summary, for the FTCS numerical scheme to be stable, the amplification factor γ must satisfy the stability criterion:

(11.29)

11.4 Convergence

The convergence of a numerical scheme is defined as where the final computed solution $\widehat{T}_i^n$ approaches the exact solution of the PDE $T(x_i, t_n)$ as the spatial and temporal grids are refined:

(11.30)

Lax's equivalence theorem states that given a well-posed linear initial value problem using a consistent numerical scheme, the method is convergent if and only if it is stable.

Therefore the convergence of a numerical scheme can be determined by separately considering whether the scheme is: (1) consistent, i.e. the truncation error vanishes for an infinitesimally resolved spatial and temporal grids; and (2) stable, i.e. the solution error remains bounded. This theorem only holds for linear problems, although we have solved several linear PDEs in this course, and often non-linear PDEs are solved using linearisation.

11.5 Exercises

Question 1

Apply the consistency analysis for the Dufort-Frankel numerical scheme, which has the discretisation given by:

$$\frac{T_i^{n+1} - T_i^{n-1}}{2\Delta t} - \alpha \frac{T_{i+1}^n - (T_i^{n+1} + T_i^{n-1}) + T_{i-1}^n}{\Delta x^2} = 0 \tag{11.31}$$

and is second order accurate in space and time.

Question 2

Show that the Backward in Time Central in Space (BTCS) numerical scheme shown below is unconditionally stable.

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} - \alpha \frac{T_{i+1}^{n+1} - 2T_i^{n+1} + T_{i-1}^{n+1}}{\Delta x^2} = 0 \tag{11.32}$$

Chapter 12

Optimisation methods[a]

a *Extra resource for those who are interested: recorded lectures by Dr Julius Pfrommer, KIT:* `https://www.youtube.com/watch?v=4_jiFQXPAsw`.
Additional reading: Stephen Boyd and Lieven Vandenberghe. Convex Optimization. Cambridge University Press, 2004.

Determining the optimal solution for a given problem is a common task for engineers, and in this section we cover some fundamental principals and concepts for optimisation methods.

Types of optimisation problems in engineering:

- Cost optimisation of manufacturing a car.
- Performance optimisation of satellite trajectory.
- Trade-off between cost and performance.

Examples of optimisation in computer vision:

- Stereo matching
- Image denoising
- Image deblurring

these problems typically use an energy function to describe the solution quality, and are very high-dimensional.

Examples of optimisation in nature:

- Aerodynamics of wings on birds (e.g. minimise drag)
- Soap film and bubbles

- Protein folding

and these processes are naturally driven by minimum energy states and the principle of least action.

12.1 Definitions

The variables (free parameters) $\vec{x}$ must belong to some appropriate solution space $\vec{x} \in X$. Formally, we can write our optimisation problems:

(12.1)

where $\vec{x}^*$ is the optimum set of variables, and is called the minimiser.

In other words, we seek to minimise $\mathcal{F}(\vec{x})$ on the domain X in order to find $\vec{x}^*$ which yields the minimum, i.e. $\mathcal{F}(\vec{x}^*)$. We know that $\vec{x}^*$ corresponds to a minimum if $\nabla\mathcal{F}(\vec{x}^*) = 0$ and $\nabla^2\mathcal{F}(\vec{x}^*) \geqslant 0$ (provided $\mathcal{F}$ is twice differentiable).

Gradient and partial derivatives

The gradient of a function $\mathcal{F}(\vec{x})$ is given by a column of partial derivatives:

$$= \left\{ \quad \right\} \tag{12.2}$$

which can be used to find the tangent plane (first degree Taylor expansion).

Hessian matrix

The second partial derivatives of a function $\mathcal{F}(\vec{x})$ gives the Hessian matrix:

$$= \left[\quad \right] \tag{12.3}$$

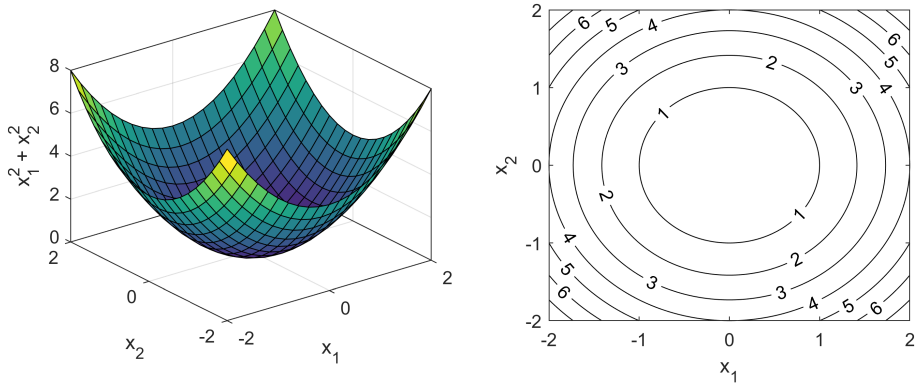
which can be used to form a second degree Taylor expansion (e.g. parabolic shape). Note: the order of differentiation does not typically matter (see Schwarz's Theorem), and the Hessian is symmetric, i.e. $\nabla^2\mathcal{F}(\vec{x}) = \left(\nabla^2\mathcal{F}(\vec{x})\right)^{\mathrm{T}}$.

Visualising objective functions

Consider several objective functions with only a single variable $\mathcal{F}(x_1)$:

12.2 Convexity

If the objective function is strictly convex then there is only one minimiser, which means we'll find it by simply walking downhill. For example, $\mathcal{F}(\vec{x}) = x_1^2 + x_2^2$:



Convexity of sets

A set X is convex if for any two points within the set, the set contains the entire line segment between those two points. The convex hull of a set is the set of all convex combinations of the points within X.

Convexity of functions

A function $\mathcal{F}$ which maps from an input set X is convex if X is convex and any of the following equivalent conditions are met:

- The epigraph of $\mathcal{F}$ is a convex set.

Python demonstration
Jensen's inequality

- Jensen's inequality is satisfied for all $\vec{\boldsymbol{x}}_a, \vec{\boldsymbol{x}}_b \in X$ and $0 \leqslant \theta \leqslant 1$.

$$\mathcal{F}(\theta\vec{\boldsymbol{x}}_a + (1-\theta)\vec{\boldsymbol{x}}_b) \leqslant \theta\mathcal{F}(\vec{\boldsymbol{x}}_a) + (1-\theta)\mathcal{F}(\vec{\boldsymbol{x}}_b) \tag{12.4}$$

- The function $\mathcal{F}$ is twice differentiable and the Hessian $\nabla^2\mathcal{F}(\vec{\boldsymbol{x}})$ is positive semi-definite for all $\vec{\boldsymbol{x}} \in X$.

Examples

The function $\mathcal{F}(x_1) = x_1^2$ is convex because $\mathcal{F}''(x_1) = 2 > 0$, whereas the function $\mathcal{F}(x_1) = x_1^3$ is neither convex nor concave because $\mathcal{F}''(x_1) = 6x_1$ which is positive for $x_1 > 0$ (convex) and negative for $x_1 < 0$ (concave). A linear combination of variables are convex, i.e. $\mathcal{F}(\vec{\boldsymbol{x}}) = \vec{\boldsymbol{a}}^{\mathrm{T}}\vec{\boldsymbol{x}} + b$ where $\vec{\boldsymbol{a}}$ is a vector of coefficients and b is a scalar.

12.3 Brute force approaches

Of course we could in theory find the optimum solution by checking every possible combination of variables and selecting the best one. However, quite often in practice, this approach is impractical and beyond computationally expensive.

A couple of naive but systematic methods for finding the optimum solution are:

- Monte Carlo experiments where the free parameters $\vec{x}$ are randomly chosen and the objective function evaluated at each iteration.
- Grid search where the parameter space is discretised into a grid of points (i.e. a parameter sweep) which are the locations that the objective function is calculated.

12.4 Steepest descent

The concept and motivation of the steepest descent method is that we will improve our solution by walking down the hill in the steepest direction, $-\nabla\mathcal{F}(\vec{x})$. The algorithm is an iterative method starting from an initial point $\vec{x}^{(0)}$ and stepping to the next point given by:

(12.5)

where h is the step factor. This process is repeated until the magnitude of the gradient reaches some tolerance, $||\nabla\mathcal{F}(\vec{x}^{(k)})|| < \varepsilon$. However, convergence is not guaranteed. What happens for large h? What is the definition of a large h?

Python demonstration
Steepest descent

12.5 Line search

For ensuring convergence with the steepest descent method, we need to use an additional line search to check whether the next step is actually an improvement. Suppose that we label the descent step direction as $\vec{\boldsymbol{d}} = -\nabla\mathcal{F}(\vec{\boldsymbol{x}})$, then we ideally would choose a step factor h given by:

(12.6)

which would give us the optimum solution in the direction of steepest descent. However that would require evaluating $\mathcal{F}(\vec{\boldsymbol{x}})$ along many points on this line, and in practice, the selection of h is instead chosen using heuristics.

One simple and efficient heuristic method is the backtracking line search. Starting with an initially large step factor, iteratively reduce h (e.g. halving with $h = ph$ using $p = 0.5$) until the minimum descent steepness criteria is reached (namely the Armijo condition):

(12.7)

where $0 < \beta < 1$ and is commonly $\beta \approx \mathcal{O}(10^{-4})$. Small $\beta \approx 0$ means that any descent is valid (possibly only small improvements on $\mathcal{F}(\vec{\boldsymbol{x}})$), whereas large $\beta \approx 1$ means that the descent needs to be almost as steep as the gradient (yielding tiny step sizes).

Python demonstration
Line search

12.6 Imposing constraints

The first two types of optimisation problems in engineering we discussed earlier were subjected to constraints (either the performance exceeding some requirements, or the cost being within a budget). Although, we're only studying optimisation methods for unconstrained problems. However, we can transform such a constrained problem (with a hard inequality constraint) with a soft constraint by adding a penalty term to the objective function:

(12.8)

where $\mathcal{P}(\vec{x})$ is a penalty term and weighted by a factor α. For example, the constraint $x - 5 \leqslant 0$ can be applied with $\mathcal{P}(\vec{x}) = \max(0, x - 5)$. This penalty can be made more severe by squaring, e.g. $\mathcal{P}(\vec{x}) = \max(0, x - 5)^2$ (known as a quadratic loss function). Although now we may have introduced a bias to the position of the optimal solution! The application of a penalty term has a similar effect to using Tikhonov regularisation (typically used for smoothing or regularising ill-posed problems).

12.7 Simulation-driven design

The classical engineering design process involves: (1) planning; (2) concept design; (3) preliminary design; (4) detailed design; (5) verification; and (6) production. Typically, the numerical simulations (e.g. CFD, FEA) are performed during the detailed design and verification stages. The concept of simulation-driven design is that using the simulation technology earlier in the design process. For example during the early exploration phase, topological optimisation methods could be used to study a wide range of distinct designs that may be unintuitive to experienced practitioners. After a rough design has been selected, a more detailed optimisation study could be used to finalise the product.

12.8 Exercises

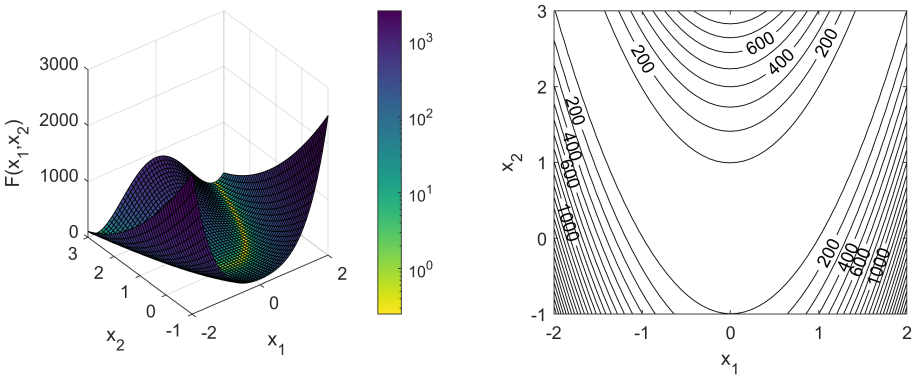
Question 1

We are going to test our steepest descent algorithm on the Rosenbrock function[a] (also known as the banana function) which is given by:

$$\mathcal{F}(\vec{x}) = (a - x_1)^2 + b(x_2 - x_1^2)^2 \tag{12.9}$$

where $\vec{x} = (x_1, x_2)$ are the variables, and $a = 1, b = 100$ are some typical parameters. This function is plotted below and has a global minimum at $(1, 1)$.

For more examples of test functions to use for verifying optimisation methods, check `https://en.wikipedia.org/wiki/Test_functions_for_optimization`. *The Himmelblau's function is an example of a function with multiple minima; which one would our steepest descent method reach?*



(a) Comment on the convexity of this function by first analysing the plots above, and then second by using the Hessian matrix $\nabla^2 \mathcal{F}(\vec{x})$.

(b) Starting with $\vec{x}^{(0)} = (-1, 2)$, try searching for the global minimum using a step factor $h = 1$ and explain the problem. Next, try $h = 10^{-3}$ and determine the number of iterations required to reach within a distance of 10^{-3} from the minimum.

(c) Now return to a step factor $h = 1$, and using a line search to ensure convergence, again determine the number of iterations required and then compare to your earlier results.

Appendix A

Introduction to COMSOL

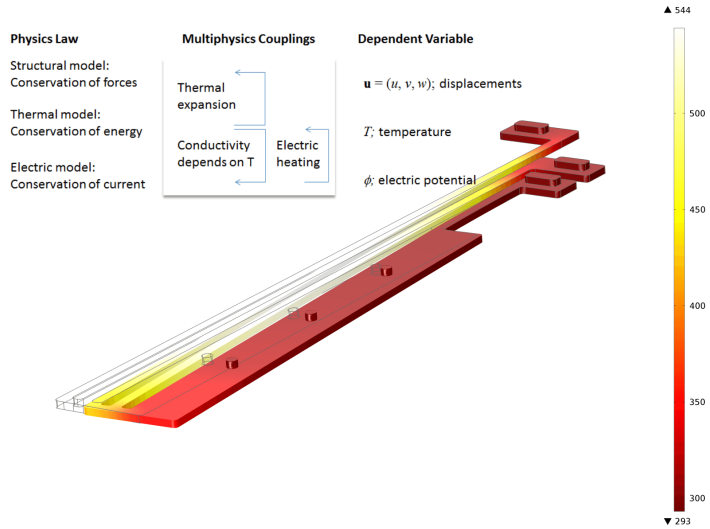


Figure A.1: A microelectromechanical systems (MEMS) actuator uses the joule heating effect to create thermal expansion of the two hot arms (white and yellow colours) that carry an electric current. The thermal expansion creates a structural displacement that moves the actuator. This is a typical case of multiphysics in FEA. Source: `https://www.comsol.com/`.

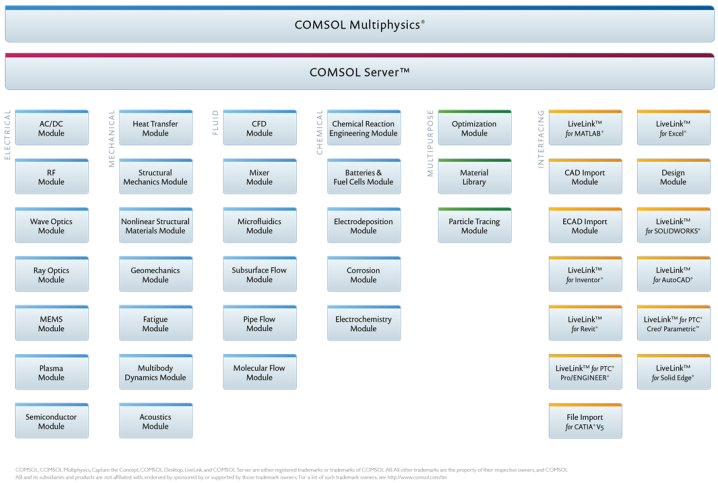
A.1 About COMSOL Multiphysics

COMSOL is a powerful Finite Element simulation platform:

See `https://www.comsol.com/multiphysics/finite-element-method` for extra reading on the FE method.

- Many modules expand the simulation platform with dedicated user interfaces and tools for optimisation, electrical, mechanical, fluid flow, and chemical applications.
- Enables cross-disciplinary product development and a unified simulation platform.
- COMSOL has a comprehensive material library which builds on the already powerful material definitions available in the core product and the add-on modules.
- Many interfacing products connect COMSOL Multiphysics simulations with spreadsheet, technical computing, CAD, and ECAD.

Figure A.2: COMSOL product suite.



A.2 Modelling workflow

1. Pre-processing: includes definitions (parameters, user-defined functions), geometry, materials, physics, and mesh.
2. Solution: select a solver with appropriate settings.
3. Results: analyse and report using data sets, tables and plots.

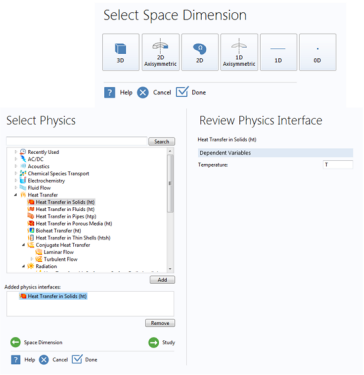


Figure A.3: Model Wizard, available when creating a new model.

A.2.1 Model wizard

At the start of the modelling process, the Model Wizard assists in selecting the right physics by selecting from a tree that contains all physics interfaces. The available physics interfaces will depend on the supported license and selected dimension (for example, 3-D, 2-D axisymmetric, 2-D, 1-D axisymmetric, 1-D, or 0-D).

A.2.2 Physics interfaces

COMSOL Multiphysics and the physics-based add-on modules consist of physics interfaces appropriate for the physics they support:

- A physics interface is a dedicated user interface for a particular area of physics such as solid mechanics, laminar flow, heat transfer in solids, or electrostatics.
- Physics interfaces also include studies/solver types, and analysis, visualisation, and post-processing tools specific to the physics being simulated.

A set of fundamental physics interfaces is available in the COMSOL Multiphysics core product. The add-on modules contain additional and more sophisticated physics interfaces.

A.2.3 Study

The Model Wizard will also assist you in selecting a Study node for your physics or multiphysics combination. Each Study node consists of a solver or a set of solvers that is adapted to your selected physics. Later on in the modelling process, the COMSOL Desktop environment allows you to change and optimise the solvers settings.

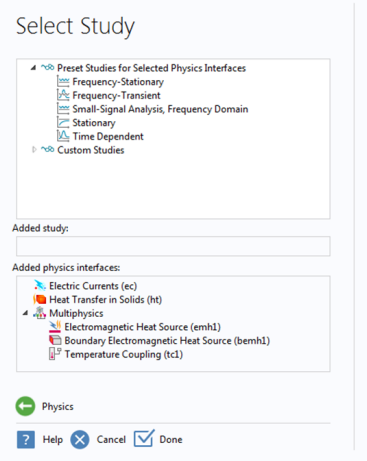


Figure A.4: Study selection within the Model Wizard.

A.2.4 Model tree

The Model Builder window, which includes the model tree, gives an overview of the model and all the functionality and operations needed for building and solving a model as well as processing the results.

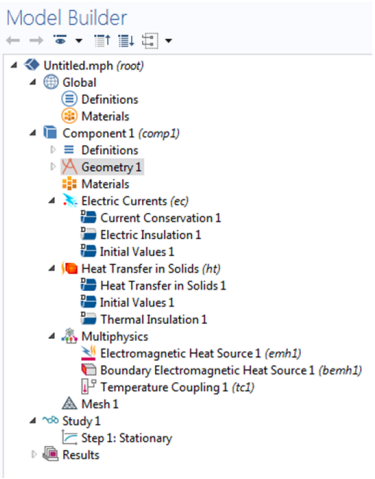


Figure A.5: The model tree as it appears with the Joule Heating multiphysics interface. The Multiphysics node contains the necessary physics couplings for Joule Heating.

A.2.5 Geometry modelling

The core product includes geometry modelling for 3-D, 2-D, and 1-D:

- Hybrid modelling: combine solids, surfaces, curves, and points.
- Boolean operations: union, intersections, and difference.
- Extrude and revolve 2-D profiles from Work Planes.

A.2.6 Meshing

The following meshing capabilities are included:

- Automatic tet mesher: with physics-based settings or manual override.
- Semi-automatic swept: meshing and mapped meshing.
- Boundary layer: mesh for CFD.
- Adaptive meshing: with built-in and user-defined error estimation.
- Moving mesh: with the Arbitrary Lagrangian-Eulerian (ALE) method.
- Automatic mesher: for several physics interfaces, adapts the mesh density and type with respect to the boundary conditions.

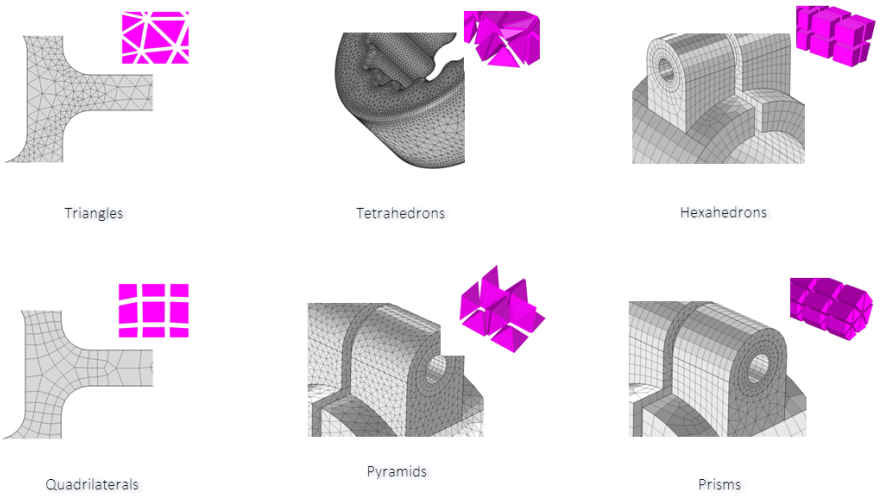


Figure A.6: Various styles of mesh elements: triangles, tetrahedrons, hexahedrons, quadrilaterals, pyramids, and prisms.

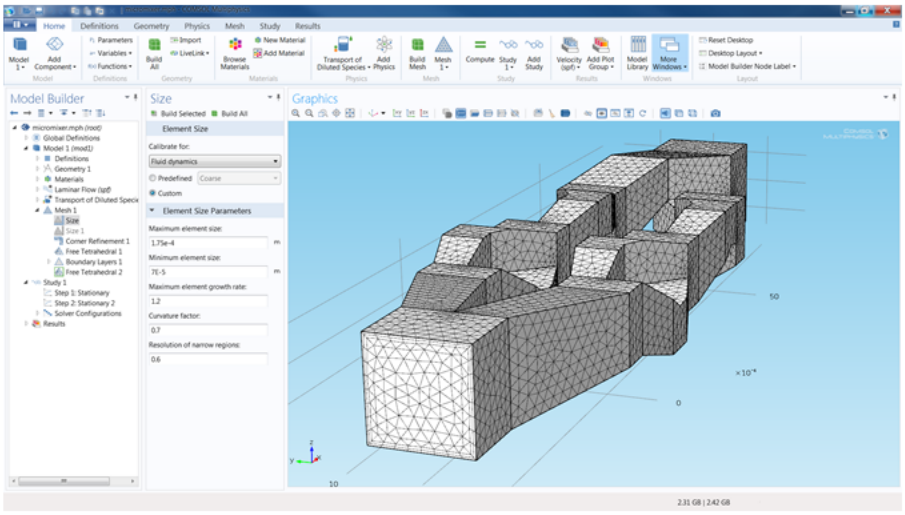


Figure A.7: Meshing for a CFD simulation with an automatically created hybrid mesh consisting of a prismatic boundary layer mesh close to the walls and a tet mesh in the bulk.

A.2.7 Materials

Key features of the materials (fluids or solids):

- A small material library is included in the core product.
- Many of the physics-based add-on modules provide additional material libraries.
- The Material Library product contains ≈ 2500 materials including many alloys with temperature-dependent properties.
- Materials can be spatially varying (inhomogeneous), anisotropic, and discontinuous.
- A special physics interface is available to define material coordinate systems that follow curved shapes – important when defining composite structures.
- Materials can be non-linear and have properties that depend on field quantities and their derivatives.

A.2.8 Physics setting

The model tree can contain one or more physics interfaces with settings for volumetric domains, boundary conditions, conditions on edges and at points.

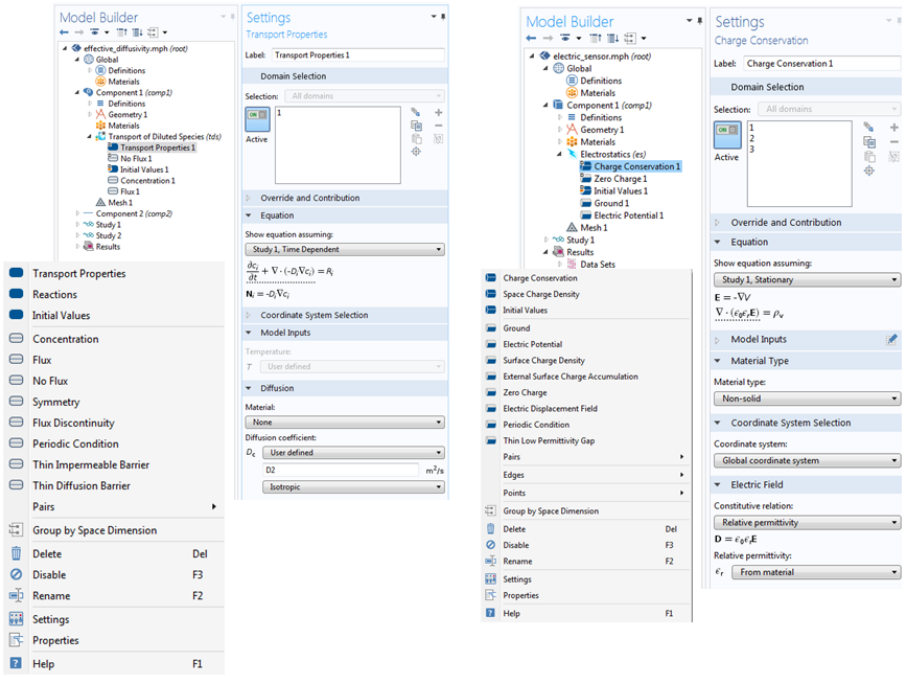


Figure A.8: Left: transport of diluted species. Right: electrostatics.

A.2.9 Study

A model may contain one or more `Study` nodes. Each `Study` holds a solver configuration including settings for parametric sweeps, solver types, etc. A `Study` may have one or more study steps to create a composite solver configuration, for example a `Stationary` solver step followed by a `Time-Dependent` solver step. Automatic solver suggestions are included when you choose physics interfaces from the `Model Wizard`. For standard simulation tasks you do not need to modify the low-level solver settings.

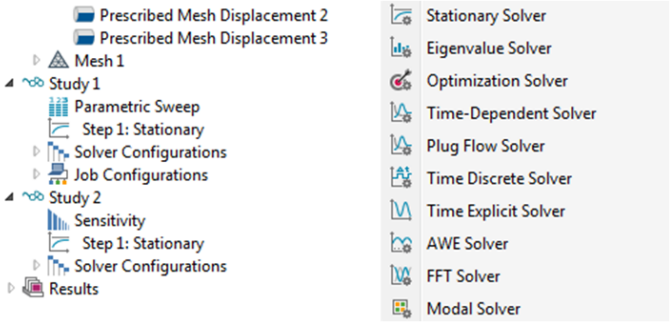


Figure A.9: `Study 1` contains a parametric sweep of stationary simulations, and `Study 2` has a sensitivity analysis on a stationary simulation.

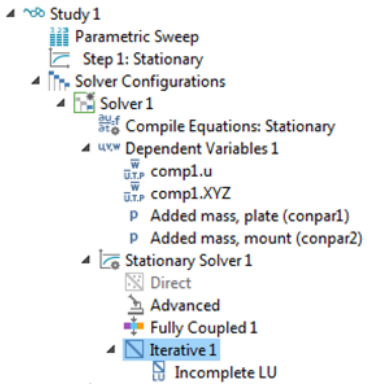


Figure A.10: Low-level study settings.

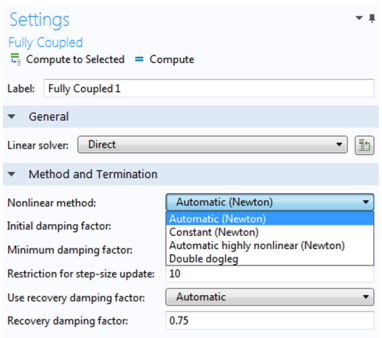


Figure A.11: Nonlinear solvers.

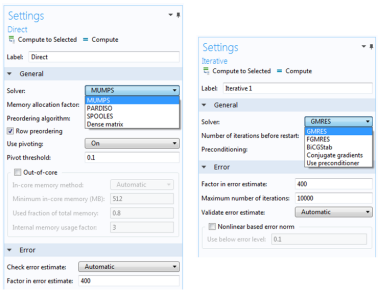


Figure A.12: Direct and iterative solvers.

Figure A.13: Cooling of a heat sink with air flow.

A.2.10 Low-level study settings

A Study node can be expanded to access low-level settings. An outer level may consist of, for example, a non-linear solver or an eigenvalue solver. Solvers can be fully coupled or segregated. The lowest-level settings control which solvers are used for the linearised system matrices.

The system matrix solvers are divided into:

- Direct solvers
- Iterative solvers

A.2.11 Non-linear solvers

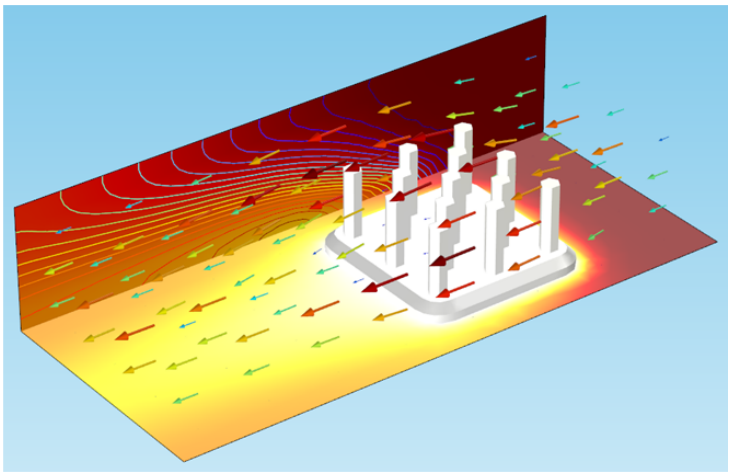
COMSOL automatically detects non-linearities and uses a non-linear solver when needed. Optionally, by using the Fully Coupled solver option you can take control over the non-linear solver settings and select between different Newton methods and a Double dogleg method.

A.2.12 Direct and iterative solvers

Direct solvers are robust but consumes more memory than iterative solvers. Iterative solvers may need tuning for optimal performance but can be very memory-conservative.

A.2.13 Visualisation

The core product includes advanced visualisation tools including: volume plots, surface plots, contour and isosurface plots, streamline/flux-line, tube, and ribbon plots, arrow plots, mesh plots, animations, 1-D, 2-D, and 3-D plot types. Multiple plot types can be combined into one visualisation.



A.2.14 Reports

Simulation reports can be generated to Word® and HTML formats. Reports are available at several levels of detail: Brief, Intermediate, Complete, and Custom.

A.3 Documentation, help and examples

Documentation set:

- The COMSOL Multiphysics core product as well as the add-on products provide `Documentation` in PDF and HTML formats.
- Access via `File > Help > Documentation (Ctrl + F1)`.

Dynamic help:

- The `Help` window provides context-dependent help texts about windows and model tree nodes.
- Access via `File > Help > Help (F1)`.

Model application libraries:

- The `Application Libraries` are collections of MPH-files with accompanying documentation that includes the theoretical background and step-by-step instructions.
- You can use the step-by-step instructions and the MPH-files as a template for your own modelling and applications.

Making effective use of documentation and the model libraries:

- The COMSOL reference manual (1742 pages) is not necessarily the most effective starting point in working out how to model a new problem.
- It is usually more helpful to find and work through similar problems from the various model libraries.
- Model libraries exist for each type of physics under multiphysics, but also under the separate application modules.
- You may need to work through a number of different models to cover different aspects of your problem... one for the geometry or meshing, one for the physics and boundary conditions... etc.
- The user manual is then useful for clarifying details on specific commands or features that are not clear from the worked problems.

- You can also use the SEARCH option to search through the entire set of COMSOL documentation for a particular key word.

COMSOL Learning Center:

- Additional resource for learning the software by watching demonstrations.
- Online videos: `https://www.comsol.com/learning-center`

A.4 Demonstration on solving a simple PDE

Consider the Laplace equation:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0 \tag{A.1}$$

on a $L \times H$ rectangular computational domain. This equation is subject to the following boundary conditions:

$$\left.\frac{\partial T}{\partial x}\right|_{x=0} = 0 \tag{A.2}$$

$$\left.\frac{\partial T}{\partial y}\right|_{y=0} = 0 \tag{A.3}$$

$$T(x, y = H) = x \tag{A.4}$$

$$T(x = L, y) = y \tag{A.5}$$

Using a structured mesh, display the heat flux $\vec{q} = -k\nabla T$ with an Arrow Surface.

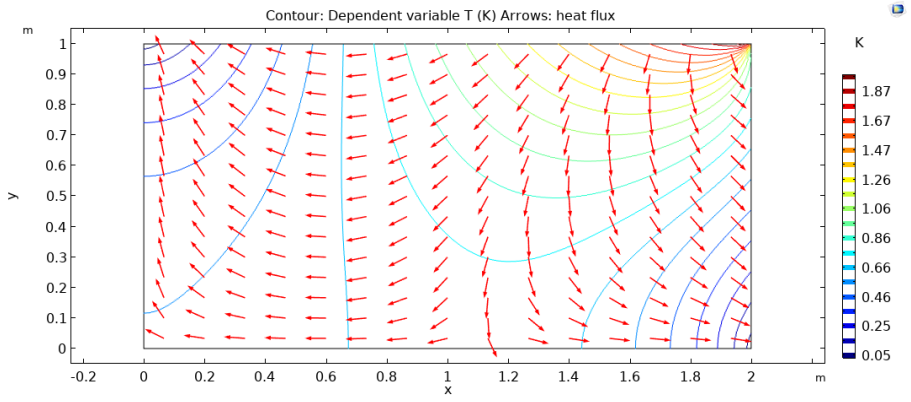


Figure A.14: Solution to Equation A.1.

Appendix B

Geometry modelling[a]

a *Sources for the following two sections: COMSOL user guide, "Computational Engineering" by M. Schäfer, and "An introduction to CFD" by Tu, Yeoh, and Liu*

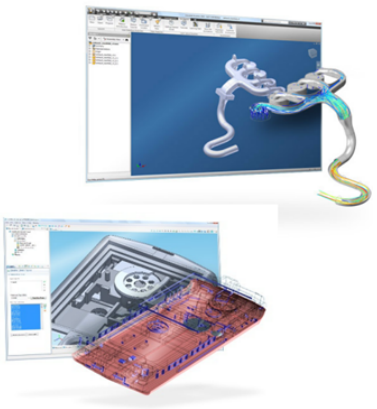
B.1 Introduction

Creating the model geometry is the first step in a simulation. The purpose of the geometric model are threefold:

1. Describe the physical shape of the object on which material properties, load, and constraints can be defined.
2. The geometric model is used to create the mesh.
3. The geometrical model allows the parametric investigation of shape variations.

B.2 Importing CAD geometries into COMSOL

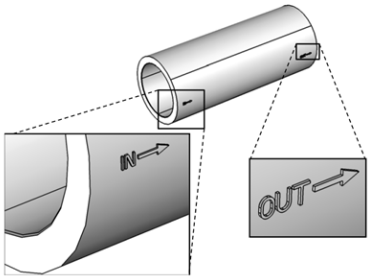
Live Links and CAD import:



- Useful for when using other software to create geometries
- LiveLinks for SolidWorks, Inventor, and Pro/E
  - Bidirectional updates of geometry dimensions
- CAD Import Module supports the following formats
  - Autodesk Inventor assembly (.ipt, .iam)
  - Pro/ENGINEER (.prt, .asm)
  - SolidWorks (.sldprt, .sldasm)
  - Parasolid (.x_t, xmt_txt, .x_b, xmt_bin)
  - ACIS (.sat, .sab), STEP (.step), IGES (.iges)

B.3 Considerations for modelling

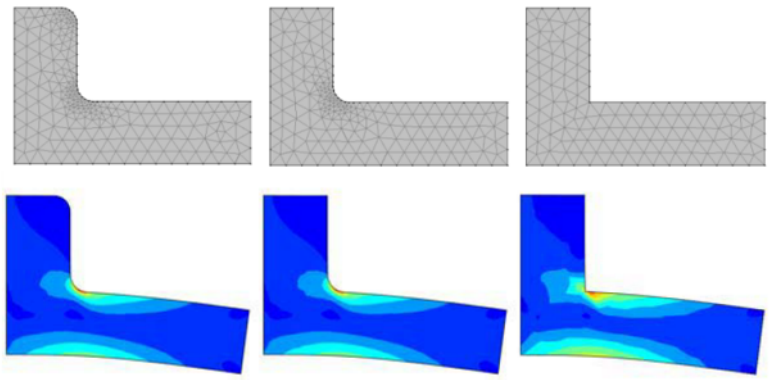
B.3.1 Deciding what to model

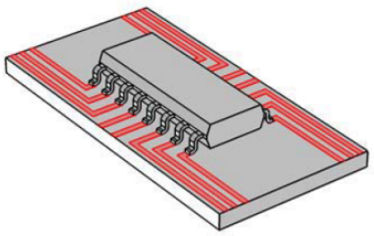


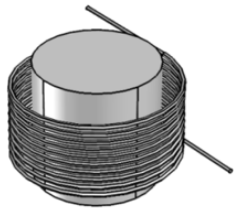
- A CAD model that is useful for analysis is different from the fully detailed CAD model used for manufacturing purposes.
- The CAD model for analysis need only contain enough geometric detail to obtain accurate analysis results.
- It is important not to oversimplify the CAD model.

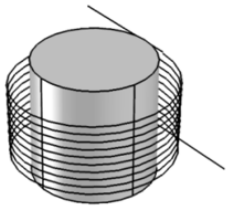
Example of a fillet in structural analysis:

- A simple structural element with fillets on the outside and inside corners (left) showing the computed stresses.
- The fillet on the outside corner can be removed to simplify the model without affecting the computation of the peak stresses, or the peak deflections in any significant way (middle).
- If the fillet on the inside corner is removed, then the stresses are not computed correctly.









- Automatic meshing tools (usually) give meshes that model accurately the geometry rather than the physics. Knowledge of the physics must often be added to create an efficient mesh.

B.3.2 Geometry aspect ratio

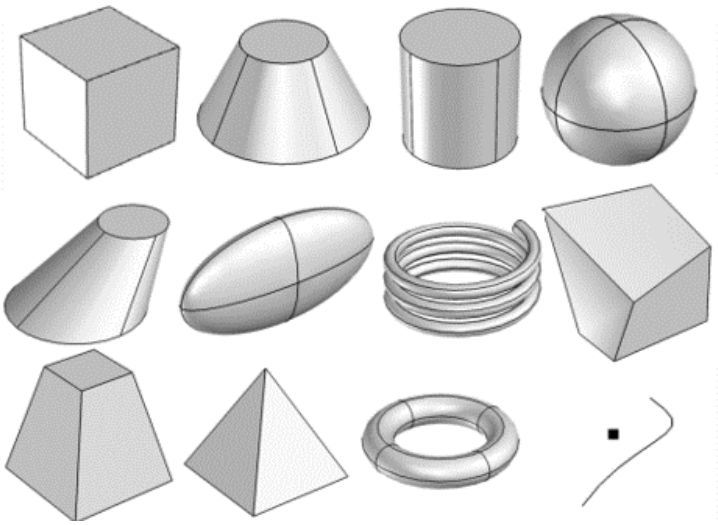
- In practice, for reasons concerning meshing and memory requirements, always explore 2-D and 1-D modelling of 3-D problems.
- For example, model the traces on circuit boards as 2-D surfaces (instead of as volumes) in which state variables do not vary through the thickness.

- Furthermore, model the wire winding around a cylinder in 1-D if the dependent variables (e.g. temperature, current) varies only with distance along the wire.
- Bar, beam, plane stress and plate/shell structures are cases in solid mechanics where the geometry needed for the analysis can be reduced to one or two dimensions.

B.4 How to build a model

B.4.1 Building blocks

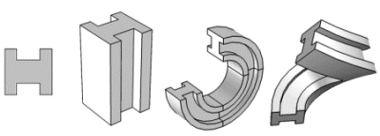
- Operations used in geometry creation can be thought of as adding, removing, or subdividing, lumps of clay.
- COMSOL provides a set of basic 3-D primitives: block, cone, cylinder, sphere, eccentric cone, ellipsoid, helix, hexahedron, pyramid, tetrahedron, torus, point, and curve.



- A collection of these primitives can be used to represent the true shape of your part.

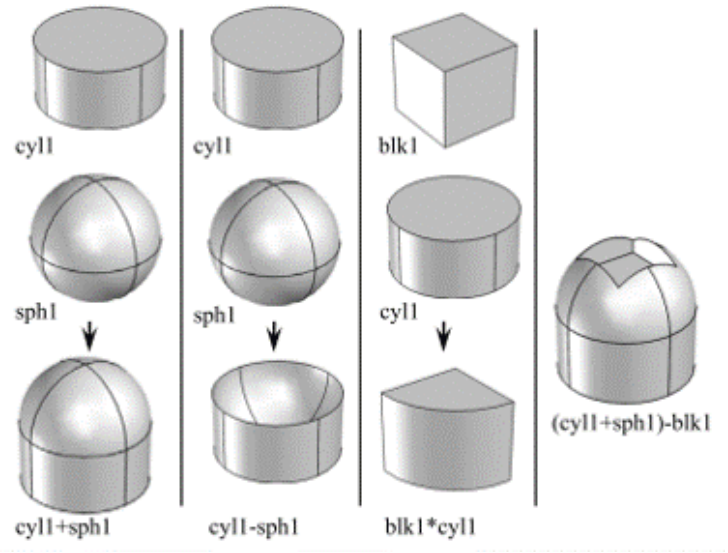
B.4.2 Non-primitive geometries

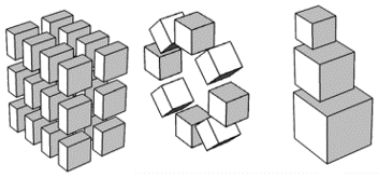
- For more complex shapes, it is also possible to use a 2-D sketch of an outline, and extrude, revolve, or sweep that outline along a path.
- Sketch the outline on a Workplane before applying the extrude, revolve, or sweep operations.



B.4.3 Boolean operations

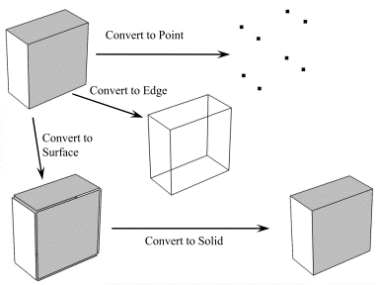
- Once a set of primitives (or other bodies) are defined, it is possible to perform a sequence of boolean operations on them.
- For example, apply a union, difference, intersection, or compose (several Boolean operations) of two or more bodies.



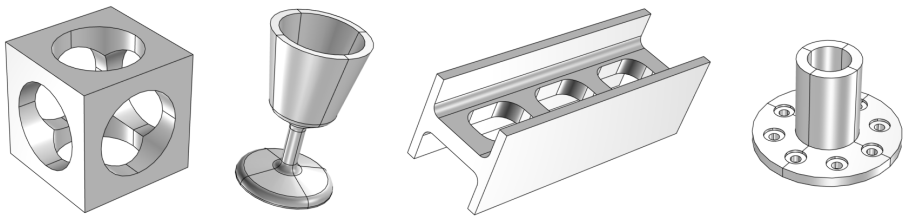


B.4.4 Transformations

- Objects defined from the above operations can also be moved, scaled, copied, and rotated.
- A set of domains can be converted to its constitutive boundaries, edges, or points. Conversely, a set of boundaries can be converted to a domain.
- The final step within the COMSOL geometry sequence is, by default, a Form Union operation. This performs a Boolean union of all objects within the geometry sequence.



B.4.5 Examples



COMSOL demonstration

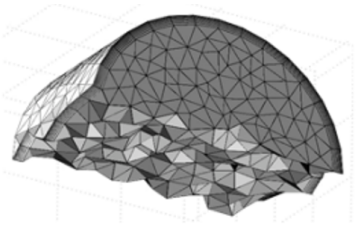
Geometry creation examples.

Appendix C

Fundamentals of mesh generation

C.1 Introduction

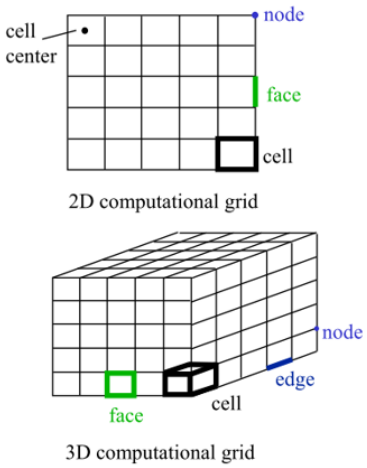
The purpose of a finite element mesh is to subdivide a complex geometry up into smaller pieces, or elements, which can be used to solve the physics within the geometry. A mesh is used to represent the structure and to compute the solution.



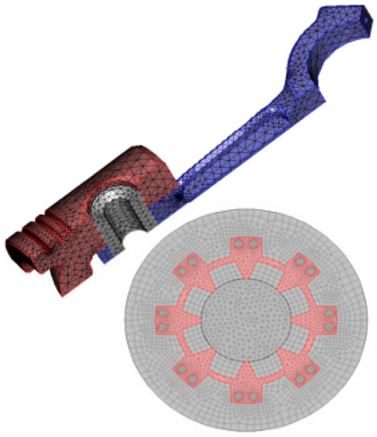
C.1.1 Importance of the mesh

- Grid has a significant impact on:
  - Rate of convergence (or even lack of convergence)
  - Solution accuracy
  - CPU time required
- Importance of mesh quality for good solutions:
  - Grid density
  - Adjacent cell length/volume ratio
  - Skewness
  - Boundary layer mesh
  - Mesh refinement through adaptation

C.1.2 Mesh terminology

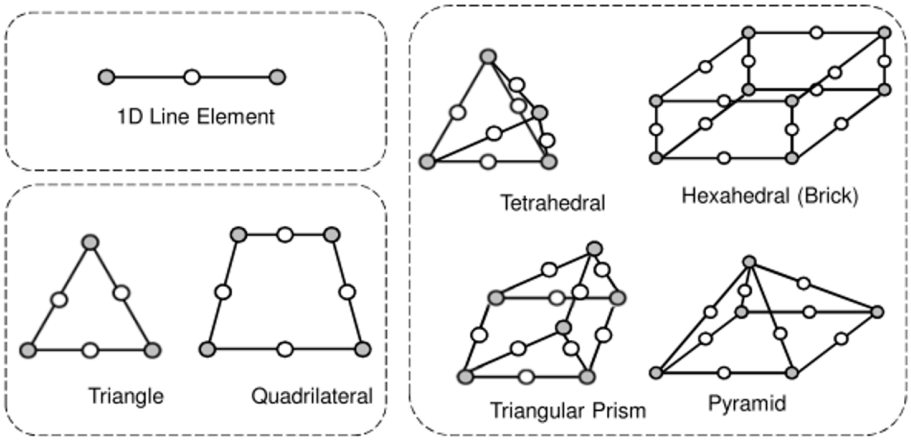


- Cell: control volume into which the domain is broken up (also known as an element).
- Node: grid point.
- Cell centre: centre of a cell.
- Edge: boundary of a face.
- Face: boundary of a cell.
- Zone: grouping of nodes, faces, and cells.
  - Wall boundary zone
  - Fluid cell zone
- Domain: group of node, face and cell zones.



C.1.3 Element types

- 1-D: discretised domains into intervals.
- 2-D: triangular and quadrilateral.
- 3-D: tetrahedral, hexahedral, prism, and pyramid.
- A mesh can be built from a combination of various element types.

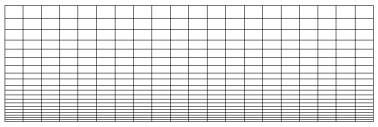


All of these elements have midpoint nodes. The variation of the dependent variable (e.g. displacement, temperature, acoustic pressure) is quadratic with x, y, or z. These are often the default elements used in COMSOL and in many other FE codes.

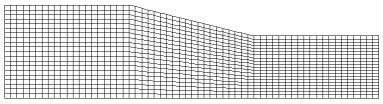
C.2 Mesh types

C.2.1 Structured mesh

- There are directions along which the number of grid points is always the same, i.e. grid lines must pass throughout the domain.
- Can use i, j, k indexing to locate neighbouring cells.
- Advantage: easy to generate, low data storage and management requirement, leads to well-behaved algebraic systems of equations.
- Disadvantage: structured grids do not handle complex geometries well, and do not easily lend themselves to automatic grid generation.



Structured Cartesian mesh: the problem is covered by a regular grid.

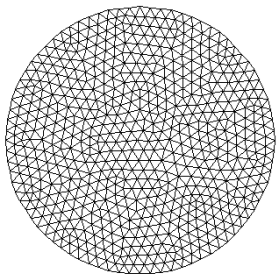


Structured boundary-fitted mesh: all boundary parts of the problem are approximated by grid lines — preserves boundary integrity.

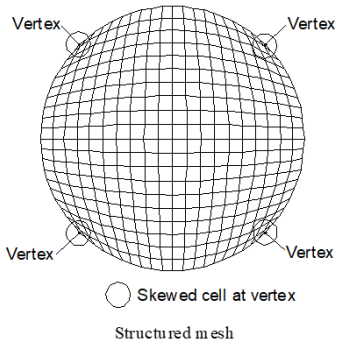
C.2.2 Unstructured mesh

- No regularity in the arrangement of grid cells and points.
- Advantage: the possibility of distributing the grid cells arbitrarily across the problem domain provides the best flexibility for accurate modelling of the geometry.
- Disadvantage: the data structure which relates one mesh point to its neighbour is much more complicated and needs to be stored.

$\Longrightarrow$ As a general guideline, one should always try to generate a grid as structured as possible!



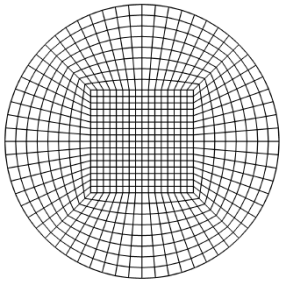
Unstructured mesh



Structured mesh

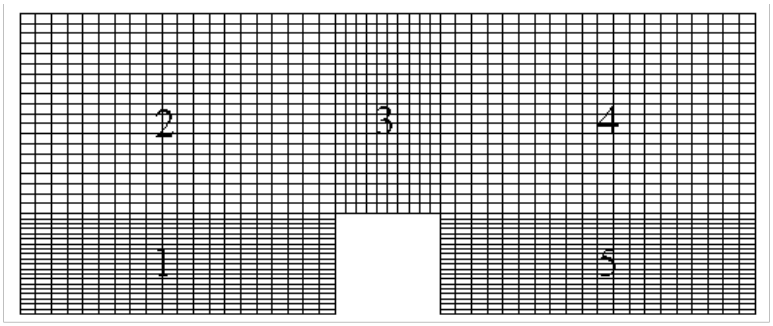
C.2.3 Example: mesh of a circle

- A single structured grid produces a mesh with skewed elements at the vertices.
- An unstructured grid can have a uniformly distributed set of elements (similar sizes and aspect ratios).
- Splitting the circle into five regions and then applying a structured grid to each region, provides a quality structured mesh (often referred to as an O-H type grid).



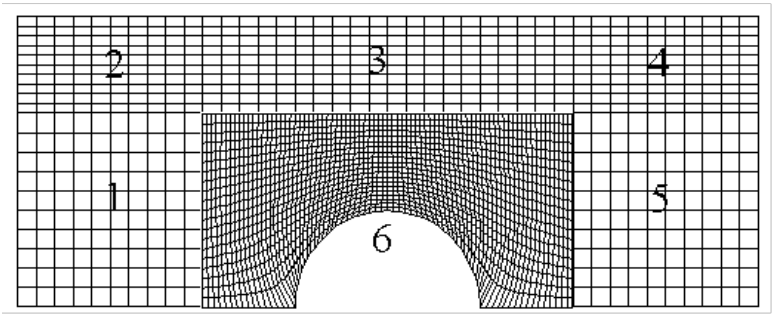
O-H type grid

C.2.4 Other mesh types



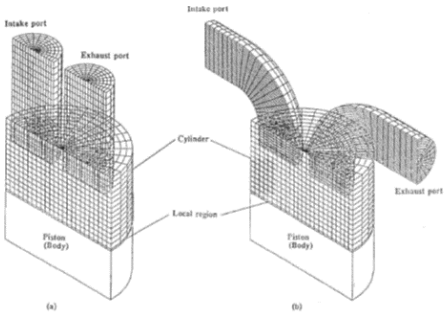
Matching cell faces

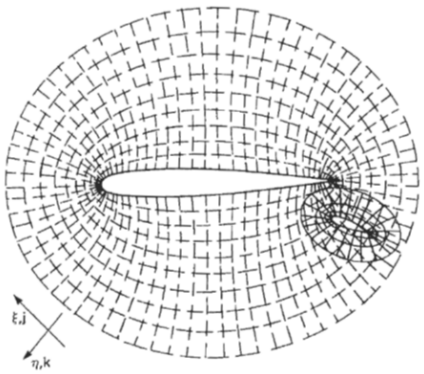
Multi-block structured mesh with matching and non-matching cell faces.



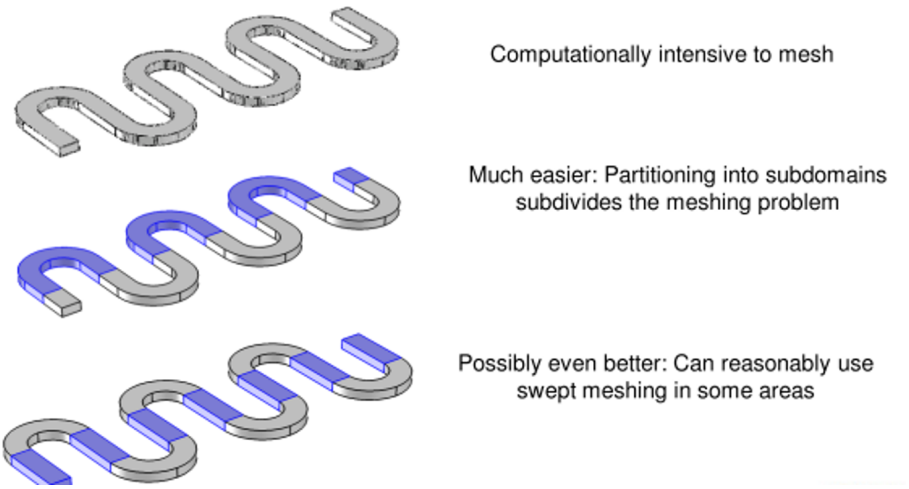
Non-Matching cell faces

Overlapping ("chimera") grids are common in FD schemes, although not necessary and seldom used for FE models.



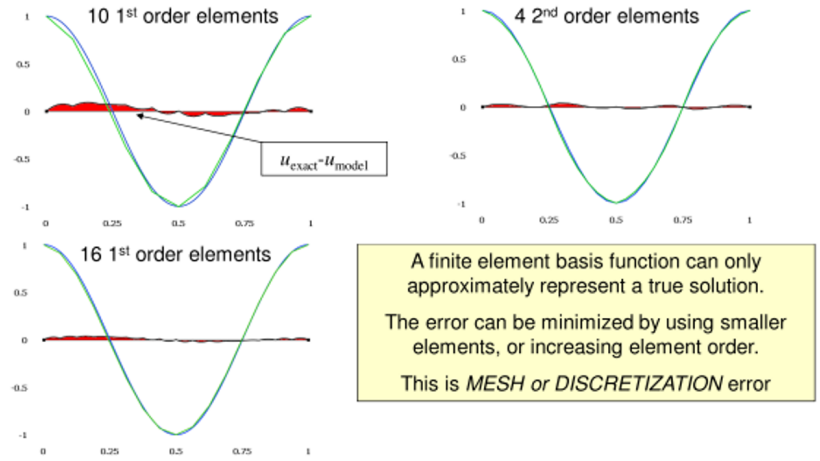


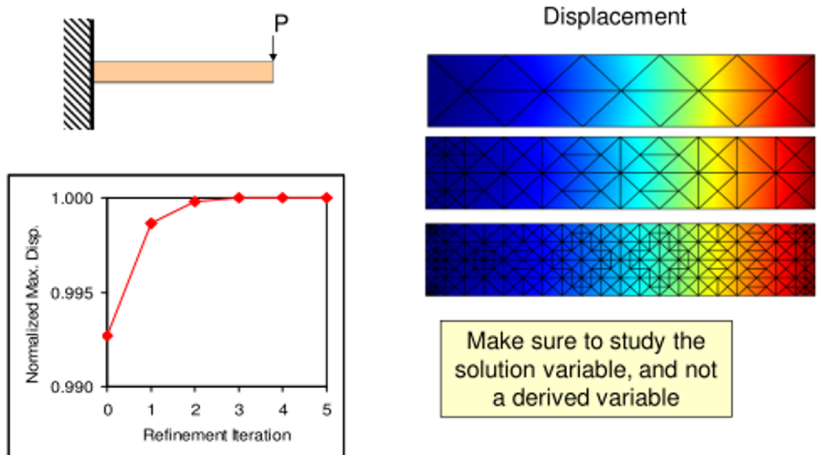
Partitioning the geometry into smaller regions allows better meshing strategies.

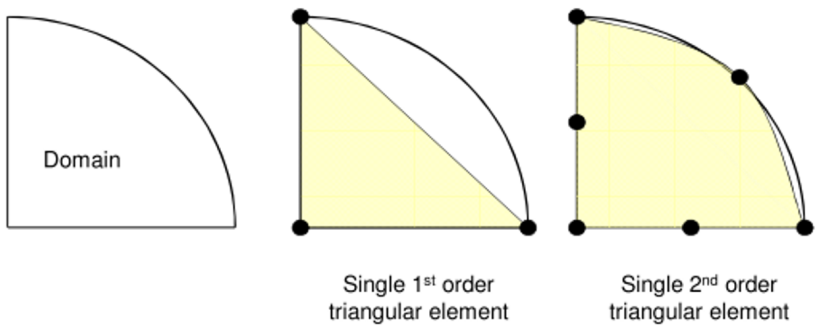


C.3 Grid accuracy

The more elements, the better, but this has some practical limits. How much accuracy do you really need? For a large problem, halving the element size may increase the solution time by a factor of 20 for 2-D, and 100 for 3-D.







Element shape functions discretise curved edges. Isoparametric elements use polynomial expressions to approximate the domain shape.

Round-off error (ROE):

- ROE——Digits

  7 digits——Single precision

  15 digits——Double precision

$A+B+C \neq A+C+B$

Given a=+7777777
b=-7777776
c=0.4444432

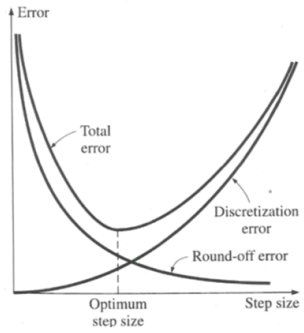
Find D = a + b + c
E = a + c + b

Solution: D = 7777777-7777776+0.4444432
= 1 + 0.4444432
= 1.444443 (correct)
E = 7777777+0.4444432-7777776
= 7777777 - 7777776
= 1.0 (error = 30.8%)

Example :

A simple arithmetic operation performed with a computer in a single precision using seven significant digits

Total error:



As the mesh size or time step size decreases, the discretisation error decreases ! but the round-off error increase!

No. of Computations ↑

Accumulated ROE ↑

Fine mesh.... coarse mesh

C.3.1 Guidelines for meshing the geometry

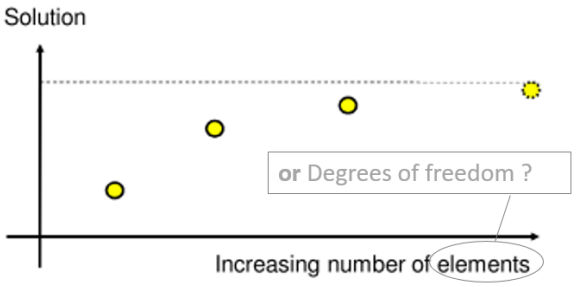
- Geometrically straight edges can be precisely represented with a single element.
- A 90° arc meshed with two 2nd order elements represents the geometry to ≈ 0.1 % accuracy.
- A 90° arc meshed with eight 1st order elements represents the geometry to ≈ 1 % accuracy.
  - Most CFD problems use 1st order elements by default.
  - Thermal stress problems use 1st order elements for the thermal problem.
- Use gradual transitions between small and large elements.

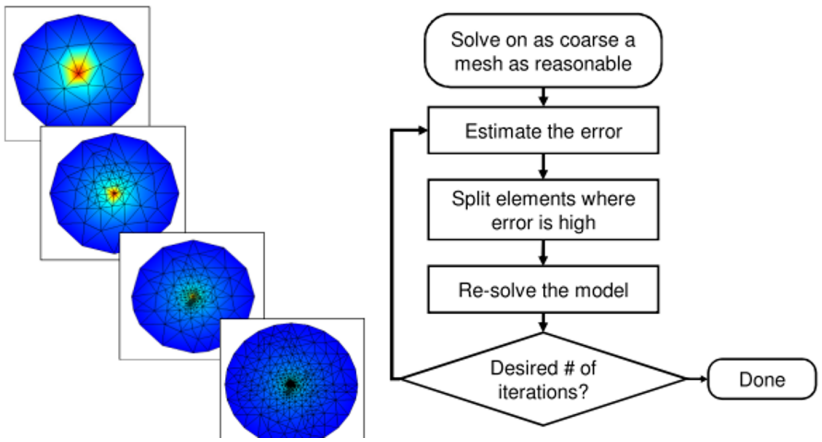
C.3.2 Guidelines for meshing refinement

1. Start with a mesh that you believe will resolve the gradients on the solution that you expect, use the previously mentioned techniques to get an answer.
2. If it is easy to manually put more elements in regions where you observe steep gradients in the solution, then do so. Monitor the convergence of the solution as you proceed.
3. Use adaptive mesh refinement to automatically refine the mesh, but be prepared to devote time and RAM to the solution.
4. If the solution itself does not change, *to a tolerance that you consider acceptable*, then the solution is converged.

How finely do we need to mesh?

- Start as coarse as possible to represent the geometry.
- Compute a solution.
- Refine, refine, refine, and monitor what is of interest to *you*.
- Alternatively: use adaptive mesh refinement.





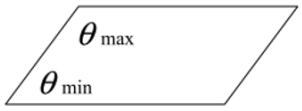
The adaptive mesh refinement algorithm. Note: if it is only taking a few seconds/minutes to run, it may be more cost effective just to refine the mesh everywhere — your own time is often the most expensive resource.

C.4 Mesh quality

C.4.1 Skewness

- Common measure of quality is based on equiangle skew.
- Definition of equiangle skew:

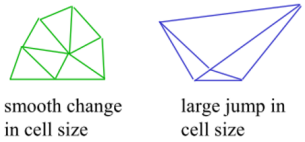
$$\max\left(\frac{\theta_{\max}-\theta_e}{180-\theta_e}, \frac{\theta_e-\theta_{\min}}{\theta_e}\right) \tag{C.1}$$



where $\theta_{\max}$ is the largest angle in the face or cell, $\theta_{\min}$ the smallest angle, θ_e the angle for the equiangular face or cell (e.g. 60° for a triangle, or 90° for a square).

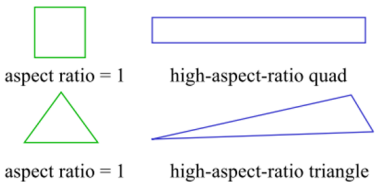
- Range of skewness is from 0 (best case) to 1 (worst case).

C.4.2 Smoothness



- The change in element size should be gradual (i.e. smooth) between neighbouring elements.

C.4.3 Aspect ratio



- The aspect ratio is the ratio of the longest edge length to shortest edge length.
- An ideal aspect ratio is equal to 1, which is true for an equilateral triangle or a square.

C.4.4 Striving for a quality mesh

- A poor quality grid will cause inaccurate solutions and/or slow convergence.
- Minimise the equiangle skew:
  - Hex and quad cells: skewness should not exceed 0.85
  - Tri's: skewness should not exceed 0.85
  - Tets: skewness should not exceed 0.9
- Minimise local variations in cell size:
  - e.g. adjacent cells should have a "size ratio" below 20 %

Value of Skewness	0-0.25	0.25-0.50	0.50-0.80	0.80-0.95	0.95-0.99	0.99-1.00
Cell Quality	excellent	good	acceptable	poor	sliver	degenerate

C.5 Generation of structured grids

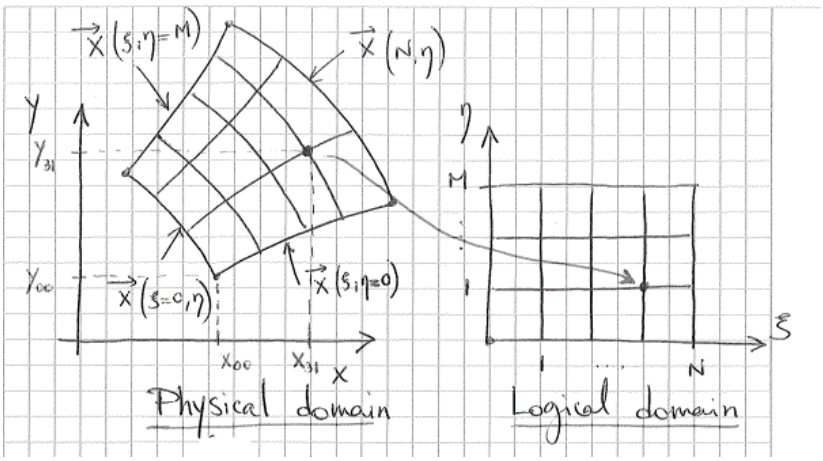
There are two main methods for generating structured grids:

1. Algebraic grid generation
2. Elliptic grid generation

Both techniques rely on finding a unique mapping

$$(x, y) = (x(\xi, \eta), y(\xi, \eta)) \qquad \text{or} \qquad (\xi, \eta) = (\xi(x, y), \eta(x, y)) \quad \text{(C.2)}$$

between given discrete values $\xi = 0, 1, ..., N$ and $\eta = 0, 1, ..., M$ (logical or computational domain) and the physical coordinates (x, y).

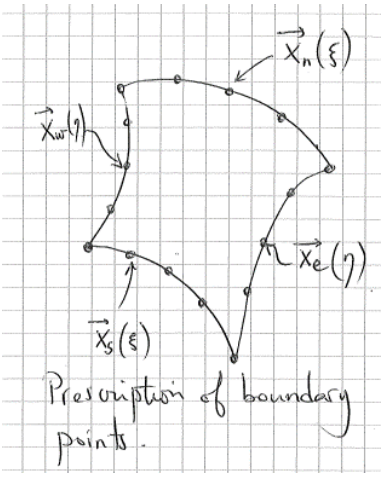


C.5.1 Algebraic grid generation

1. Prescribe grid points at the boundary of the physical domain:

$$\vec{x}(\xi, 0) = \vec{x}_s(\xi), \quad \vec{x}(\xi, M) = \vec{x}_n(\xi) \quad \text{for} \quad \xi = 0, ..., N \quad \text{(C.3)}$$

$$\vec{x}(0, \eta) = \vec{x}_w(\eta), \quad \vec{x}(N, \eta) = \vec{x}_e(\eta) \quad \text{for} \quad \eta = 0, ..., M \quad \text{(C.4)}$$

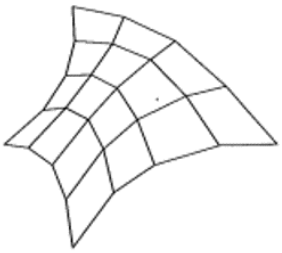


2. The points within the interior of the domain are determined from the boundary grid points using linear interpolation:

$$\begin{aligned} \vec{x}(\xi, \eta) \;=\; & (1 - \tfrac{\eta}{M})\vec{x}_s(\xi) + \tfrac{\eta}{M}\vec{x}_n(\xi) + (1 - \tfrac{\xi}{N})\vec{x}_w(\eta) + \tfrac{\xi}{N}\vec{x}_e(\eta) \\ & - \tfrac{\xi}{N}\left[\tfrac{\eta}{M}\vec{x}_n(M) + (1 - \tfrac{\eta}{M})\vec{x}_s(M)\right] \\ & - (1 - \tfrac{\xi}{N})\left[\tfrac{\eta}{M}\vec{x}_n(0) + (1 - \tfrac{\eta}{M})\vec{x}_s(0)\right] \end{aligned} \quad \text{(C.5)}$$

Features of the algebraic grid generation method:

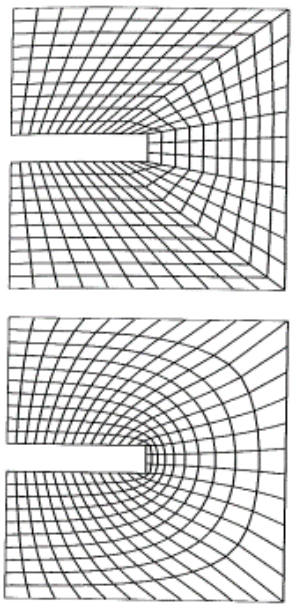
- The coordinates of all interior points (for $\xi = 1, ..., N - 1$ and $\eta = 1, ..., M - 1$) are defined in terms of the boundary points.



Grid produced using the transfinite interpolation technique. Available within COMSOL as the "mapped" approach.

- Also known as the transfinite interpolation.
- Advantages:
  - Very easy to implement.
  - Little computational effort.
- Disadvantages:
  - Not always applicable.
  - Boundary irregularities can propagate in the domain, resulting in a poor mesh quality.

C.5.2 Elliptic grid generation



Comparison of grids produced using the transfinite interpolation technique (top) and elliptic grid generation (bottom).

Another idea which relies on solving a PDE to generate the mesh:

$$\xi_{xx} + \xi_{yy} = 0 \tag{C.6}$$

$$\eta_{xx} + \eta_{yy} = 0 \tag{C.7}$$

which is solved in the physical domain.

However, for the actual determination of the grid coordinates, these equations must be solved within the logical domain, i.e. dependent and independent variables have to be interchanged. Applying the chain rule, the following equations result:

$$c_1 x_{\xi\xi} - 2c_2 x_{\xi\eta} + c_3 x_{\eta\eta} = 0 \tag{C.8}$$

$$c_1 y_{\xi\xi} - 2c_2 y_{\xi\eta} + c_3 y_{\eta\eta} = 0 \tag{C.9}$$

solved for x and y in the logical domain, subject to the conditions in Equations C.3 and C.4, and with:

$$c_1 = x_\eta^2 + y_\eta^2, \quad c_2 = x_\xi x_\eta + y_\xi y_\eta, \quad c_3 = x_\xi^2 + y_\xi^2 \tag{C.10}$$

Features of the elliptic grid generation method:

- The governing equations are non-linear and require an iterative solution process.
- Main advantage: the method generates smooth orthogonal grids[a].
- Main disadvantage: computationally more expensive.

[a] *Useful in FD applications where the equations themselves are mapped to a different coordinate system before being discretised. Although, not so important in FE models.*

C.6 Generation of unstructured grids

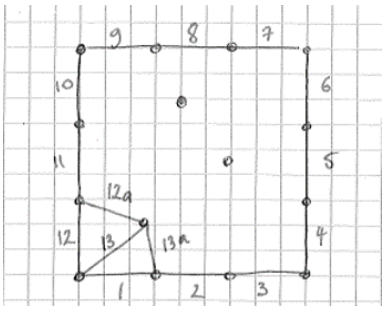
Unstructured mesh generation procedures have been developed to automate the mesh generation process. There are two main techniques which are widely used:

- Advancing front methods.
- Delaunay triangulation.

C.6.1 Advancing front method

- Dates back to the mid 80s.
- The mesh is constructed by adding mesh elements starting at the boundary. Iterating results in the propagation of a front which is the border between the meshed and unmeshed region.
- Advantage of this technique: possibility of the automatic generation of the interior points with good quality triangles.
- Disadvantage: the computational cost of this technique since as the front advances, it must compute which is the nearest point or introduce a new one according to complex rules.

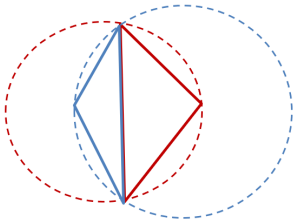
Procedure for the advancing front method:



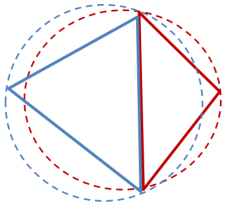
1. Edges on the boundary are numbered and stored in a "front vector".
2. The node closest to the last boundary edge is located and a new triangle created.
3. Edge 12 is removed from the front vector and replaced by edges 12a and 13.
4. The node closest to edge 13 is located and a new triangle is formed.
5. Edge 13 is removed and replaced by 13a.
6. And so on...

C.6.2 Delaunay triangulation

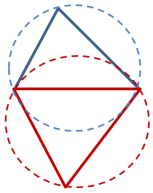
- Investigated since the early 80s.
- Provides a criterion which can be used to automatically select a triangulation for a set of points in the computational domain that is of reasonable quality.



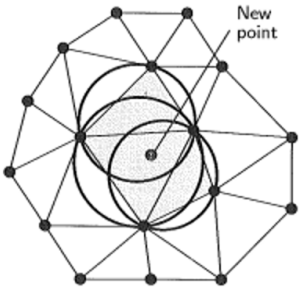
✗

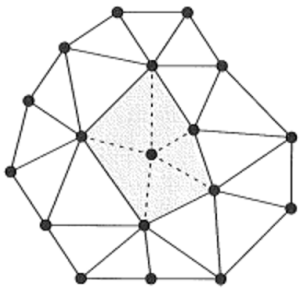


✓



✓





- Can be used to automatically generate a "good" triangulation of a set of points in the computational domain.
- The Delaunay criterion for an acceptable mesh is that the circumscribing circle for any triangle (or circumscribing sphere for a tetrahedron) does not include any other vertex point of the mesh.

Delaunay triangulation algorithm:

1. Start from an initial triangulation.
2. Add a new point in the triangulation.
3. Identify all triangles whose circumcircle contain the new point.
4. Delete these triangles.
5. A new triangulation is generated by connecting the new point with the corner points of the polygon which results from the deletion of the triangles.
6. And so on...

$\Longrightarrow$ The exact same concept can be applied to tetrahedra.

C.6.3 Next steps

We have only introduced the rudiments of grid generation. It is a vast topic which is also very mature. Most simulation packages come with an "easy", automated way to generate grids. You don't need to do it yourself except in the most exceptional circumstances.

Good freeware exists to generate unstructured meshes. One of the best is GMSH (`https://gmsh.info/`).

Commercial codes have their own proprietary mesh generators. ICEM CFD is perhaps the industry standard, and is hard to beat.

Appendix D

COMSOL Laboratories

D.1 Lab 1: Model builder

The purpose of this lab is to familiarise yourself with the COMSOL Desktop environment by performing a structural analysis of a wrench. If you have time and interest afterwards, work through the second example on modelling a busbar (a multiphysics problem involving electric current and energy).

D.1.1 Analysis of a wrench

Open COMSOL by following the instructions on Ako | Learn[a]. Next, open the Introduction to COMSOL Multiphysics documentation via `File > Help >` Documentation and follow the instructions for Example 1: Structural Analysis of a Wrench[b].

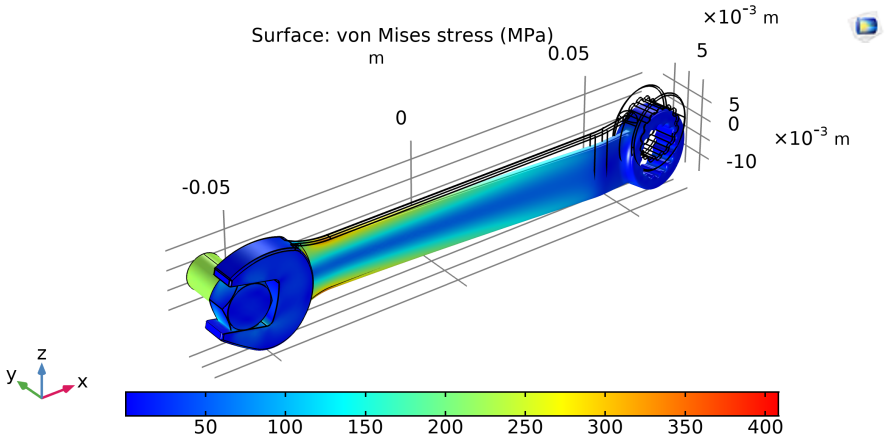


Figure D.1: Stress on a wrench while tightening a bolt with 150 N of force.

a *Create a shortcut on your desktop for easy access:* `Right click > Send to > Desktop (create shortcut)`. *Save your COMSOL models on the local disk (e.g. create a folder in* `C:\Temp\`*) to speed up read/write access. Remember to move your files to your* `P:` *drive when finished.*

b *You may also like to skim through the first section of this document on the COMSOL Desktop to supplement our brief introduction of COMSOL above.*

D.1.2 Hints & tips

- After modifying settings upstream of the Study 1 node (e.g. geometry or boundary conditions), the system of equations need to be solved again with = Compute and then the Results updated.
- Use the Image Snapshot button within the graphics window to produce quality images of the results.

D.2 Lab 2: Geometry and meshing

The purpose of this lab is to give you some experience generating geometry and meshes in COMSOL, and also to think about the relationship between mesh refinement, solution time and solution accuracy.

D.2.1 Tapered cantilever with two load cases

Follow the instructions in COMSOL Documentation > Application Libraries > COMSOL Multiphysics > Structural Mechanics > Tapered Cantilever with Two Load Cases.

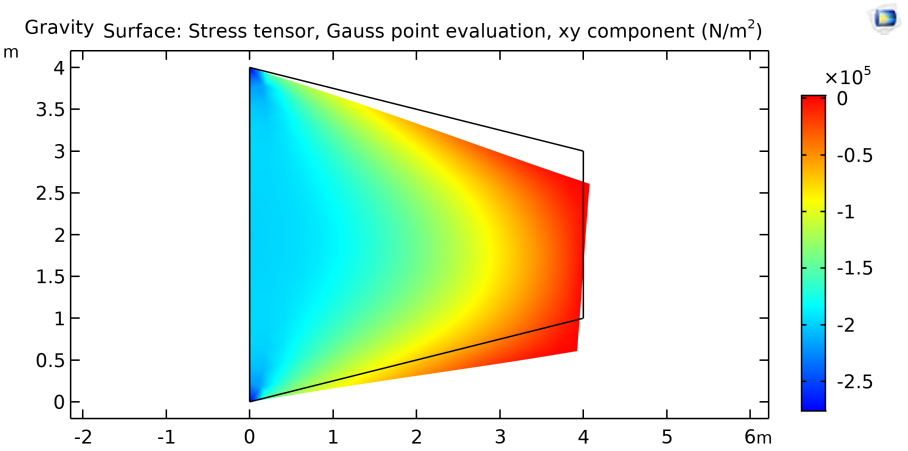


Figure D.2: In-plane shear stress of a tapered cantilever for the gravity loading case. Note: the deformation is exaggerated.

D.2.2 Meshing in COMSOL (2-D)

In 2-D, COMSOL allows you to create:

- unstructured ("free") meshes formed from triangles or quadrilaterals
- structured ("mapped") meshes formed from quadrilaterals

We are going to explore several settings within the Mesh node to get some idea of how these two mesh types work.

Default mesh

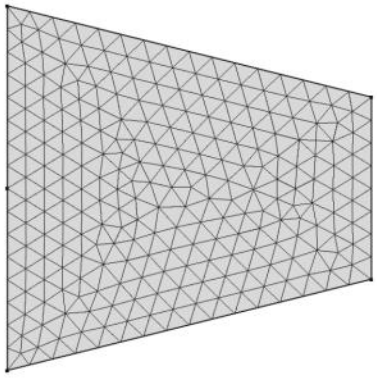


Figure D.3: Default triangular mesh.

So far, we have computed a solution by using the default unstructured mesh.

Examine the details of this mesh:

1. Component 1 > Mesh 1 and check that the default Element size is set to Normal.
2. Refine the mesh by changing the Element size to Fine, Finer, Extra fine etc. In each case, Build All to generate the new mesh.
3. Set the Element size back to Normal, and Build All, to return to the original mesh.

We need to Build All to create or update the mesh after changes. Although, COMSOL may perform this task automatically for you via = Compute.

Delaunay mesh

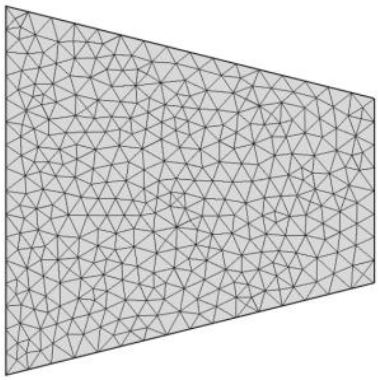


Figure D.4: Delaunay mesh.

The free mesh algorithm in COMSOL can apply two different methods to generate an unstructured triangular mesh; the "Advancing front" and "Delaunay triangulation". Each produces a different mesh. The default is the "Advancing front" method, and to view the corresponding Delaunay mesh for this problem:

1. Mesh 1 > Settings set Sequence type to User-controlled mesh.
2. Free Triangular 1 > Tessellation set Method to Delaunay.
3. Build All which gives the Delaunay mesh.

There is no obvious advantage in using one type of mesh over the other so you can generally stick to the default method.

Unstructured quad mesh

The free mesh option in COMSOL defaults to an unstructured mesh of triangular elements for 2-D problems. However, you can override this setting to generate an unstructured quad mesh. Let's try creating a new mesh rather than by replacing the existing mesh:

1. Right click Component 1 > Add Mesh, which adds Mesh 2.
2. Right click Mesh 2 > Free Quad and then Build All.

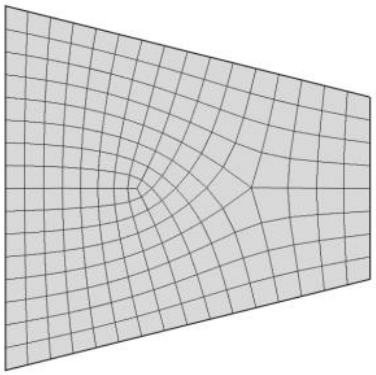


Figure D.5: Free quad mesh.

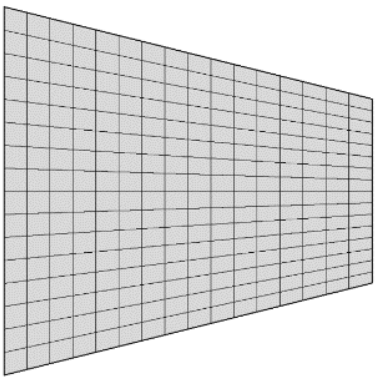


Figure D.6: Mapped mesh

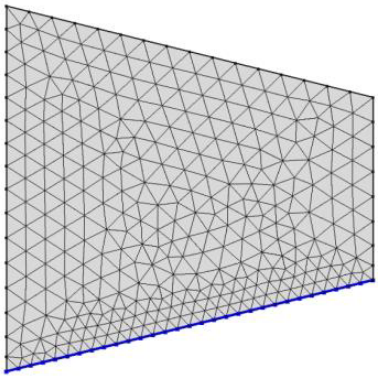


Figure D.7: Refined mesh along lower edge.

3. Mesh 2 > Size and explore mesh resolutions by changing the Element Size to coarser and finer meshes.

The form of the unstructured meshes depend on what algorithm is used for generation, and is changed between COMSOL versions (i.e. your mesh might not look precisely the same as those above). The method can be forced to an older version via Free Quad 1 > Tessellation and change the Method.

Structured (mapped) mesh

For generating a structured mesh:

1. Right click Component 1 > Add Mesh, which adds Mesh 3.
2. Right click Mesh 3 > Mapped, then Build All, which generates a structured mesh as shown below.
3. Mesh 3 > Size and change the resolution of the mesh as done earlier for the unstructured cases.

Further customisation of the mesh resolution can be achieved from Size > Element Size and changing from Predefined to Custom, and then editing the Element Size Parameters.

Local mesh refinement

In addition to refining the entire mesh, we can refine the element size locally (e.g. along an edge or within a region).

Refinement along an edge:

1. Right click Component 1 > Add Mesh, which adds Mesh 4.
2. Mesh 4 > Mesh Settings and change the Sequence type to User-controlled mesh.
3. Right click Free Triangular 1 > Distribution.
4. Distribution 1 > Boundary Selection select the lower boundary.
5. Distribution 1 > Distribution set the Number of elements to 30 and Build All.

Refinement over a specific sub-region of the mesh:

1. Right click Component 1 > Add Mesh, which adds Mesh 5.
2. Right click Mesh 5 > Free Triangular and Build All.

3. Right click Mesh 5 > Modifying Operations > Refine.

4. Refine 1 > Refine Elements in Box enable Specify bounding box, click Draw Box and then draw a box around the lower right corner.

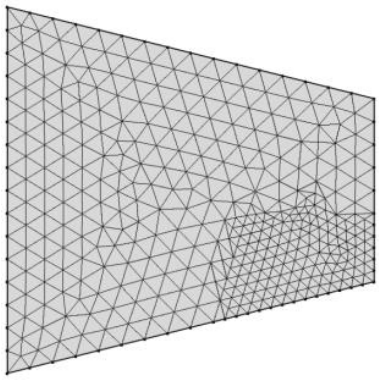


Figure D.8: Local area refinement.

D.2.3 Meshing in COMSOL (3-D)

In 3-D, COMSOL allows you to create:

- unstructured ("free") meshes of tetrahedral elements
- structured ("swept") meshes of hexahedra or of triangular prisms

You have already generated unstructured meshes of tetrahedra in Lab 1 (Section D.1) with the wrench model. This mesh setting is the default option in COMSOL.

Swept meshing

Structured meshes are generated using the "sweep" option. This is described in the COMSOL documentation at: COMSOL Multiphysics > COMSOL Multiphysics Reference Manual > Meshing > Meshing Operations and Attributes > Swept.

Summary: a swept mesh is generated by first creating a mesh on the "source" face of a subdomain. If a mesh has already been created on this face by using one of the methods already described for 2-D meshing, this mesh will be "swept". If no mesh is present, one will be created automatically and then "swept". In either event, the 2-D mesh on the source face is extruded along the sweep axis to the "destination" face. The sweep direction can be curved and the destination and source faces can have quite different shapes. The mesh on the source face will "morph" onto that of the destination face as long as the topology is consistent.

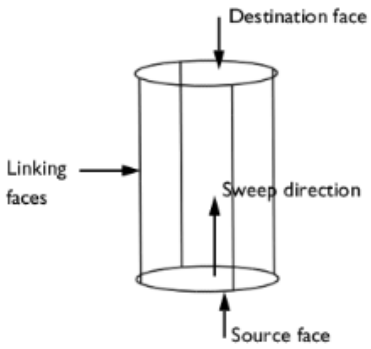


Figure D.9: Classification of the boundaries about a domain used for swept meshing. Schematic from the COMSOL documentation.

Swept mesh example

A simple illustration of a swept mesh:

1. Open a new 3-D model in COMSOL (ignore physics interfaces).
2. Right click Geometry 1 > Block.
3. Block 1 > Size and Shape set Width: 2, Depth: 0.5, Height: 1.
4. Build All Objects which creates the following block.

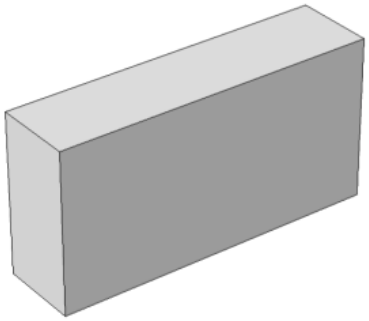


Figure D.10: Simple block (front is the source face, rear is destination face)

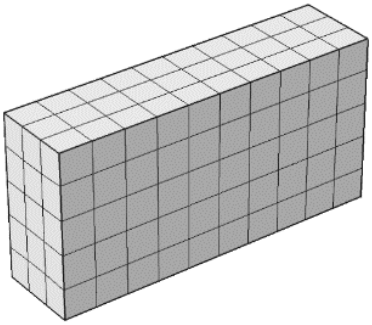


Figure D.11: Simple block with swept mesh

5. Right click Mesh 1 > Swept.
6. Swept 1 > Source Faces set the LHS face (boundary 1).
7. Swept 1 > Destination Faces set the RHS face (boundary 6).
8. Build All to generate a structured, swept mesh.
9. Swept 1 > Sweep Method set Face meshing method to Triangular (generate prisms), then Build All to generate a mesh with swept triangular elements (prisms or extruded triangles).

Alternatively, a similar 3-D swept mesh could be created by first generating a 2-D mesh on the source face; using either a mapped or free mesh. For this approach, right click Mesh 1 > Boundary Generators and select either Free Triangular, Free Quad, or Mapped, and then select the LHS face. Note: the Swept 1 node within the Model Builder should be positioned after the source face mesh setting (e.g. Free Triangular 1).

D.2.4 Structured versus unstructured meshes

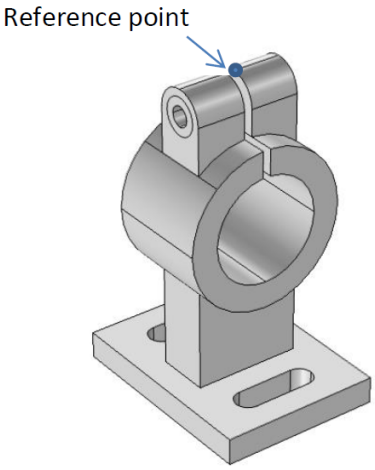


Figure D.12: Feeder clamp

The model problem "Deformation of a Feeder Clamp" uses a sweep to mesh the geometry. The model file and PDF documentation for this problem is available at https://www.comsol.com/model/deformation-of-a-feeder-clamp-115. The feeder_clamp.mph model file already has the geometry, swept mesh, physics, etc, and you do not need to redo the model.

It is generally accepted that while unstructured meshes give converged solutions for finite element problems if the mesh is sufficiently refined, and are generally much quicker to generate than structured meshes, the latter are often more efficient in the sense that they give a more accurate result for a given number of degrees of freedom.

We are going to test this hypothesis by performing the following study with the feeder clamp model, starting with the original structured mesh:

1. = Compute the solution and record the number of degrees of freedom and solution time (available in the Messages window for each run).
2. Calculate and record the displacement in the x-direction at the reference point (located at the highest point) to four significant figures. Hint: right click Derived Values > Point Evaluation and evaluate the expression u (list available expressions

with `Ctrl + Spacebar`).

3. Repeat steps 1 and 2 for increasingly refined unstructured tetrahedral meshes[a] with predefined element sizing: `Coarse`, `Normal`, `Fine`, ..., and `Extremely fine`. Note: the `Extremely fine` mesh may take a moment, and you may prefer to perform a parametric sweep of mesh resolutions instead of the predefined settings. Using a parameter sweep[b], as from Lab 1 (Section D.1), will enable easier post-processing and comparison between meshes.

4. Plot the resulting displacements against the number of degrees of freedom for each of the above solutions. Try using either the log or linear scales for the horizontal axis (number of degrees of freedom).

5. What degree of accuracy do you think that the finest mesh has resolved the reference displacement, i.e. to how many significant figures?

6. Does the data plotted on your graph support the hypothesis that a structured mesh is more accurate than an unstructured one for a given problem size (number of degrees of freedom)?

7. What is your best estimate of the rate at which the solution time scales with problem size? If you were to double the number of degrees of freedom of the finest mesh, for how long would you expect this job to run?

a *For the structured mesh case (the original swept sequence in the model file), only run with the original refinement (a structured mesh cannot be generated with a coarse grid, as there needs to be at least one element in each direction, e.g. the radial coordinate of the collar). Either overwrite the existing Mesh 1 with an unstructured mesh (right click* ***Mesh 1 > Delete Sequence****) or create Mesh 2, 3, etc, as done for the cantilever problem. However, when solving, you need to select which mesh to solve with from* ***Study 1 > Step 1: Stationary > Mesh Selection****.*

b *While plotting the displacement against degrees of freedom can be done within COMSOL, for this lab, simply record the output from your simulations into Excel or Python and plot there. We'll look at plotting results from a parametric sweep in our next lab, analysing a classic beam problem.*

D.3 Lab 3: Solid mechanics

In this lab we will investigate the convergence and accuracy of finite element models for a simple benchmark problem in solid mechanics, and look at the effect of element order on the solution accuracy. We will construct from scratch a 2-D COMSOL model for a simple beam problem and look at the convergence of these models with mesh resolution and element order. In this case we can also compare the numerical solutions to an exact solution given by beam theory.

D.3.1 A classic beam problem

A rectangular section beam of length L, depth h and thickness t is simply supported at each end and loaded with a uniformly distributed load of magnitude w per unit length. This loading case is a classic strength of materials beam problem, and is shown in Figure D.13.

We know (you check!) that simple beam theory predicts:

- A maximum downward vertical displacement (at the centre of the span) of magnitude $5wL^4/384EI$.
- A maximum tensile stress (at the lower surface at the centre of the span) given by $\sigma_x = wL^2h/16I$.

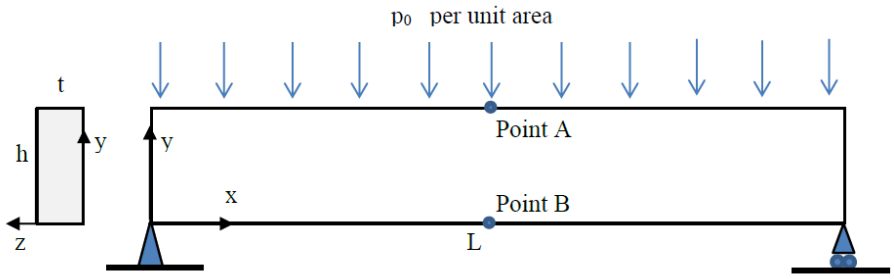


Figure D.13: A simply-supported beam with a uniform pressure p_0 applied.

We are going to create a 2-D COMSOL model for this configuration and treat the beam as a plane stress problem. See `https://www.quora.com/Solid-Mechanics-What-are-the-differences-between-plane-stress-and-plane-strain-conditions` for a discussion on the plane stress assumption.

Use the following dimensions and material properties: $L = 1.0\,\text{m}$, $h = 0.1\,\text{m}$, $t = 0.04\,\text{m}$, $E = 200\,\text{GPa}$, $\nu = 0.3$, and $p_0 = 1\,\text{MPa}$. Confirm that the axial stress and vertical deflection at point B for the above parameters are predicted by beam theory to be: 75.0 MPa and 0.781 25 mm.

D.3.2 COMSOL model

Geometry

1. Open a new 2-D model in COMSOL and choose `Structural Mechanics` > `Solid Mechanics` with a `Stationary` study.
2. `Component 1` > `Solid Mechanics` > `Settings` > `2D Approximation` set `Plane strain` to `Plane stress`, and `Thickness` > `d` to `0.04`.
3. Right click `Geometry 1` > `Rectangle`, set `Width: 1` and `Height: 0.1`.
4. Right click `Geometry 1` > `More Primitives` > `Point` and place a point at A.
5. Repeat for a point at B.

Tip: or you could instead right click `Point 1`, duplicate and modify the coordinates.

Materials

1. Right click Materials > Blank Material and populate the Materials Contents with $E = 200\,\text{GPa}$, $\nu = 0.3$, and $\rho = 7500\,\text{kg/m}^3$.

Hint: be careful with units, and remember `200[GPa]` is equivalent to `200e9[Pa]`.

Boundary conditions

1. Right click Solid Mechanics > Points > Fixed Constraint and select the bottom left hand corner of the rectangle (point 1).
2. Right click Solid Mechanics > Points > Prescribed Displacement and constrain the bottom right corner (point 5) by setting Prescribed in y direction to 0 (i.e. a roller support).
3. Right click Solid Mechanics > Boundary Load, select the upper edge (boundaries 3 and 5) and set Load type:Pressure and p:1[MPa].

Discretisation

Under Component 1 > Solid Mechanics > Settings > Discretization, explore the discretisation settings for the Displacement field. The linear, quadratic, cubic, quartic, and quintic options refer to the order of the element shape functions[a] (e.g. linear approximates the solution over the domain with piecewise linear functions, quadratic uses piecewise quadratic functions).

a *We derive and discuss shape functions in the lectures, but if you are interested, read `https://www.comsol.com/support/knowledgebase/1270` for an introduction to these shape functions and the difference between serendipity and Lagrange types.*

Mesh

We are going to use a structured grid to discretise our computational domain, because we demonstrated earlier that these meshes gave marginally better convergence than unstructured ones. Instead of manually adjusting the number of elements in each direction for each sequentially refined mesh, we are going to parametrise the number of elements in the y-direction and then calculate the corresponding elements in the x-direction.

1. Global Definitions > Parameters 1 > Parameters create a new parameter with Name:Ny and Expression:2.
2. Right click Mesh 1 > Mapped.
3. Right click Mapped 1 > Distribution, select the left and right edges (boundaries 1 and 6), and set Number of elements:Ny.
4. Right click Mapped 1 > Distribution, select the top and

bottom edges (boundaries 2, 3, 4, and 5), and set `Number of elements:2*Ny`. Note: the total number of elements along the top and bottom edges are `2*2*Ny` because these edges are split in half (at points A and B).

5. Right click `Mesh 1` > `Build All`.

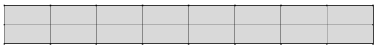


Figure D.14: Initial mesh of 2×8 quadrilateral elements.

Solving

We are now going to solve for the displacement fields (dependent variables for solid mechanics type problems) for a series of increasingly refined meshes.

1. Right click `Study 1` > `Parametric Sweep` and add a sweep `Parameter name:Ny` for `Parameter value list:2^range(1,1,5)` (which is equivalent to `Parameter value list:2,4,8,16,32`).

2. `Compute`

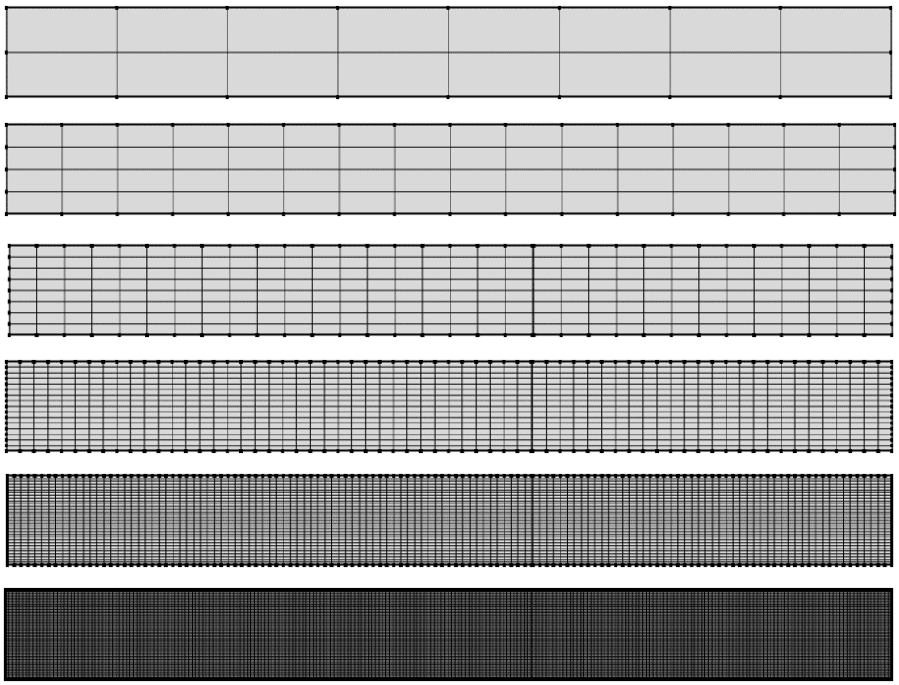


Figure D.15: Mesh sequence of increasingly refined grids.

COMSOL uses a solving algorithm with quadratic elements (as we specified under the `Solid Mechanics` interface) to solve the equations of motion until a converged solution that satisfies the boundary conditions is reached. The `Messages` window provides overall statistics for the job such as the number of degrees of freedom and total process time.

Tip: further detailed information is available within the `Log` window.

Results

Evaluate the displacement and stress at the centre where beam theory predicts both values are greatest (i.e at point B):

1. Right click `Derived Values` > `Point Evaluation` and select point B (point 3).
2. `Point Evaluation 1` > `Data set` `Dataset:Study 1/Parametric Solutions 1 (sol2)` and `Parameter selection (Ny):All`.
3. `Point Evaluation 1` > `Expressions` add `Expression: numberofdofs` for evaluating the number of degrees of freedom.
4. `Point Evaluation 1` > `Expressions` add `Expression:v` with `Unit:mm` for evaluating the y-component of the displacement field.
5. `Point Evaluation 1` > `Expressions` add `Expression: solid.sp1Gp` with `Unit:MPa` for evaluating the first principal stress (Gauss point evaluation).
6. Right click `Derived Values` > `Clear and Evaluate All`.

Analysis

Plot the predicted displacements and stresses at point B against problem size (numbers of degrees of freedom) and compare with the analytical values predicted by beam theory. Assess convergence or otherwise of the resulting data to a limiting value as the mesh is refined. Repeat the process for linear and cubic elements and assess the effectiveness or otherwise of varying the element order.

Tip: solutions (which contain the data including dependent variable fields) can be preserved by creating a copy via right clicking `Parametric Solutions 1` > `Solution` > `Copy`.

D.3.3 Example models for verification

A couple of problems in `File` > `Application Libraries` > `Structural Mechanics Module` > `Verification Examples` are `elliptic_membrane` and `thick_plate` which you may find interesting. It is not necessary to perform every step, but flick through the sequence of instructions and get some idea of what is being modelled and the methods used (e.g. creating multiple studies).

D.4 Lab 4: Solving general PDEs

In this lab we are going to make use of the `Coefficient Form PDE` physics interface to solve the heat equation, and then compare our solution with the `Heat Transfer in Solids` physics interface

by exporting the data to Python and plotting. Spatially-dependent boundary conditions and source terms will be used as examples of more complex conditions. Since we're solving the same governing equations, we expect our results to be identical.

D.4.1 Heat Transfer in Solids

First, we are using the purpose-built physics interface to solve this heat problem.

1. Create a new model with the `Model Wizard` in `2D`, with the `Heat Transfer in Solids` physics interface, and `Time Dependent` study.
2. Create a square with the default settings (side lengths of 1 m and positioned with the corner at the origin).
3. Apply concrete as the material.
4. Under `Heat Transfer in Solids > Settings > Equation` review the equation COMSOL is solving and how it relates to our heat equation from the lecture notes. The key terms in the equation are:
$$d_z \rho c_p \frac{\partial T}{\partial t} + \nabla \cdot (-d_z k \nabla T) = d_z Q \tag{D.1}$$
where d_z is the arbitrary unit depth (as this problem is two dimensional).
5. Set the initial temperature to `0[degC]` and apply the following Dirichlet boundary conditions in units of °C:
   - $T(x, 0) =$ `0[degC]`
   - $T(0, y) =$ `0[degC]`
   - $T(x, 1) =$ `x*100[K/m]+0[degC]`
   - $T(1, y) =$ `y*100[K/m]+0[degC]`
6. Apply a `Heat Source` given by: `x*(1-x)*y*(1-y)*1e5[W/m^7]` (which is spatially-dependent). This heat could be generated by something underneath the concrete, and is focussed at the centroid.
7. Create a suitable mesh, e.g. mapped and reasonably fine, set the output times to `range(0,1,24)` hours, and `Compute`.

D.4.2 Coefficient Form PDE

Second, we will use a more generic physics interface, the △u `Coefficient Form PDE`, to solve the same problem as a demonstration that COMSOL can be used for solving arbitrary PDEs. By default, the dependent variable for the heat equation was T and for this generic form is u, although either of these can be renamed from under their respective settings.

1. With your current model from above (a good time to save!), add another physics interface e.g. `Physics > Add Physics` and select △u `Coefficient Form PDE` for `Component 1`.

2. Under △u `Coefficient Form PDE > Settings > Units` set `Dependent quantity:Temperature` and `Source quantity:Heat source`.

3. Review the equation COMSOL is solving for in this physics interface via `Coefficient Form PDE 1 > Equation` and compare with earlier. The key terms in this equation are:

$$d_a \frac{\partial u}{\partial t} + \nabla \cdot (-c \nabla u) = f \tag{D.2}$$

   where the other coefficients are zero (i.e. not included within our governing heat equation).

4. Matching coefficients from Equation D.1 gives the following terms to enter:

   - $d_a = \rho c_p =$ `ht.rho*ht.Cp`
   - $c = k =$ `ht.k_iso`
   - $f = Q =$ `x*(1-x)*y*(1-y)*1e5[W/m^7]`

5. Apply the same initial and Dirichlet boundary conditions as earlier.

6. Using the same mesh and time settings as earlier, = `Compute`.

Tip: check that your solutions are matching within COMSOL by plotting the difference between the two temperature fields, e.g. `u-T` *using a surface plot.*

D.4.3 Exporting to Python

There are several techniques to export data from COMSOL, the most straightforward approach is to copy and paste values from a table. We are going to work through a couple of methods of extracting the nodal data in 1-D, and then in 2-D, and import into Python.

1-D cut line

1. Right click `Datasets > Cut Line 2D` and create a horizontal line through the centroid, i.e. from $(0, 0.5)$ to $(1, 0.5)$.
2. Right click `Cut Line 2D 1 > Add Data to Export`
3. `Results > Export > Data 1 > Settings > Output` and `Browse` to choose where to save the file. Review the other settings and then `Export` (by default, both T and u for all time outputs are exported).
4. Open the data file and inspect the format. There are eight lines for meta data and one line for the column headers. The first two columns are for x and y coordinates and the subsequent columns are alternating between T and u.
5. Open `Python` and import using functions such as `np.loadtxt`, e.g. `d = np.loadtxt('lab4_cutline.txt', skiprows=9)`
6. Plot time evolution graphs, e.g. `plt.plot(d[:,0], d[:,2::2])` or difference: `plt.plot(d[:,0], d[:,2::2] - d[:,3::2])`

2-D complete domain

There are ways of exporting with the topology intact, e.g. as an unstructured VTK file, although we're going to look at exporting the data values as a list of points and interpolate within Python to generate a surface plot.

1. Right click `Export > Data` and set `Time selection:Last`, then choose a file save location and `Export`.
2. Open `Python` and import as earlier.
3. Setup the grid: `X, Y = np.meshgrid(np.linspace(0,1), np.linspace(0,1))`
4. Setup the interpolation function for each dependent variable, e.g.: `T = griddata(d[:,0:2], d[:,2], (X,Y))`
5. Plot a surface with `ax.plot_surface(X, Y, T)`

Of course these interpolation functions could be called at other coordinates (we created another grid within Python and interpolated our values from COMSOL onto this grid). For further information on surface plotting see... `https://matplotlib.org/stable/gallery/mplot3d/surface3d.html`.

D.5 Lab 5: Optimisation studies

The lab this week is focussed on exploring three different optimisation methods for minimising the mass of a beam which is subjected to a displacement constraint and a distributed load. The problem is solved using parameter (vertical position of two vertices), shape (lower boundary profile), and topology (free to remove interior material) optimisations. Follow the instructions from the PDF document at `https://www.comsol.com/model/design-optimization-of-a-beam-71731` where the model file is also available.

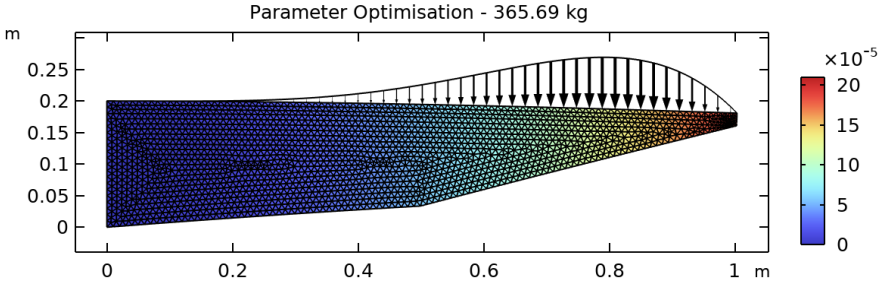


Figure D.16: Parameter optimisation of beam.

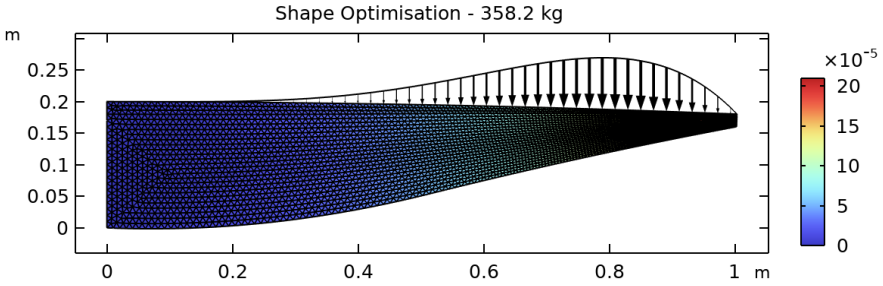


Figure D.17: Shape optimisation of beam.

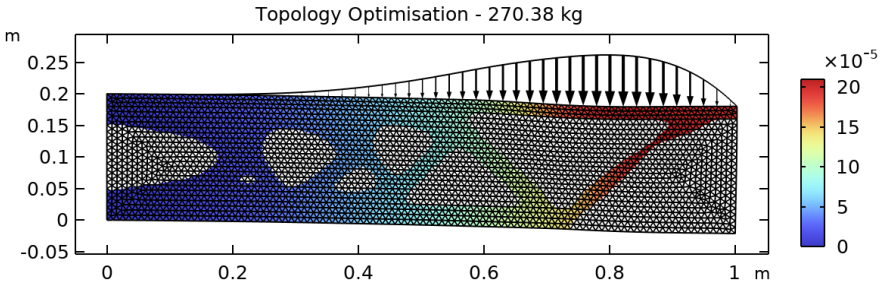


Figure D.18: Topology optimisation of beam.

For further reading and details on the Optimization Module in COMSOL, see `COMSOL Documentation > Optimization Module > Introduction`.

Appendix E

Solutions for exercises

E.1 Partial Differential Equations

Q1(a) Plug $T = \alpha T_1 + \beta T_2$ into the Laplace equation:

$$\frac{\partial^2}{\partial x^2}(\alpha T_1 + \beta T_2) + \frac{\partial^2}{\partial y^2}(\alpha T_1 + \beta T_2) = 0$$

$$\alpha \frac{\partial^2 T_1}{\partial x^2} + \beta \frac{\partial^2 T_2}{\partial x^2} + \alpha \frac{\partial^2 T_1}{\partial y^2} + \beta \frac{\partial^2 T_2}{\partial y^2} = 0$$

$$\alpha \underbrace{\left(\frac{\partial^2 T_1}{\partial x^2} + \frac{\partial^2 T_1}{\partial y^2} \right)}_{\substack{= 0 \text{ because} \\ T_1 \text{ is a solution}}} + \beta \underbrace{\left(\frac{\partial^2 T_2}{\partial x^2} + \frac{\partial^2 T_2}{\partial y^2} \right)}_{\substack{= 0 \text{ because} \\ T_2 \text{ is a solution}}} = 0$$

$\implies$ $T = \alpha T_1 + \beta T_2$ is also a solution.

Q1(b) Plug $T = \alpha T_1 + \beta T_2$ into the Poisson equation:

$$\alpha \underbrace{\left(\frac{\partial^2 T_1}{\partial x^2} + \frac{\partial^2 T_1}{\partial y^2} \right)}_{\substack{= f \text{ because} \\ T_1 \text{ is a solution}}} + \beta \underbrace{\left(\frac{\partial^2 T_2}{\partial x^2} + \frac{\partial^2 T_2}{\partial y^2} \right)}_{\substack{= f \text{ because} \\ T_2 \text{ is a solution}}} = (\alpha + \beta) f \neq f$$

$\implies$ $T = \alpha T_1 + \beta T_2$ is not a solution of the Poisson equation. The Poisson equation is not homogeneous.

Q1(c) For $T = 2xy$:

$$\frac{\partial T}{\partial x} = 2y; \quad \frac{\partial^2 T}{\partial x^2} = 0; \quad \frac{\partial T}{\partial y} = 2x; \quad \frac{\partial^2 T}{\partial y^2} = 0$$

$$\implies \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0 + 0 = 0 \text{ as required.}$$

For $T = x^3 - 3xy^2$:

$$\frac{\partial T}{\partial x} = 3x^2 - 3y^2; \quad \frac{\partial^2 T}{\partial x^2} = 6x; \quad \frac{\partial T}{\partial y} = -6xy; \quad \frac{\partial^2 T}{\partial y^2} = -6x$$

$$\implies \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 6x - 6x = 0 \text{ as required.}$$

Q2(a) $A = 1$, $B = 0$, $C = 1$, $B^2 - 4AC < 0 \implies$ elliptic

Q2(b) $A = k$, $B = 0$, $C = 0$, $B^2 - 4AC = 0 \implies$ parabolic

Q2(c) $A = 1$, $B = 0$, $C = -c^2$, $B^2 - 4AC > 0 \implies$ hyperbolic

Q2(d) $A = y$, $B = 0$, $C = 1$, $B^2 - 4AC = -4y \implies$ elliptic for $y > 0$, parabolic for $y = 0$, hyperbolic for $y < 0$

Q3(a) Any function of y only, $f(y)$, or a constant, e.g.

$$\frac{\partial}{\partial x}(f(y)) = 0$$

Q3(b) Consider the general solution of the ODE $\frac{d^2u}{dx^2} - u = 0$:

$$u(x) = Ae^x + Be^{-x}$$

Now the solution of the PDE will have the constants as a function of y:

$$u(x, y) = A(y)e^x + B(y)e^{-x}$$

Plug this solution into the given PDE:

$$\frac{\partial u}{\partial x} = A(y)e^x - B(y)e^{-x}; \quad \frac{\partial^2 u}{\partial x^2} = A(y)e^x + B(y)e^{-x}$$

$\implies$ solution of the PDE as $\frac{\partial^2 u}{\partial x^2} - u = 0$ as required.

Q4 Consider the first term, $v(x + ct)$, and use a change of variables, $r = x + ct$, and the chain rule:

$$\frac{\partial u}{\partial t} = \frac{\partial v}{\partial t} = \frac{dv}{dr}\frac{\partial r}{\partial t} = \frac{dv}{dr}c$$

$$\frac{\partial^2 u}{\partial t^2} = \frac{\partial}{\partial t}\left(\frac{\partial u}{\partial t}\right) = \frac{\partial}{\partial t}\left(c\frac{dv}{dr}\right) = \frac{d}{dr}\left(c\frac{dv}{dr}\right)\frac{\partial r}{\partial t} = c^2\frac{d^2 v}{dr^2}$$

$$\frac{\partial u}{\partial x} = \frac{\partial v}{\partial x} = \frac{\partial v}{\partial r}\frac{\partial r}{\partial x} = \frac{dv}{dr}$$

$$\frac{\partial^2 u}{\partial x^2} = \frac{\partial}{\partial x}\left(\frac{\partial u}{\partial x}\right) = \frac{\partial}{\partial x}\left(\frac{dv}{dr}\right) = \frac{d}{dr}\left(\frac{dv}{dr}\right)\frac{\partial r}{\partial x} = \frac{d^2 v}{dr^2}$$

As above: $\dfrac{\partial^2 u}{\partial t^2} = c^2 \dfrac{d^2 v}{dr^2}$; $\quad \dfrac{\partial^2 u}{\partial x^2} = \dfrac{d^2 v}{dr^2} \implies \dfrac{\partial^2 u}{\partial t^2} = c^2 \dfrac{\partial^2 u}{\partial x^2}$ as required.

$\implies v(x+ct)$ will be a solution of the wave equation, and a similar method can be applied to show that $w(x-ct)$ is also a solution.

E.2 Elliptic PDEs: derivation and analytical method

Q1 Dirichlet b.c.: the value of the dependent variable is imposed at the boundary. Neumann b.c.: the value of the gradient of the dependent variable is imposed at the boundary.

Q2 We wish to solve the Laplace equation $\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$ subject to $T(x=0,y) = T(x=L,y) = T(x,y=0) = 0\,^\circ\text{C}$ and $T(x,y=W) = T_0$. We seek a solution of the form:

$$T(x,y) = \big(A\sin(\mu x) + B\cos(\mu x)\big)\big(Ce^{\mu y} + De^{-\mu y}\big)$$

- b.c. $T(x=0,y) = B(Ce^{\mu y} + De^{-\mu y}) = 0 \implies B = 0$

  $$\implies T(x,y) = \sin(\mu x)\big(C'e^{\mu y} + D'e^{-\mu y}\big)$$

  with $C' = AC$ and $D' = AD$

- b.c. $T(x=L,y) = \sin(\mu L)\big(C'e^{\mu y} + D'e^{-\mu y}\big) = 0$
  $\implies \mu L = n\pi \implies \mu = \frac{n\pi}{L}$

  $$\implies T(x,y) = \sin\left(\frac{n\pi x}{L}\right)\left(C'e^{\frac{n\pi}{L}y} + D'e^{-\frac{n\pi}{L}y}\right)$$

- b.c. $T(x,y=0) = \sin\left(\frac{n\pi x}{L}\right)(C' + D') = 0 \implies C' = -D'$

  $$\implies T(x,y) = \sin\left(\frac{n\pi x}{L}\right) 2C' \sinh\left(\frac{n\pi y}{L}\right)$$

  or $T_n(x,y) = C_n \sin\left(\frac{n\pi x}{L}\right)\sinh\left(\frac{n\pi y}{L}\right)$ with $C_n = 2C'$.

- b.c. $T(x,y=W) = \sum_{n=1}^{\infty} C_n \sin\left(\frac{n\pi x}{L}\right)\sinh\left(\frac{n\pi W}{L}\right) = T_0$ and using the Fourier sine series:

  $$T_0 = \sum_{n=1}^{\infty} A_n T_0 \sin\left(\frac{n\pi x}{L}\right) \qquad \text{with} \quad A_n = \begin{cases} \frac{4}{n\pi} & \text{odd } n \\ 0 & \text{even } n \end{cases}$$

  $$\implies C_n = \begin{cases} \frac{4T_0}{n\pi \sinh\left(\frac{n\pi W}{L}\right)} & \text{odd } n \\ 0 & \text{even } n \end{cases}$$

The final solution is:

$$T(x,y) = \sum_{n=1,3,5,\ldots} \frac{4T_0}{n\pi \sinh\left(\frac{n\pi W}{L}\right)} \sin\left(\frac{n\pi x}{L}\right) \sinh\left(\frac{n\pi y}{L}\right)$$

The heat flux through the top boundary is:

$$\dot{q}_y = -k \left.\frac{\partial T}{\partial y}\right|_{y=W}$$

$$\frac{\partial T}{\partial y} = \sum_{n=1,3,5,\ldots} \frac{4T_0}{n\pi \sinh\left(\frac{n\pi W}{L}\right)} \sin\left(\frac{n\pi x}{L}\right) \frac{n\pi}{L} \cosh\left(\frac{n\pi y}{L}\right)$$

$$-k \left.\frac{\partial T}{\partial y}\right|_{y=W} = -k \sum_{n=1,3,5,\ldots} \frac{4T_0}{n\pi \sinh\left(\frac{n\pi W}{L}\right)} \sin\left(\frac{n\pi x}{L}\right) \frac{n\pi}{L} \cosh\left(\frac{n\pi W}{L}\right)$$

The total heat flux through the top boundary:

$$\begin{aligned}
\dot{Q} &= \int_0^L \dot{q}_y(x)dx \\
&= \int_0^L -k \sum_{n=1,3,5,\ldots} \frac{4T_0}{n\pi \sinh\left(\frac{n\pi W}{L}\right)} \frac{n\pi}{L} \cosh\left(\frac{n\pi W}{L}\right) \sin\left(\frac{n\pi x}{L}\right) dx \\
&= -k \sum_{n=1,3,5,\ldots} \frac{4T_0}{n\pi \sinh\left(\frac{n\pi W}{L}\right)} \frac{n\pi}{L} \cosh\left(\frac{n\pi W}{L}\right) \int_0^L \sin\left(\frac{n\pi x}{L}\right) dx \\
&= -k \sum_{n=1,3,5,\ldots} \frac{4T_0}{n\pi \sinh\left(\frac{n\pi W}{L}\right)} \frac{n\pi}{L} \cosh\left(\frac{n\pi W}{L}\right) \left[-\frac{L}{n\pi} \cos\left(\frac{n\pi x}{L}\right)\right]_0^L \\
&= -k \sum_{n=1,3,5,\ldots} \frac{8T_0}{n\pi \sinh\left(\frac{n\pi W}{L}\right)} \cosh\left(\frac{n\pi W}{L}\right)
\end{aligned}$$

Q3 We already know that from Q2:

$$T_2(x,y) = \sum_{n=1,3,5,\ldots} \frac{400}{n\pi \sinh\left(\frac{n\pi W}{L}\right)} \sin\left(\frac{n\pi x}{L}\right) \sinh\left(\frac{n\pi y}{L}\right)$$

Switching the roles of x and y, we can also easily find $T_1(x,y)$:

$$T_1(x,y) = \sum_{n=1,3,5,\ldots} \frac{200}{n\pi \sinh\left(\frac{n\pi L}{W}\right)} \sin\left(\frac{n\pi y}{W}\right) \sinh\left(\frac{n\pi x}{W}\right)$$

Lastly, we calculate $T_3(x,y)$, where we assume a solution of the form:

$$T_3(x,y) = \big(A\sin(\mu y) + B\cos(\mu y)\big)\big(Ce^{\mu x} + De^{-\mu x}\big)$$

and apply the b.c.s to determine the constants A, B, C, D:

- b.c. $T_3(x, y=0) = B\big(Ce^{\mu x} + De^{-\mu x}\big) = 0 \implies B = 0$
- b.c. $T_3(x, y=W) = \sin(\mu W)\big(C'e^{\mu x} + D'e^{-\mu x}\big) = 0 \implies \mu = \frac{n\pi}{W}$

- b.c. $T_3(x = L, y) = \sin\left(\frac{n\pi y}{W}\right)\left(C' e^{\frac{n\pi}{W}L} + D' e^{-\frac{n\pi}{W}L}\right) = 0$
  $\implies D' = -e^{\frac{2n\pi L}{W}} C'$

  $$\implies T_n(x, y) = C_n \sin\left(\frac{n\pi y}{W}\right)\left(e^{\frac{n\pi}{W}x} - e^{\frac{2n\pi L}{W}} e^{-\frac{n\pi}{W}x}\right)$$

  There are an infinite number of solutions: one for each n. We need to find the correct combination of T_n in order to match the final boundary condition.

- b.c. $T_3(x = 0, y) = \sum_{n=1}^{\infty} C_n \sin\left(\frac{n\pi y}{W}\right)\left(1 - e^{\frac{2n\pi L}{W}}\right) = 75$ but the function $f(y) = 75$ for $0 \leqslant y \leqslant W$ can be written as:

$$75 = \sum_{n=1}^{\infty} 75 A_n \sin\left(\frac{n\pi y}{W}\right) \qquad \text{with} \quad A_n = \begin{cases} \frac{4}{n\pi} & \text{odd } n \\ 0 & \text{even } n \end{cases}$$

$$\implies C_n = \frac{75}{1 - e^{\frac{2n\pi L}{W}}} A_n$$

The final solution for $T_3(x, y)$ is:

$$T_3(x, y) = \sum_{n=1,3,5,\ldots} \frac{300}{n\pi\left(1 - e^{\frac{2n\pi L}{W}}\right)} \sin\left(\frac{n\pi y}{W}\right)\left(e^{\frac{n\pi}{W}x} - e^{\frac{2n\pi L}{W}} e^{-\frac{n\pi}{W}x}\right)$$

The solution to the full problem is then obtained by adding together $T(x, y) = T_1(x, y) + T_2(x, y) + T_3(x, y)$.

Q4(a) For the given stream function we can calculate the velocity components:

$$u_r = \frac{1}{r}\frac{\partial \varphi}{\partial \theta} = \frac{1}{r} U\left(r - \frac{a^2}{r}\right)\cos\theta = U\left(1 - \frac{a^2}{r^2}\right)\cos\theta$$

$$u_\theta = -\frac{\partial \varphi}{\partial r} = -U\left(1 + \frac{a^2}{r^2}\right)\sin\theta$$

The no-penetration condition requires that $u_r(r = a, \theta) = 0$:

$$u_r(r = a, \theta) = U\left(1 - \frac{a^2}{a^2}\right)\cos\theta = 0$$

$\implies$ the no-penetration condition is fulfilled.

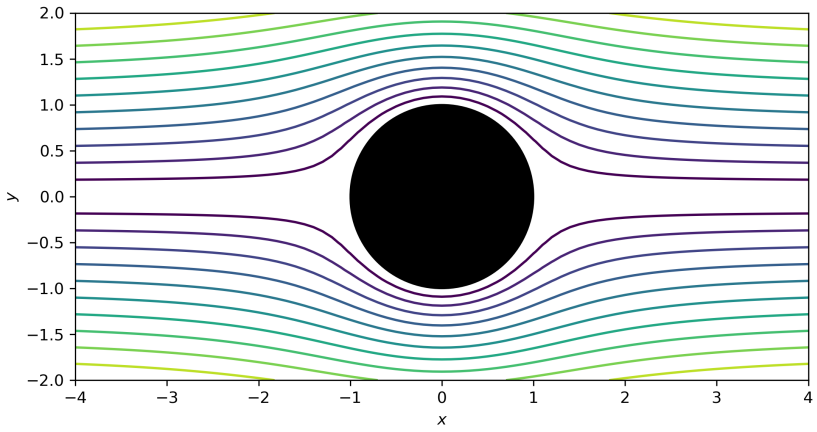
The flow field must also satisfy the boundary condition that $\vec{V} = U\hat{\mathbf{i}}$ as $r \to \infty$, or in polar coordinates: $u_r = U\cos\theta$ and $u_\theta = -U\sin\theta$ (unidirectional flow in the x-direction). As $r \to \infty$:

$$\lim_{r\to\infty} u_r = \lim_{r\to\infty} U\left(1 - \frac{a^2}{r^2}\right)\cos\theta = U\cos\theta$$

$$\lim_{r\to\infty} u_\theta = \lim_{r\to\infty} -U\left(1+\frac{a^2}{r^2}\right)\sin\theta = -U\sin\theta$$

Therefore this flow field defined by the given stream function satisfies the boundary conditions.

Q4(b) In order to plot the streamlines, we only need to plot lines of constant stream function (isocontours).



Q5(a) The only possible solution is of the form:

$$w(x,y) = \big(A\cos(\mu x) + B\sin(\mu x)\big)\big(C\cosh(\mu y) + D\sinh(\mu y)\big)$$

Note: we have replaced the exponentials with hyperbolic trig.

- b.c. $w(x=0,y) = A\big(C\cosh(\mu y) + D\sinh(\mu y)\big) = 0 \implies A = 0$

  $$\implies w(x,y) = \sin(\mu x)\big(C'\cosh(\mu y) + D'\sinh(\mu y)\big)$$

  where $C' = BC$ and $D' = BD$.

- b.c. $w(x,y=0) = C'\sin(\mu x) = 0 \implies C' = 0$

  $$\implies w(x,y) = D'\sin(\mu x)\sinh(\mu y)$$

- b.c. $w(x=a,y) = D'\sin(\mu a)\sinh(\mu y) = 0 \implies \sin(\mu a) = 0$
  $\implies \mu = \frac{n\pi}{a} \implies w_n(x,y) = D'\sin\left(\frac{n\pi x}{a}\right)\sinh\left(\frac{n\pi y}{a}\right)$

  $$\implies w(x,y) = \sum_{n=1}^{\infty} D_n \sin\left(\frac{n\pi x}{a}\right)\sinh\left(\frac{n\pi y}{a}\right)$$

- b.c. $w(x, y = b) = \sum_{n=1}^{\infty} D_n \sin\left(\frac{n\pi x}{a}\right) \sinh\left(\frac{n\pi b}{a}\right) = w_0 \sin\left(\frac{\pi x}{a}\right)$

$$\implies n = 1 \qquad \text{and} \qquad D_1 = \frac{w_0}{\sinh\left(\frac{\pi b}{a}\right)}$$

The final solution is:

$$w(x, y) = \frac{w_0}{\sinh\left(\frac{\pi b}{a}\right)} \sin\left(\frac{\pi x}{a}\right) \sinh\left(\frac{\pi y}{a}\right)$$

Q5(b) As for part (a), the only possible solution is of the form:

$$w(x, y) = \big(A\cos(\mu x) + B\sin(\mu x)\big)\big(C\cosh(\mu y) + D\sinh(\mu y)\big)$$

- b.c. $w(x = 0, y) = A\big(C\cosh(\mu y) + D\sinh(\mu y)\big) = 0 \implies A = 0$

$$\implies w(x, y) = \sin(\mu x)\big(C'\cosh(\mu y) + D'\sinh(\mu y)\big)$$

where $C' = AC$ and $D' = AD$.

- b.c. $w(x, y = 0) = C'\sin(\mu x) = 0 \implies C' = 0$

$$\implies w(x, y) = D'\sin(\mu x)\sinh(\mu y)$$

- b.c. $\left.\frac{\partial w}{\partial x}\right|_{x=a} = \mu D'\cos(\mu a)\sinh(\mu y) = 0 \implies \cos(\mu a) = 0 \implies$
$\mu = \frac{(2n-1)\pi}{2a} \implies w_n(x, y) = D'\sin\left(\frac{(2n-1)\pi x}{2a}\right)\sinh\left(\frac{(2n-1)\pi y}{2a}\right)$

$$\implies w(x, y) = \sum_{n=1}^{\infty} D_n \sin\left(\frac{(2n-1)\pi x}{2a}\right)\sinh\left(\frac{(2n-1)\pi y}{2a}\right)$$

- b.c. $w(x, y = b) = \sum_{n=1}^{\infty} D_n \sin\left(\frac{(2n-1)\pi x}{2a}\right)\sinh\left(\frac{(2n-1)\pi b}{2a}\right)$
$= w_0 \sin\left(\frac{\pi x}{2a}\right) \implies n = 1$ and $D_1 = \frac{w_0}{\sinh\left(\frac{\pi b}{2a}\right)}$

The final solution is:

$$w(x, y) = \frac{w_0}{\sinh\left(\frac{\pi b}{2a}\right)} \sin\left(\frac{\pi x}{2a}\right) \sinh\left(\frac{\pi y}{2a}\right)$$

E.4 Elliptic PDEs: Finite Difference method solutions

Q1(a) For an internal grid point:

$$T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j} = 0$$

For a grid point on the bottom boundary:

$$T_{i+1,1} + T_{i-1,1} + T_{i,2} + T_{i,0} - 4T_{i,j} = 0$$

but we also have the boundary condition:

$$\left.\frac{\partial T}{\partial y}\right|_{y=0} = 0 \implies \frac{T_{i,2} - T_{i,0}}{2\Delta y} = 0 \implies T_{i,0} = T_{i,2}$$

and substituting into the previous equation:

$$T_{i+1,1} + T_{i-1,1} + 2T_{i,2} - 4T_{i,j} = 0$$

Likewise, on the left boundary, we have:

$$T_{1,j+1} + T_{1,j-1} + 2T_{2,j} - 4T_{1,j} = 0$$

and at the bottom left corner:

$$2T_{1,2} + 2T_{2,1} - 4T_{1,1} = 0$$

For implementation with the Liebmann method, these equations need to be rearranged:

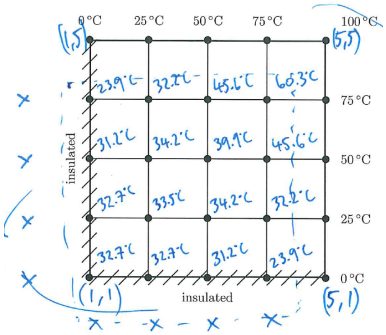
$$T_{i,j} = \frac{1}{4}(T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1}) \quad \text{for} \quad 2 \leqslant i \leqslant 4, \quad 2 \leqslant j \leqslant 4$$

$$T_{i,1} = \frac{1}{4}(T_{i+1,1} + T_{i-1,1} + 2T_{i,2}) \quad \text{for} \quad 2 \leqslant i \leqslant 4$$

$$T_{1,j} = \frac{1}{4}(T_{1,j+1} + T_{1,j-1} + 2T_{2,j}) \quad \text{for} \quad 2 \leqslant j \leqslant 4$$

$$T_{1,1} = \frac{1}{4}(2T_{1,2} + 2T_{2,1})$$

Referring to results in the Python code, we find that the Liebmann method converges in 59 iterations for a stopping criterion of 0.01 %.



Q1(b) See the figure beside with the numeric values added to each degree of freedom (dof). In order to solve the resulting system of equations using the direct method, we need to build the matrix $\underline{\underline{A}}$ and right hand vector $\vec{b}$.

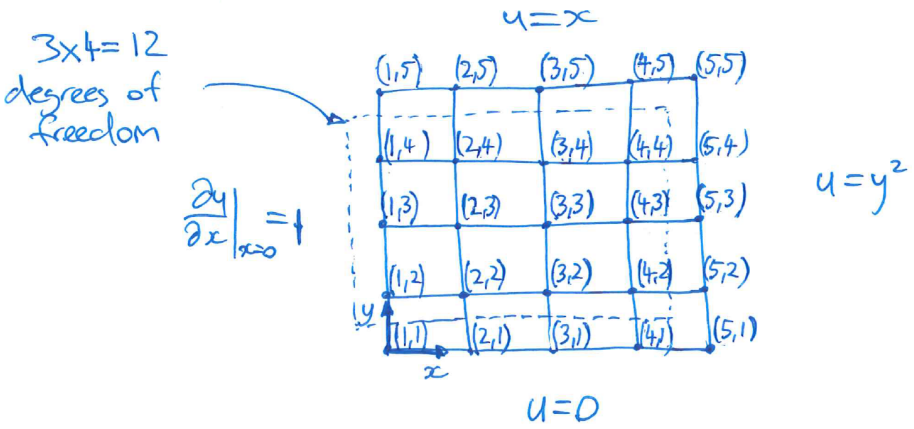
Following the same process as we did for Equation 3.16 but now with dof on the left and bottom boundaries, we have a 16×16 matrix of coefficients $\underline{\underline{A}}$, rhs vector $\vec{b}$ of size 16×1 and dependent variable vector $\vec{T}$ of size 16×1.

See the Python code for the entries, and then solving using the function `np.linalg.solve` gives the same results as the Liebmann method within three significant figures.

λ	Iterations
1	59
1.1	49
1.2	40
1.3	32
1.4	24
1.5	17
1.6	23
1.7	32
1.8	49
1.9	100
2	No convergence

Q1(c) Yes, the rate of convergence can be improved with over-relaxation as shown in the table beside, although note that not all λ values improve the rate. The optimal value (corresponding to the fewest iterations required) is $\lambda = 1.5$.

Q2 First, draw the computational domain. Note: values of u on the top and right hand side boundaries are spatially dependent, e.g. $u_{5,2} = y^2 = (\frac{1}{4})^2$.



For the internal grid points, we can use the same discretisation (central difference) from Equation 3.14:

$$u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j} = 0$$

For the middle grid points on the left boundary, we introduce a ghost node because of the Neumann b.c.:

$$u_{2,j} + u_{0,j} + u_{1,j+1} + u_{1,j-1} - 4u_{1,j} = 0$$

so we discretise the b.c. and rearrange for the ghost node:

$$\left.\frac{\partial u}{\partial x}\right|_{x=0} = 1 \implies \frac{u_{2,j} - u_{0,j}}{2\Delta x} = 1 \implies u_{0,j} = u_{2,j} - 2\Delta x$$

and substituting:

$$2u_{2,j} + u_{1,j+1} + u_{1,j-1} - 4u_{1,j} = 2\Delta x = 2 \times \frac{1}{4} = \frac{1}{2}$$

then the system of equations $\underline{\underline{A}}\vec{u} = \vec{b}$ can be populated.

E.5 Transient PDEs

Q1 The governing equation for this problem is:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \quad \text{subject to} \quad \begin{cases} T(x=0,t) = 0 \\ T(x=L,t) = 0 \\ T(x,t=0) = T_0 \end{cases}$$

Solution of the form:

$$T(x,t) = \big(A\cos(\lambda x) + B\sin(\lambda x)\big)e^{-\alpha\lambda^2 t}$$

- b.c. $T(x=0,t) = Ae^{-\alpha\lambda^2 t} = 0 \implies A = 0$

- b.c. $T(x=L,t) = B\sin(\lambda L)e^{-\alpha\lambda^2 t} = 0$

  $$\text{choose} \quad \lambda = \frac{n\pi}{L} \quad \text{with} \quad n = 1,2,3,...$$

  one particular solution will therefore be:

  $$T_n = B_n \sin\left(\frac{n\pi x}{L}\right) e^{-\alpha\left(\frac{n\pi}{L}\right)^2 t}$$

  taking a combination of all of these solutions:

  $$T(x,t) = \sum_{n=1}^{\infty} T_n(x,t)$$

- i.c. $T(x,t=0) = T_0 = \sum_{n=1,3,5,...} \frac{4T_0}{n\pi}\sin\left(\frac{n\pi x}{L}\right)$ using the Fourier sine series of $f(x) = T_0$ for $0 \leqslant x \leqslant L$, and then matching coefficients between these:

  $$T(x,t=0) = \sum_{n=1}^{\infty} B_n \sin\left(\frac{n\pi x}{L}\right) = \sum_{n=1,3,5,...} \frac{4T_0}{n\pi}\sin\left(\frac{n\pi x}{L}\right)$$

  $$\implies B_n = \frac{4T_0}{n\pi} \quad \text{if} \quad n = 1,3,5,... \quad \text{and} \quad B_n = 0 \quad \text{otherwise.}$$

Finally, the solution:

$$T(x,t) = \sum_{n=1,3,5,...} \frac{4T_0}{n\pi}\sin\left(\frac{n\pi x}{L}\right) e^{-\alpha\left(\frac{n\pi}{L}\right)^2 t}$$

and refer to the Python script which calculates this solution. Trial and error shows that it takes $\approx$ 36 minutes for the temperature to drop below 5 °C.

Q2 We consider a solution of the form:

$$T(x,t) = \big(A\cos(\lambda x) + B\sin(\lambda x)\big)e^{-\alpha\lambda^2 t}$$

- b.c. $T(x=0,t) = Ae^{-\alpha\lambda^2 t} = 0 \implies A = 0$
- b.c. $\left.\frac{\partial T}{\partial x}\right|_{x=L} = \lambda B\cos(\lambda L)e^{-\alpha\lambda^2 t}$

  $\implies \cos(\lambda L) = 0 \implies \lambda L = (n-\frac{1}{2})\pi \quad \text{where} \quad n = 1,2,3,...$

  the combination of particular solutions are:

$$T(x,t) = \sum_{n=1}^{\infty} B_n \sin\left((n-\tfrac{1}{2})\frac{\pi x}{L}\right) e^{-\alpha\left((n-\frac{1}{2})\frac{\pi}{L}\right)^2 t}$$

- i.c. $T(x,t=0) = \sum_{n=1}^{\infty} B_n \sin\left((n-\frac{1}{2})\frac{\pi x}{L}\right) = T_0 \sin\left(\frac{3\pi x}{2L}\right)$

$$\implies B_n = T_0 \quad \text{and} \quad n = 2$$

The final solution is:

$$T(x,t) = T_0 \sin\left(\frac{3\pi x}{2L}\right) e^{-\alpha\left(\frac{3\pi}{2L}\right)^2 t}$$

Q3(a) The PDE is 2nd order because of the 2nd order derivatives in T.

Q3(b) Yes, the PDE is linear in T.

Q3(c) No, because the heat source Q does not involve T.

Q3(d) $A = k,\ B = 0,\ C = k \implies B^2 - 4AC = 0 - 4k^2 < 0 \implies$ elliptic.

Q3(e) We assume ρ, c, and k are uniform throughout and therefore do not discretise those variables.

$$\rho c\left(u_{i,j}\frac{T_{i+1,j} - T_{i-1,j}}{2\Delta x} + v_{i,j}\frac{T_{i,j+1} - T_{i,j-1}}{2\Delta y}\right)$$

$$= k\left(\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2}\right) + Q_{i,j}$$

Given that $\Delta x = \Delta y$, and grouping coefficients of T:

$$\left(\rho c\frac{u_{i,j}}{2\Delta x} - \frac{k}{\Delta x^2}\right)T_{i+1,j} + \left(-\rho c\frac{u_{i,j}}{2\Delta x} - \frac{k}{\Delta x^2}\right)T_{i-1,j} + \left(\rho c\frac{v_{i,j}}{2\Delta x} - \frac{k}{\Delta x^2}\right)T_{i,j+1}$$

$$+\left(-\rho c\frac{v_{i,j}}{2\Delta x}-\frac{k}{\Delta x^2}\right)T_{i,j-1}+\left(\frac{4k}{\Delta x^2}\right)T_{i,j}=Q_{i,j}$$

Q3(f) Substituting for those dimensionless variables into the governing equation:

$$\rho c\left(V_0\tilde{u}\frac{T_0}{L}\frac{\partial\tilde{T}}{\partial\tilde{x}}+V_0\tilde{v}\frac{T_0}{L}\frac{\partial\tilde{T}}{\partial\tilde{y}}\right)=k\left(\frac{T_0}{L^2}\frac{\partial^2\tilde{T}}{\partial\tilde{x}^2}+\frac{T_0}{L^2}\frac{\partial^2\tilde{T}}{\partial\tilde{y}^2}\right)+Q$$

Dividing through by $\frac{\rho c V_0 T_0}{L}$:

$$\tilde{u}\frac{\partial\tilde{T}}{\partial\tilde{x}}+\tilde{v}\frac{\partial\tilde{T}}{\partial\tilde{y}}=\frac{k}{\rho c V_0 L}\left(\frac{\partial^2\tilde{T}}{\partial\tilde{x}^2}+\frac{\partial^2\tilde{T}}{\partial\tilde{y}^2}\right)+\frac{L}{\rho c V_0 T_0}Q$$

Since $\alpha=\frac{k}{\rho c}$ is the thermal conductivity, we have:

$$\tilde{u}\frac{\partial\tilde{T}}{\partial\tilde{x}}+\tilde{v}\frac{\partial\tilde{T}}{\partial\tilde{y}}=\frac{\alpha}{V_0 L}\left(\frac{\partial^2\tilde{T}}{\partial\tilde{x}^2}+\frac{\partial^2\tilde{T}}{\partial\tilde{y}^2}\right)+\frac{L}{\rho c V_0 T_0}Q$$

This equation is dimensionless because the left hand side is clearly dimensionless as only dimensionless variables are present. Note: the dimensionless number $\mathrm{Pe}=\frac{V_0 L}{\alpha}$ is called the Peclet number.

Q4(a) In order to make the governing equations dimensionless, we first need to introduce those dimensionless variables into the governing equations. First, consider the terms which involve derivatives (using the chain rule):

$$\frac{\partial u}{\partial x}=\frac{\partial(V_0\tilde{u})}{\partial x}=V_0\frac{\partial\tilde{u}}{\partial\tilde{x}}\frac{d\tilde{x}}{dx}=\frac{V_0}{D}\frac{\partial\tilde{u}}{\partial\tilde{x}}$$

$$\frac{\partial^2 u}{\partial x^2}=\frac{\partial}{\partial x}\left(\frac{\partial u}{\partial x}\right)=\frac{\partial}{\partial x}\left(\frac{V_0}{D}\frac{\partial\tilde{u}}{\partial\tilde{x}}\right)=\frac{V_0}{D}\frac{\partial}{\partial\tilde{x}}\left(\frac{\partial\tilde{u}}{\partial\tilde{x}}\right)\frac{d\tilde{x}}{dx}=\frac{V_0}{D^2}\frac{\partial^2\tilde{u}}{\partial\tilde{x}^2}$$

Likewise, for the other derivatives:

$$\frac{\partial u}{\partial y}=\frac{V_0}{D}\frac{\partial\tilde{u}}{\partial\tilde{y}}\quad,\qquad\frac{\partial v}{\partial x}=\frac{V_0}{D}\frac{\partial\tilde{v}}{\partial\tilde{x}}\quad,\qquad\frac{\partial v}{\partial y}=\frac{V_0}{D}\frac{\partial\tilde{v}}{\partial\tilde{y}}\quad,$$

$$\frac{\partial^2 u}{\partial y^2}=\frac{V_0}{D^2}\frac{\partial^2\tilde{u}}{\partial\tilde{y}^2}\quad,\qquad\frac{\partial^2 v}{\partial x^2}=\frac{V_0}{D^2}\frac{\partial^2\tilde{v}}{\partial\tilde{x}^2}\quad,\qquad\frac{\partial^2 v}{\partial y^2}=\frac{V_0}{D^2}\frac{\partial^2\tilde{v}}{\partial\tilde{y}^2}\quad.$$

Substituting these dimensionless variables and derivatives into the governing equations give:

$$\rho\left(\frac{V_0^2}{D}\tilde{u}\frac{\partial\tilde{u}}{\partial\tilde{x}}+\frac{V_0^2}{D}\tilde{v}\frac{\partial\tilde{u}}{\partial\tilde{y}}\right)=-\frac{p_0}{D}\frac{\partial\tilde{p}}{\partial\tilde{x}}+\mu\left(\frac{V_0}{D^2}\frac{\partial^2\tilde{u}}{\partial\tilde{x}^2}+\frac{V_0}{D^2}\frac{\partial^2\tilde{u}}{\partial\tilde{y}^2}\right)$$

and dividing throughout by $\frac{\rho V_0^2}{D}$:

$$\tilde{u}\frac{\partial \tilde{u}}{\partial \tilde{x}} + \tilde{v}\frac{\partial \tilde{u}}{\partial \tilde{y}} = -\frac{\mu}{\rho V_0 D}\frac{\partial \tilde{p}}{\partial \tilde{x}} + \frac{\mu}{\rho V_0 D}\left(\frac{\partial^2 \tilde{u}}{\partial \tilde{x}^2} + \frac{\partial^2 \tilde{u}}{\partial \tilde{y}^2}\right)$$

which is dimensionless because the left hand side is dimensionless. Remembering that $\text{Re} = \frac{\rho V_0 D}{\mu}$ is the Reynolds number, we can rewrite as:

$$\tilde{u}\frac{\partial \tilde{u}}{\partial \tilde{x}} + \tilde{v}\frac{\partial \tilde{u}}{\partial \tilde{y}} = -\frac{1}{\text{Re}}\frac{\partial \tilde{p}}{\partial \tilde{x}} + \frac{1}{\text{Re}}\left(\frac{\partial^2 \tilde{u}}{\partial \tilde{x}^2} + \frac{\partial^2 \tilde{u}}{\partial \tilde{y}^2}\right)$$

$$\implies \text{Re}\left(\tilde{u}\frac{\partial \tilde{u}}{\partial \tilde{x}} + \tilde{v}\frac{\partial \tilde{u}}{\partial \tilde{y}}\right) = -\frac{\partial \tilde{p}}{\partial \tilde{x}} + \left(\frac{\partial^2 \tilde{u}}{\partial \tilde{x}^2} + \frac{\partial^2 \tilde{u}}{\partial \tilde{y}^2}\right)$$

for the first equation (momentum in the x-direction), likewise the second equation:

$$\text{Re}\left(\tilde{u}\frac{\partial \tilde{v}}{\partial \tilde{x}} + \tilde{v}\frac{\partial \tilde{v}}{\partial \tilde{y}}\right) = -\frac{\partial \tilde{p}}{\partial \tilde{y}} + \left(\frac{\partial^2 \tilde{v}}{\partial \tilde{x}^2} + \frac{\partial^2 \tilde{v}}{\partial \tilde{y}^2}\right) - \frac{1}{\text{Fr}^2}$$

where $\text{Fr} = \frac{V_0}{\sqrt{gD}}$ is the Froude number (dimensionless), which describes the ratio of the flow inertia to that from gravity.

We can see that if Re is small, then the convection terms on the left hand side have a small (and negligible for $\text{Re} \to 0$) influence on the fluid dynamics.

Q4(b) For this case:

$$\text{Re} = \frac{\rho V_0 D}{\mu} = \frac{1430 \times 0.1 \times 0.1}{100} = 0.143 \ll 1$$

which suggests that the terms on the left hand side are negligible as discussed above.

E.6 Transient PDEs: numerical solutions

Q1(a) Discretising with FTCS:

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = \alpha \frac{T_{i-1}^n - 2T_i^n + T_{i+1}^n}{\Delta x^2}$$

$$\implies T_i^{n+1} = T_i^n + \lambda(T_{i-1}^n - 2T_i^n + T_{i+1}^n) \quad \text{where} \quad \lambda = \frac{\alpha \Delta t}{\Delta x^2}$$

and see Python script (set $\lambda = 0.45$).

Q1(b) Now with $\lambda = 0.55$, the solution leads to instability and does not converge.

Q1(c) Discretisation is the same as Equation 6.12. The Crank–Nicolson scheme is implicit, and we do not expect any issues with stability. Refer to the Python script which implements this numerical scheme and confirms that the solution is stable for both $\lambda = 0.45$ and $\lambda = 0.55$.

Q2(a) Introduce the dimensionless variables into the governing PDE:

$$\frac{\partial c}{\partial t} = \frac{\partial}{\partial t}\Big(c_0 + (c_s - c_0)\tilde{c}\Big) = \frac{\partial c_0}{\partial t} + \frac{\partial}{\partial t}\Big((c_s - c_0)\tilde{c}\Big)$$

$$= (c_s - c_0)\frac{\partial \tilde{c}}{\partial \tilde{t}}\frac{d\tilde{t}}{dt} = \frac{c_s - c_0}{L^2} D \frac{\partial \tilde{c}}{\partial \tilde{t}}$$

Likewise:

$$\frac{\partial c}{\partial x} = \frac{c_s - c_0}{L}\frac{\partial \tilde{c}}{\partial \tilde{x}} \quad \text{and} \quad \frac{\partial^2 c}{\partial x^2} = \frac{c_s - c_0}{L^2}\frac{\partial^2 \tilde{c}}{\partial \tilde{x}^2}$$

Combining:

$$\frac{c_s - c_0}{L^2} D \frac{\partial \tilde{c}}{\partial \tilde{t}} = D \frac{c_s - c_0}{L^2}\frac{\partial^2 \tilde{c}}{\partial \tilde{x}^2}$$

$$\implies \frac{\partial \tilde{c}}{\partial \tilde{t}} = \frac{\partial^2 \tilde{c}}{\partial \tilde{x}^2} \quad \text{as required.}$$

For the boundary and initial conditions:

- b.c. $c(x = 0, t) = c_s \implies \tilde{c} = \frac{c_s - c_0}{c_s - c_0} = 1 \implies \tilde{c}(\tilde{x} = 0, \tilde{t} > 0) = 1$
- b.c. $c(x = L, t) = c_0 \implies \tilde{c} = \frac{c_0 - c_0}{c_s - c_0} = 0 \implies \tilde{c}(\tilde{x} = 1, \tilde{t} > 0) = 0$
- i.c. $c(x, t = 0) = c_0 \implies \tilde{c} = \frac{c_0 - c_0}{c_s - c_0} = 0 \implies \tilde{c}(\tilde{x}, \tilde{t} = 0) = 0$

Q2(b) Discretising with BTCS:

$$\frac{\tilde{c}_i^{n+1} - \tilde{c}_i^n}{\Delta \tilde{t}} = \frac{\tilde{c}_{i-1}^{n+1} - 2\tilde{c}_i^{n+1} + \tilde{c}_{i+1}^{n+1}}{\Delta \tilde{x}^2}$$

$$\implies -\lambda \tilde{c}_{i-1}^{n+1} + (1 + 2\lambda)\tilde{c}_i^{n+1} - \lambda \tilde{c}_{i+1}^{n+1} = \tilde{c}_i^n$$

where $\lambda = \frac{\Delta \tilde{t}}{\Delta \tilde{x}^2}$. This discretised equation holds for all interior points (the unknowns, as Dirichlet conditions are applied on the ends).

Q2(c) The BTCS has the advantage of being unconditionally stable $\implies$ no restriction on the time step size. However, each time step requires the solution of a linear system of equations $\implies$ increased algorithm complexity and computation time for each time step.

E.9 Finite Element application (1-D)

Q1(a) We are going to use the method of weighted residuals: *Element ①*

$$\int_{x_1}^{x_2} \left(\frac{d^2\widetilde{T}}{dx^2} + \alpha \right) N_i \, dx = 0 \quad \text{for} \quad i = 1, 2$$

with $\widetilde{T} = N_1 T_1 + N_2 T_2$.

First, consider $\int_{x_1}^{x_2} \frac{d^2\widetilde{T}}{dx^2} N_i \, dx$ using integration by parts:

$$\int_{x_1}^{x_2} \frac{d^2\widetilde{T}}{dx^2} N_i \, dx = \left[N_i \frac{d\widetilde{T}}{dx} \right]_{x_1}^{x_2} - \int_{x_1}^{x_2} \frac{d\widetilde{T}}{dx} \frac{dN_i}{dx} \, dx$$

- For $N_i = N_1$:

  $$\int_{x_1}^{x_2} \frac{d^2\widetilde{T}}{dx^2} N_1 \, dx = N_1(x_2) \left. \frac{d\widetilde{T}}{dx} \right|_{x_2} - N_1(x_1) \left. \frac{d\widetilde{T}}{dx} \right|_{x_1} - \int_{x_1}^{x_2} \frac{d\widetilde{T}}{dx} \frac{dN_1}{dx} \, dx$$

  and we know $\frac{d\widetilde{T}}{dx} = \frac{T_2 - T_1}{x_2 - x_1}$ and $\frac{dN_1}{dx} = \frac{-1}{x_2 - x_1}$.

  $$\implies \int_{x_1}^{x_2} \frac{d^2\widetilde{T}}{dx^2} N_1 \, dx = - \left. \frac{d\widetilde{T}}{dx} \right|_{x_1} + \int_{x_1}^{x_2} \frac{T_2 - T_1}{(x_2 - x_1)^2} \, dx$$

  $$= - \left. \frac{d\widetilde{T}}{dx} \right|_{x_1} + \frac{T_2 - T_1}{(x_2 - x_1)^2} \int_{x_1}^{x_2} dx$$

  $$= - \left. \frac{d\widetilde{T}}{dx} \right|_{x_1} + \frac{T_2 - T_1}{x_2 - x_1}$$

- Similarly for $N_i = N_2$:

  $$\int_{x_1}^{x_2} \frac{d^2\widetilde{T}}{dx^2} N_2 \, dx = \left. \frac{d\widetilde{T}}{dx} \right|_{x_2} + \frac{T_1 - T_2}{x_2 - x_1}$$

Second, consider $\int_{x_1}^{x_2} \alpha N_i \, dx$ for $i = 1, 2$:

- For $N_i = N_1$:

  $$\int_{x_1}^{x_2} \alpha N_1 \, dx = \alpha \int_{x_1}^{x_2} N_1 \, dx = \alpha \frac{x_2 - x_1}{2}$$

- For $N_i = N_2$:

  $$\int_{x_1}^{x_2} \alpha N_2 \, dx = \alpha \int_{x_1}^{x_2} N_2 \, dx = \alpha \frac{x_2 - x_1}{2}$$

Combining equations for $i = 1$ gives the first element equation:

$$-\left.\frac{d\widetilde{T}}{dx}\right|_{x_1} + \frac{T_2 - T_1}{x_2 - x_1} + \alpha\frac{x_2 - x_1}{2} = 0$$

where $x_2 - x_1 = \Delta x$:

$$\implies \frac{1}{\Delta x}(T_1 - T_2) = -\left.\frac{d\widetilde{T}}{dx}\right|_{x_1} + \alpha\frac{\Delta x}{2}$$

Similarly combining equations for $i = 2$ gives:

$$\left.\frac{d\widetilde{T}}{dx}\right|_{x_2} + \frac{T_1 - T_2}{x_2 - x_1} + \alpha\frac{x_2 - x_1}{2} = 0$$

$$\implies \frac{1}{\Delta x}(-T_1 + T_2) = \left.\frac{d\widetilde{T}}{dx}\right|_{x_2} + \alpha\frac{\Delta x}{2}$$

In matrix form, the element equations for element ①:

$$\frac{1}{\Delta x}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}\begin{Bmatrix} T_1 \\ T_2 \end{Bmatrix} = \begin{Bmatrix} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_1} + \alpha\frac{\Delta x}{2} \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_2} + \alpha\frac{\Delta x}{2} \end{Bmatrix}$$

Similarly for elements ②, ③ and ④:

Element ②

$$\frac{1}{\Delta x}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}\begin{Bmatrix} T_2 \\ T_3 \end{Bmatrix} = \begin{Bmatrix} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_2} + \alpha\frac{\Delta x}{2} \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_3} + \alpha\frac{\Delta x}{2} \end{Bmatrix}$$

Element ③

$$\frac{1}{\Delta x}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}\begin{Bmatrix} T_3 \\ T_4 \end{Bmatrix} = \begin{Bmatrix} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_3} + \alpha\frac{\Delta x}{2} \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_4} + \alpha\frac{\Delta x}{2} \end{Bmatrix}$$

Element ④

$$\frac{1}{\Delta x}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}\begin{Bmatrix} T_4 \\ T_5 \end{Bmatrix} = \begin{Bmatrix} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_4} + \alpha\frac{\Delta x}{2} \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_5} + \alpha\frac{\Delta x}{2} \end{Bmatrix}$$

Q1(b) The assembly process involves placing the element stiffness matrices $\underline{\underline{K}}^e$ into the global stiffness matrix $\underline{\underline{K}}^g$ which has a size of 5×5 because there are 5 degrees of freedom. We start with an empty $\underline{\underline{K}}^g$ and accumulate:

$$\text{Adding} \quad \underline{\underline{K}}^e_1 : \frac{1}{\Delta x} \begin{bmatrix} 1 & -1 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\text{Adding} \quad \underline{\underline{K}}^e_2 : \frac{1}{\Delta x} \begin{bmatrix} 1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\text{Adding} \quad \underline{\underline{K}}^e_3 \text{ and } \underline{\underline{K}}^e_4 : \frac{1}{\Delta x} \begin{bmatrix} 1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 1 \end{bmatrix} = \underline{\underline{K}}^g$$

Similarly for the global forcing vector $\vec{F}^g$:

$$\text{Adding} \quad \vec{F}^e_1 : \begin{Bmatrix} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_1} + \alpha\frac{\Delta x}{2} \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_2} + \alpha\frac{\Delta x}{2} \\ 0 \\ 0 \\ 0 \end{Bmatrix}$$

$$\text{Adding} \quad \vec{F}_2^e : \begin{Bmatrix} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_1} + \alpha\frac{\Delta x}{2} \\ \alpha\Delta x \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_3} + \alpha\frac{\Delta x}{2} \\ 0 \\ 0 \end{Bmatrix}$$

$$\text{Adding} \quad \vec{F}_3^e \text{ and } \vec{F}_4^e : \begin{Bmatrix} -\left.\frac{d\widetilde{T}}{dx}\right|_{x_1} + \alpha\frac{\Delta x}{2} \\ \alpha\Delta x \\ \alpha\Delta x \\ \alpha\Delta x \\ \left.\frac{d\widetilde{T}}{dx}\right|_{x_5} + \alpha\frac{\Delta x}{2} \end{Bmatrix} = \vec{F}^g$$

Q1(c) As $L = 1\,\text{m} \implies \Delta x = \frac{L}{4} = 0.25\,\text{m}$. The left hand side boundary condition requires $T_1 = 0$ and the right requires $\left.\frac{d\widetilde{T}}{dx}\right|_{x_5} = 0$. The first equation:

$$\frac{1}{\Delta x}T_1 - \frac{1}{\Delta x}T_2 = -\left.\frac{d\widetilde{T}}{dx}\right|_{x_1} + \alpha\frac{\Delta x}{2}$$

$$\implies \left.\frac{d\widetilde{T}}{dx}\right|_{x_1} - 4T_2 = 25$$

The second equation:

$$-\frac{1}{\Delta x}T_1 + \frac{2}{\Delta x}T_2 - \frac{1}{\Delta x}T_3 = \alpha\Delta x$$

$$\implies 8T_2 - 4T_3 = 50$$

The third equation:

$$-\frac{1}{\Delta x}T_2 + \frac{2}{\Delta x}T_3 - \frac{1}{\Delta x}T_4 = \alpha\Delta x$$

$$\implies -4T_2 + 8T_3 - 4T_4 = 50$$

The fourth equation:

$$-4T_3 + 8T_4 - 4T_5 = 50$$

The fifth equation:

$$-4T_4 + 4T_5 = 25 \quad \text{as} \quad \left.\frac{d\widetilde{T}}{dx}\right|_{x_5} = 0$$

In matrix form:

$$\begin{bmatrix} 1 & -4 & 0 & 0 & 0 \\ 0 & 8 & -4 & 0 & 0 \\ 0 & -4 & 8 & -4 & 0 \\ 0 & 0 & -4 & 8 & -4 \\ 0 & 0 & 0 & -4 & 4 \end{bmatrix} \begin{Bmatrix} \left.\frac{d\widetilde{T}}{dx}\right|_{x_1} \\ T_2 \\ T_3 \\ T_4 \\ T_5 \end{Bmatrix} = \begin{Bmatrix} 25 \\ 50 \\ 50 \\ 50 \\ 25 \end{Bmatrix}$$

Solving this linear system of equations for the vector of unknowns:

$$\begin{Bmatrix} \left.\frac{d\widetilde{T}}{dx}\right|_{x_1} \\ T_2 \\ T_3 \\ T_4 \\ T_5 \end{Bmatrix} = \begin{Bmatrix} 200\,\mathrm{K/m} \\ 43.75\,^\circ\mathrm{C} \\ 75\,^\circ\mathrm{C} \\ 93.75\,^\circ\mathrm{C} \\ 100\,^\circ\mathrm{C} \end{Bmatrix}$$

Q2 We can recognise the governing equation is the same as from Q1 if α is replaced by $-\frac{P}{A_c E}$ and therefore the discretisation process will be identical and gives:

$$\frac{1}{\Delta x}\begin{bmatrix} 1 & -1 & 0 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & 0 & -1 & 1 \end{bmatrix} \begin{Bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \end{Bmatrix} = \begin{Bmatrix} -\left.\frac{d\tilde{u}}{dx}\right|_{x_1} - \frac{P}{A_c E}\frac{\Delta x}{2} \\ -\frac{P}{A_c E}\Delta x \\ -\frac{P}{A_c E}\Delta x \\ -\frac{P}{A_c E}\Delta x \\ -\frac{P}{A_c E}\Delta x \\ \left.\frac{d\tilde{u}}{dx}\right|_{x_6} - \frac{P}{A_c E}\frac{\Delta x}{2} \end{Bmatrix}$$

Note: $\frac{\Delta x}{L} = 5 \implies$ we now have five elements and six degrees of freedom.

Applying the boundary conditions:

- b.c. $u(x=0) = u_1 = 0$:

$$\Delta x \left.\frac{d\tilde{u}}{dx}\right|_{x_1} + u_1 - u_2 = -\frac{P}{A_c E}\frac{\Delta x^2}{2}$$

$$\implies 2\left.\frac{d\tilde{u}}{dx}\right|_{x_1} - u_2 = -100 \times 10^{-9}$$

- b.c. $u(x=L) = u_6 = 0$:

$$-\Delta x \left.\frac{d\tilde{u}}{dx}\right|_{x_6} - u_5 + u_6 = -\frac{P}{A_c E}\frac{\Delta x^2}{2}$$

$$-2\left.\frac{d\tilde{u}}{dx}\right|_{x_6} - u_5 = -100 \times 10^{-9}$$

In matrix form:

$$\begin{bmatrix} 2 & -1 & 0 & 0 & 0 & 0 \\ 0 & 2 & -1 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & 0 & -1 & 2 & 0 \\ 0 & 0 & 0 & 0 & -1 & -2 \end{bmatrix} \begin{Bmatrix} \left.\frac{d\tilde{u}}{dx}\right|_{x_1} \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ \left.\frac{d\tilde{u}}{dx}\right|_{x_6} \end{Bmatrix} = \begin{Bmatrix} -100 \times 10^{-9} \\ -200 \times 10^{-9} \\ -200 \times 10^{-9} \\ -200 \times 10^{-9} \\ -200 \times 10^{-9} \\ -100 \times 10^{-9} \end{Bmatrix}$$

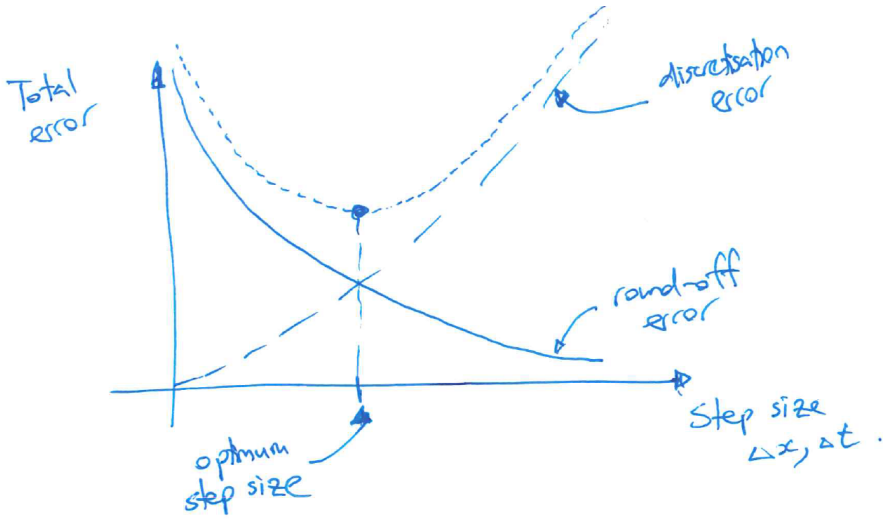
The solution of this system of equations is:

$$\begin{Bmatrix} \left.\frac{d\tilde{u}}{dx}\right|_{x_1} \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ \left.\frac{d\tilde{u}}{dx}\right|_{x_6} \end{Bmatrix} = \begin{Bmatrix} -0.25 \times 10^{-6} \\ -0.4 \times 10^{-6} \\ -0.6 \times 10^{-6} \\ -0.6 \times 10^{-6} \\ -0.4 \times 10^{-6} \\ 0.25 \times 10^{-6} \end{Bmatrix}$$

Q3 Discretisation error: arises because of the fact we approximate a problem which is continuous in space and time by a discrete form. The discretisation error can be minimised by reducing Δx and Δt.

Round-off error: arises because real numbers are approximated/truncated when they are stored on a finite number of bits on a computer. This error increases when the number of floating point operations (flops) increases. Using small Δx and Δt increases the round-off error.

The total error is the sum of both the discretisation error and the round-off error:



E.10 Finite Element extension to multiple dimensions

Q1(a) In order to calculate the shape functions, we need to find the area A_e of the element:

$$A_e = \frac{1}{2}\Big((x_2y_3 - x_3y_2) + (x_3y_1 - x_1y_3) + (x_1y_2 - x_2y_1)\Big)$$

$$= \frac{1}{2}\Big((1 \times 1 - 0.5 \times 1) + (\tfrac{1}{2} \times 0 - 0 \times 1) + (0 \times 1 - 1 \times 0)\Big) = \frac{1}{4}$$

Therefore the shape functions are:

$$N_1(x,y) = \frac{1}{2A_e}\Big((x_2y_3 - x_3y_2) + (y_2 - y_3)x + (x_3 - x_2)y\Big) = 1 - y$$

$$N_2(x,y) = \frac{1}{2A_e}\Big((x_3y_1 - x_1y_3) + (y_3 - y_1)x + (x_1 - x_3)y\Big) = 2x - y$$

$$N_3(x,y) = \frac{1}{2A_e}\Big((x_1y_2 - x_2y_1) + (y_1 - y_2)x + (x_2 - x_1)y\Big) = -2x + 2y$$

Q1(b) Substituting into $\tilde{u} = N_1u_1 + N_2u_2 + N_3u_3$:

$$\tilde{u}(\tfrac{1}{2}, \tfrac{3}{4}) = N_1(\tfrac{1}{2}, \tfrac{3}{4})u_1 + N_2(\tfrac{1}{2}, \tfrac{3}{4})u_2 + N_3(\tfrac{1}{2}, \tfrac{3}{4})u_3$$

$$= (1 - \tfrac{3}{4}) \times 0 + (2 \times \tfrac{1}{2} - \tfrac{3}{4}) \times \tfrac{1}{2} + (-2 \times \tfrac{1}{2} + 2 \times \tfrac{3}{4}) \times 1$$
$$= 0 + \tfrac{1}{8} + \tfrac{2}{4} = \tfrac{5}{8}$$

Q1(c) We'll need the gradients of $\tilde{u}$ in x and y:

$$\frac{\partial \tilde{u}}{\partial x} = \frac{\partial N_1}{\partial x} u_1 + \frac{\partial N_2}{\partial x} u_2 + \frac{\partial N_3}{\partial x} u_3 = 2u_2 - 2u_3$$
$$\frac{\partial \tilde{u}}{\partial y} = \frac{\partial N_1}{\partial y} u_1 + \frac{\partial N_2}{\partial y} u_2 + \frac{\partial N_3}{\partial y} u_3 = -u_1 - u_2 + 2u_3$$

Choosing $N_i = N_1$ first:

$$\iint_{\triangle_{123}} \Big((2u_2 - 2u_3) \times 0 + (-u_1 - u_2 + 2u_3) \times (-1) + \rho N_1 \Big) \, dxdy$$
$$= -\iint_{\triangle_{123}} (-u_1 - u_2 + 2u_3) \, dxdy + \iint_{\triangle_{123}} \rho N_1 \, dxdy$$
$$= (u_1 + u_2 - 2u_3) A_e + \rho \frac{A_e}{3} = 0$$
$$\implies u_1 + u_2 - 2u_3 = -\frac{\rho}{3}$$

Next for $N_i = N_2$:

$$\iint_{\triangle_{123}} \Big((2u_2 - 2u_3) \times 2 + (-u_1 - u_2 + 2u_3) \times (-1) + \rho N_2 \Big) \, dxdy$$
$$= \iint_{\triangle_{123}} (u_1 + 5u_2 - 6u_3) \, dxdy + \iint_{\triangle_{123}} \rho N_2 \, dxdy$$
$$= (u_1 + 5u_2 - 6u_3) A_e + \rho \frac{A_e}{3} = 0$$
$$\implies u_1 + 5u_2 - 6u_3 = -\frac{\rho}{3}$$

Lastly for $N_i = N_3$:

$$\iint_{\triangle_{123}} \Big((2u_2 - 2u_3) \times (-2) + (-u_1 - u_2 + 2u_3) \times 2 + \rho N_3 \Big) \, dxdy$$
$$= \iint_{\triangle_{123}} (-2u_1 - 6u_2 + 8u_3) \, dxdy + \iint_{\triangle_{123}} \rho N_3 \, dxdy$$
$$= (-2u_1 - 6u_2 + 8u_3) A_e + \rho \frac{A_e}{3} = 0$$
$$\implies -2u_1 - 6u_2 + 8u_3 = -\frac{\rho}{3}$$

Combining these three equations form the element equations:

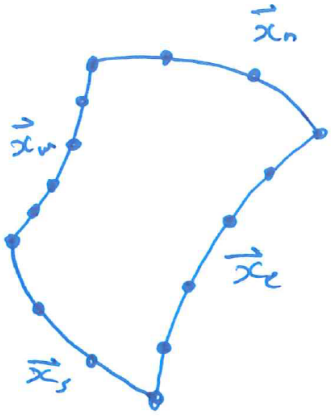
$$\begin{bmatrix} 1 & 1 & -2 \\ 1 & 5 & -6 \\ -2 & -6 & 8 \end{bmatrix} \begin{Bmatrix} u_1 \\ u_2 \\ u_3 \end{Bmatrix} = \begin{Bmatrix} -\frac{\rho}{3} \\ -\frac{\rho}{3} \\ -\frac{\rho}{3} \end{Bmatrix}$$

Q2(a) Algebraic grid generation:

1. Distribute nodes along four boundaries, where each opposite boundary has the same number of nodes, e.g. six on $\vec{x}_w$ and $\vec{x}_e$, and four on $\vec{x}_n$ and $\vec{x}_s$.

2. Nodes within the boundaries are located by linearly interpolating between the boundaries (see lecture notes for equations).

3. Coordinates are defined only in terms of the boundary points.

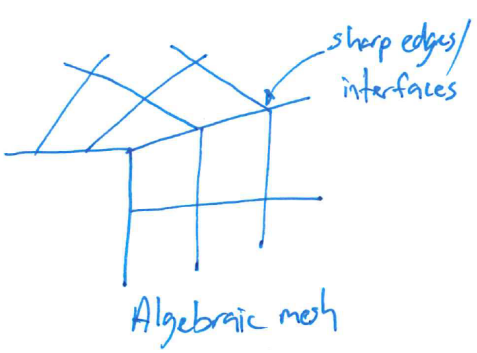


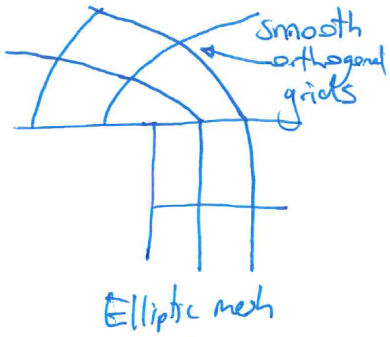
Elliptic grid generation: Solve the Laplace equation in the "physical" domain:

$$\xi_{xx} + \xi_{yy} = 0 \quad \text{and} \quad \eta_{xx} + \eta_{yy} = 0$$

but as that would require an existing mesh, instead, solve these equations in the "logical" domain.

Q2(b) We can see below that the algebraic grid generation method often creates sharp interfaces around corners.





E.11 Consistency, stability and convergence

Q1 Perform the usual Taylor series expansion about $T(x_i, t_n)$:

$$T(x_i, t_{n+1}) = T(x_i, t_n) + \Delta t \left.\frac{\partial T}{\partial t}\right|_{x_i,t_n} + \frac{\Delta t^2}{2} \left.\frac{\partial^2 T}{\partial t^2}\right|_{x_i,t_n} + \frac{\Delta t^3}{6} \left.\frac{\partial^3 T}{\partial t^3}\right|_{x_i,t_n} + \mathcal{O}(\Delta t^4)$$

$$T(x_i, t_{n-1}) = T(x_i, t_n) - \Delta t \left.\frac{\partial T}{\partial t}\right|_{x_i,t_n} + \frac{\Delta t^2}{2} \left.\frac{\partial^2 T}{\partial t^2}\right|_{x_i,t_n} - \frac{\Delta t^3}{6} \left.\frac{\partial^3 T}{\partial t^3}\right|_{x_i,t_n} + \mathcal{O}(\Delta t^4)$$

$$T(x_{i+1}, t_n) = T(x_i, t_n) + \Delta x \left.\frac{\partial T}{\partial x}\right|_{x_i,t_n} + \frac{\Delta x^2}{2} \left.\frac{\partial^2 T}{\partial x^2}\right|_{x_i,t_n} + \frac{\Delta x^3}{6} \left.\frac{\partial^3 T}{\partial x^3}\right|_{x_i,t_n} + \mathcal{O}(\Delta x^4)$$

$$T(x_{i-1}, t_n) = T(x_i, t_n) - \Delta x \left.\frac{\partial T}{\partial x}\right|_{x_i,t_n} + \frac{\Delta x^2}{2} \left.\frac{\partial^2 T}{\partial x^2}\right|_{x_i,t_n} - \frac{\Delta x^3}{6} \left.\frac{\partial^3 T}{\partial x^3}\right|_{x_i,t_n} + \mathcal{O}(\Delta x^4)$$

We know:

$$\tau_n = \frac{T(x_i, t_{n+1}) - T(x_i, t_{n-1})}{2\Delta t}$$
$$-\alpha \frac{T(x_{i+1}, t_n) - (T(x_i, t_{n+1}) + T(x_i, t_{n-1})) + T(x_{i-1}, t_n)}{\Delta x^2}$$

For the time derivative:

$$\left.\frac{\partial T}{\partial t}\right|_{x_i,t_n} + \frac{1}{6} \left.\frac{\partial^3 T}{\partial t^3}\right|_{x_i,t_n} \Delta t^2 + \mathcal{O}(\Delta t^3)$$

For the spatial derivative:

$$\frac{\alpha}{\Delta x^2} \left(\Delta x^2 \left.\frac{\partial^2 T}{\partial x^2}\right|_{x_i,t_n} - \Delta t^2 \left.\frac{\partial^2 T}{\partial t^2}\right|_{x_i,t_n} + \mathcal{O}(\Delta t^4) + \mathcal{O}(\Delta x^4) \right)$$

$$\implies \tau_n = \left.\frac{\partial T}{\partial t}\right|_{x_i,t_n} - \alpha \left.\frac{\partial^2 T}{\partial x^2}\right|_{x_i,t_n} + \alpha \frac{\Delta t^2}{\Delta x^2} \left.\frac{\partial^2 T}{\partial t^2}\right|_{x_i,t_n}$$
$$+\mathcal{O}(\Delta t^2) + \mathcal{O}\left(\frac{\Delta t^4}{\Delta x^2}\right) + \mathcal{O}(\Delta x^2)$$

Q2 The difference between BTCS and FTCS is that the right hand side terms are evaluated at the next time step, i.e. the backward Euler (implicit) method. As mentioned earlier, the solution error ε_i^n propagates in a similar fashion to the discretised equation:

$$\frac{\varepsilon_i^{n+1} - \varepsilon_i^n}{\Delta t} - \alpha \frac{\varepsilon_{i+1}^{n+1} - 2\varepsilon_i^{n+1} + \varepsilon_{i-1}^{n+1}}{\Delta x^2} = 0$$

and we showed that the next error depends on the current error:

$$\varepsilon_i^{n+1} = \gamma \varepsilon_i^n$$

substituting from above:

$$(\gamma - 1)\varepsilon_i^n = \lambda\gamma(e^{Ik\Delta x} - 2 + e^{-Ik\Delta x})\varepsilon_i^n$$

using the trig. rule $e^{Ik\Delta x} + e^{-Ik\Delta x} = 2\cos(k\Delta x)$:

$$\implies \gamma = \frac{1}{1 - 2\lambda(\cos(k\Delta x) - 1)}$$

Since $\lambda > 0$ we see that $0 \leqslant \gamma \leqslant 1 \implies$ the stability criterion $|\gamma| \leqslant 1$ is always satisfied. Note: using the double angle formula $\cos(k\Delta x) = 1 - 2\sin^2(\frac{k\Delta x}{2})$ has a more obvious conclusion:

$$\gamma = \frac{1}{1 + 4\lambda\sin^2(\frac{k\Delta x}{2})}$$

E.12 Optimisation methods

Q1(a) We can see that from choosing two points, e.g. $(-1, 2)$ and $(1, 2)$, and drawing a line connecting these two points that the epigraph is not a convex set (through the hill).

The Hessian, as calculated with the "hessian" function in sympy:

$$\nabla^2\mathcal{F}(\vec{\boldsymbol{x}}) = \begin{bmatrix} 8bx_1^2 - 4b(-x_1^2 + x_2) + 2 & -4bx_1 \\ -4bx_1 & 2b \end{bmatrix}$$

which is not positive semi-definite (checked by examining the eigenvalues), and is therefore a non-convex function.

Q1(b) A step factor of $h = 1$ is too large and the next step overshoots, leading to divergence. With an $h = 10^{-3}$ the solution converges to the optimal set of variables, the minimum at $(1, 1)$, in 17 563 iterations.

Q1(c) The procedure now only requires 5980 iterations to converge. This lower number of iterations was achieved because now the step size factor, h, is always at least as small as 10^{-3}. The line search method enforces the convergence, while having relatively larger steps.